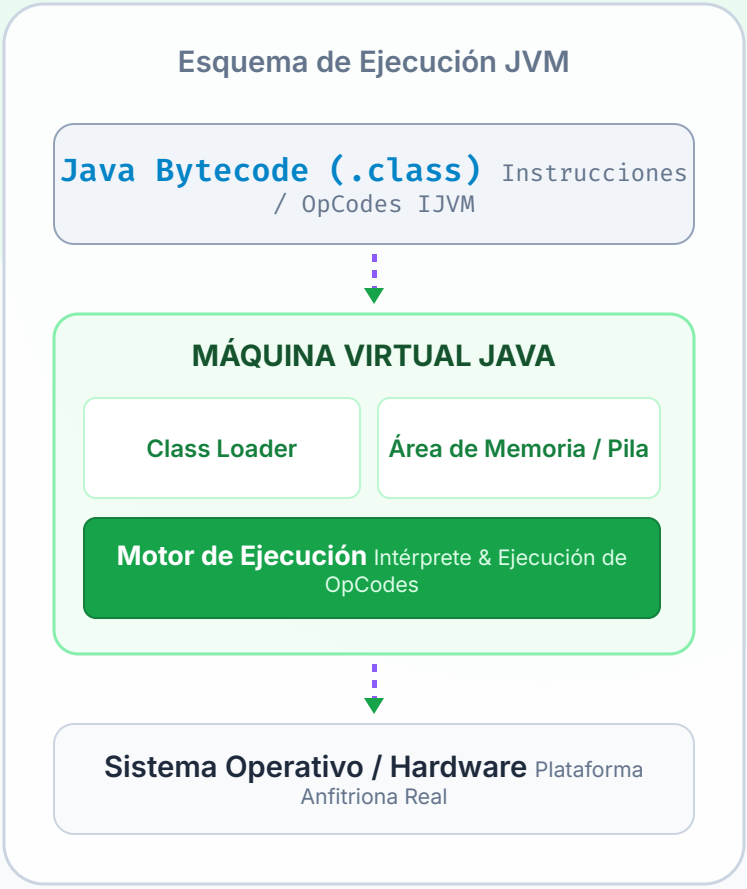
Fundamentos de Arquitectura

# Java Virtual Machine

## Arquitectura Básica

Modelado, ejecución de bytecode y principios estructurales de la plataforma lJVM.



# Java Virtual Machine

## Consideraciones

- Es una máquina que solo maneja **enteros**
  - IJVM (Integer Java Virtual Machine)

- No se contemplan operaciones de E/S, **no corresponden a la máquina virtual**
  - Ej: System.out.println()

- La instrucción ISA (assembler de JVM) INVOKEVIRTUAL (CALL), está simplificada para que solo pueda invocar métodos de su propio objeto
  - Por lo tanto convierte a esta versión de Java en un lenguaje **no orientado a objetos**, termina siendo un C simplificado.

- Máquina virtual basada en pila

# Java Virtual Machine

## Ejemplo de programa

| Fragmento Java | | Assembler IJVM | Programa IJVM |
|---|---|---|---|
| `i=j+k;` | 1 | `ILOAD j  // i=j+k` | `0x15 0x02` |
| `if(i==3)` | 2 | `ILOAD k` | `0x15 0x03` |
| `  k=0;` | 3 | `IADD` | `0x60` |
| `else` | 4 | `ISTORE i` | `0x36 0x01` |
| `  j=j-1;` | 5 | `ILOAD i  // if(i<3)` | `0x15 0x01` |
| | 6 | `BIPUSH 3` | `0x10 0x03` |
| | 7 | `IF_ICMPEQ L1` | `0x9F 0x00 0x0D` |
| | 8 | `ILOAD j  // j=j-1` | `0x15 0x02` |
| | 9 | `BIPUSH 1` | `0x10 0x01` |
| | 10 | `ISUB` | `0x64` |
| | 11 | `ISTORE j` | `0x36 0x02` |
| | 12 | `GOTO L2` | `0xA7 0x00 0x07` |
| | 13 | `L1: BIPUSH 0  // k=0` | `0x10 0x00` |
| | 14 | `ISTORE k` | `0x36 0x03` |
| | 15 | `L2:` | |

Fig 4-14

# Java Virtual Machine

## Ejemplo de programa

| Fragmento Java | Assembler IJVM | Programa IJVM |
|---|---|---|
| `i=j+k;` | `1 ILOAD j // i=j+k` | `0x15 0x02` |
| `if(i==3)` | `2 ILOAD k` | `0x15 0x03` |
| `  k=0;` | `3 IADD` | `0x60` |
| `else` | `4 ISTORE i` | `0x36 0x01` |
| `  j=j-1;` | `5 ILOAD i // if(i<3)` | `0x15 0x01` |
| | `6 BIPUSH 3` | `0x10 0x03` |
| | `7 IF_ICMPEQ L1` | `0x9F 0x00 0x0D` |
| | `8 ILOAD j // j=j-1` | `0x15 0x02` |
| | `9 BIPUSH 1` | `0x10 0x01` |
| | `10 ISUB` | `0x64` |
| | `11 ISTORE j` | `0x36 0x02` |
| | `12 GOTO L2` | `0xA7 0x00 0x07` |
| | `13 L1: BIPUSH 0 // k=0` | `0x10 0x00` |
| | `14 ISTORE k` | `0x36 0x03` |
| | `15 L2:` | |

Fig 4-14

# Java Virtual Machine

## Ejemplo de programa

| Fragmento Java | Assembler IJVM | Programa IJVM |
|---|---|---|
| `i=j+k;` | `1 ILOAD j // i=j+k` | `0x15 0x02` |
| `if(i==3)` | `2 ILOAD k` | `0x15 0x03` |
| `  k=0;` | `3 IADD` | `0x60` |
| `else` | `4 ISTORE i` | `0x36 0x01` |
| `  j=j-1;` | `5 ILOAD i // if(i<3)` | `0x15 0x01` |
| | `6 BIPUSH 3` | `0x10 0x03` |
| | `7 IF_ICMPEQ L1` | `0x9F 0x00 0x0D` |
| | `8 ILOAD j // j=j-1` | `0x15 0x02` |
| | `9 BIPUSH 1` | `0x10 0x01` |
| | `10 ISUB` | `0x64` |
| | `11 ISTORE j` | `0x36 0x02` |
| | `12 GOTO L2` | `0xA7 0x00 0x07` |
| | `13 L1: BIPUSH 0 // k=0` | `0x10 0x00` |
| | `14 ISTORE k` | `0x36 0x03` |
| | `15 L2:` | |

Fig 4-14

# Java Virtual Machine

## Ejemplo de programa

| Fragmento Java | Assembler IJVM | Programa IJVM |
|---|---|---|
| `i=j+k;` | `ILOAD j  // i=j+k` | `0x15 0x02` |
| `if(i==3)` | `ILOAD k` | `0x15 0x03` |
| `  k=0;` | `IADD` | `0x60` |
| `else` | `ISTORE i` | `0x36 0x01` |
| `  j=j-1;` | `ILOAD i  // if(i<3)` | `0x15 0x01` |
| | `BIPUSH 3` | `0x10 0x03` |
| | `IF_ICMPEQ L1` | `0x9F 0x00 0x0D` |
| | `ILOAD j  // j=j-1` | `0x15 0x02` |
| | `BIPUSH 1` | `0x10 0x01` |
| | `ISUB` | `0x64` |
| | `ISTORE j` | `0x36 0x02` |
| | `GOTO L2` | `0xA7 0x00 0x07` |
| | `L1: BIPUSH 0  // k=0` | `0x10 0x00` |
| | `ISTORE k` | `0x36 0x03` |
| | `L2:` | |

Fig 4-14

# Java Virtual Machine

## Ejemplo de programa

| Fragmento Java | | Assembler IJVM | | Programa IJVM |
|---|---|---|---|---|
| `i=j+k;` | 1 | `ILOAD j` | `// i=j+k` | `0x15 0x02` |
| `if(i==3)` | 2 | `ILOAD k` | | `0x15 0x03` |
| `  k=0;` | 3 | `IADD` | | `0x60` |
| `else` | 4 | `ISTORE i` | | `0x36 0x01` |
| `  j=j-1;` | 5 | `ILOAD i` | `// if(i<3)` | `0x15 0x01` |
| | 6 | `BIPUSH 3` | | `0x10 0x03` |
| | 7 | `IF_ICMPEQ L1` | | `0x9F 0x00 0x0D` |
| | 8 | `ILOAD j` | `// j=j-1` | `0x15 0x02` |
| | 9 | `BIPUSH 1` | | `0x10 0x01` |
| | 10 | `ISUB` | | `0x64` |
| | 11 | `ISTORE j` | | `0x36 0x02` |
| | 12 | `GOTO L2` | | `0xA7 0x00 0x07` |
| | 13 | `L1: BIPUSH 0` | `// k=0` | `0x10 0x00` |
| | 14 | `ISTORE k` | | `0x36 0x03` |
| | 15 | `L2:` | | |

Fig 4-14

# Java Virtual Machine

## Ejemplo de programa

| Fragmento Java | | Assembler IJVM | | Programa IJVM |
|---|---|---|---|---|
| `i=j+k;` | 1 | `ILOAD j` | `// i=j+k` | `0x15 0x02` |
| `if(i==3)` | 2 | `ILOAD k` | | `0x15 0x03` |
| `  k=0;` | 3 | `IADD` | | `0x60` |
| `else` | 4 | `ISTORE i` | | `0x36 0x01` |
| `  j=j-1;` | 5 | `ILOAD i` | `// if(i<3)` | `0x15 0x01` |
| | 6 | `BIPUSH 3` | | `0x10 0x03` |
| | 7 | `IF_ICMPEQ L1` | | `0x9F 0x00 0x0D` |
| | 8 | `ILOAD j` | `// j=j-1` | `0x15 0x02` |
| | 9 | `BIPUSH 1` | | `0x10 0x01` |
| | 10 | `ISUB` | | `0x64` |
| | 11 | `ISTORE j` | | `0x36 0x02` |
| | 12 | `GOTO L2` | | `0xA7 0x00 0x07` |
| | 13 | `L1: BIPUSH 0` | `// k=0` | `0x10 0x00` |
| | 14 | `ISTORE k` | | `0x36 0x03` |
| | 15 | `L2:` | | |

Fig 4-14

# Java Virtual Machine

## Instrucciones Assembler IJVM (Macro-lenguaje)

| Hex | Mnemónico | Significado |
|---|---|---|
| 0x10 | BIPUSH *byte* | Pone “byte” a la pila |
| 0x59 | DUP | Copia la palabra superior en la pila y la pone en la pila |
| 0xA7 | GOTO *offset* | Salto incondicional |
| 0x60 | IADD | Saca dos palabras de la pila; pone su suma |
| 0x7E | IAND | Saca dos palabras de la pila; pone el AND booleano |
| 0x99 | IFEQ *offset* | Saca una palabra de la pila y salta si es cero |
| 0x9B | IFLT *offset* | Saca una palabra de la pila y salta si es menor que cero |
| 0x9F | IF_ICMPEQ *offset* | Saca dos palabras de la pila; salta si son iguales |
| 0x84 | IINC *varnum const* | Añade una constante a una variable local |
| 0x15 | ILOAD *varnum* | Pone la variable local a la pila |
| 0xB6 | INVOKEVIRTUAL *disp* | Invoca un método |
| 0x80 | IOR | Saca dos palabras de la pila; Pone el OR booleano |
| 0xAC | IRETURN | Retorna de un método con valor entero |
| 0x36 | ISTORE *varnum* | Saca una palabra de la pila y la almacena en una variable local |
| 0x64 | ISUB | Saca dos palabras de la pila; empuja su diferencia |
| 0x13 | LDC_W *index* | Empuja una constante del pool de constantes a la pila |
| 0x00 | NOP | No hace nada |
| 0x57 | POP | Elimina la palabra superior de la pila |
| 0x5F | SWAP | Intercambia las dos palabras superiores de la pila |
| 0xC4 | WIDE | Prefijo; la siguiente instrucción tiene un índice de 16 bits |

Los operandos *byte*, *const* y *varnum* son de **1 byte** y los operandos *disp*, *index* y *offset* son de **2 bytes**

Fig 4-11

# Java Virtual Machine

## Instrucciones Assembler IJVM (Macro-lenguaje)

| Hex | Mnemónico | Significado |
|---|---|---|
| 0x10 | BIPUSH *byte* | Pone “byte” a la pila |
| 0x59 | DUP | Copia la palabra superior en la pila y la pone en la pila |
| 0xA7 | GOTO *offset* | Salto incondicional |
| 0x60 | IADD | Saca dos palabras de la pila; pone su suma |
| 0x7E | IAND | Saca dos palabras de la pila; pone el AND booleano |
| 0x99 | IFEQ *offset* | Saca una palabra de la pila y salta si es cero |
| 0x9B | IFLT *offset* | Saca una palabra de la pila y salta si es menor que cero |
| 0x9F | IF_ICMPEQ *offset* | Saca dos palabras de la pila; salta si son iguales |
| 0x84 | IINC *varnum const* | Añade una constante a una variable local |
| 0x15 | ILOAD *varnum* | Pone la variable local a la pila |
| 0xB6 | INVOKEVIRTUAL *disp* | Invoca un método |
| 0x80 | IOR | Saca dos palabras de la pila; Pone el OR booleano |
| 0xAC | IRETURN | Retorna de un método con valor entero |
| 0x36 | ISTORE *varnum* | Saca una palabra de la pila y la almacena en una variable local |
| 0x64 | ISUB | Saca dos palabras de la pila; empuja su diferencia |
| 0x13 | LDC_W *index* | Empuja una constante del pool de constantes a la pila |
| 0x00 | NOP | No hace nada |
| 0x57 | POP | Elimina la palabra superior de la pila |
| 0x5F | SWAP | Intercambia las dos palabras superiores de la pila |
| 0xC4 | WIDE | Prefijo; la siguiente instrucción tiene un índice de 16 bits |

Los operandos *byte*, *const* y *varnum* son de **1 byte** y los operandos *disp*, *index* y *offset* son de **2 bytes**

Fig 4-11

# Java Virtual Machine

## Instrucciones Assembler IJVM (Macro-lenguaje)

| Hex | Mnemónico | Significado |
|---|---|---|
| 0x10 | BIPUSH *byte* | Pone “byte” a la pila |
| 0x59 | DUP | Copia la palabra superior en la pila y la pone en la pila |
| 0xA7 | GOTO *offset* | Salto incondicional |
| 0x60 | IADD | Saca dos palabras de la pila; pone su suma |
| 0x7E | IAND | Saca dos palabras de la pila; pone el AND booleano |
| 0x99 | IFEQ *offset* | Saca una palabra de la pila y salta si es cero |
| 0x9B | IFLT *offset* | Saca una palabra de la pila y salta si es menor que cero |
| 0x9F | IF_ICMPEQ *offset* | Saca dos palabras de la pila; salta si son iguales |
| 0x84 | IINC *varnum const* | Añade una constante a una variable local |
| 0x15 | ILOAD *varnum* | Pone la variable local a la pila |
| 0xB6 | INVOKEVIRTUAL *disp* | Invoca un método |
| 0x80 | IOR | Saca dos palabras de la pila; Pone el OR booleano |
| 0xAC | IRETURN | Retorna de un método con valor entero |
| 0x36 | ISTORE *varnum* | Saca una palabra de la pila y la almacena en una variable local |
| 0x64 | ISUB | Saca dos palabras de la pila; empuja su diferencia |
| 0x13 | LDC_W *index* | Empuja una constante del pool de constantes a la pila |
| 0x00 | NOP | No hace nada |
| 0x57 | POP | Elimina la palabra superior de la pila |
| 0x5F | SWAP | Intercambia las dos palabras superiores de la pila |
| 0xC4 | WIDE | Prefijo; la siguiente instrucción tiene un índice de 16 bits |

Los operandos *byte*, *const* y *varnum* son de **1 byte** y los operandos *disp*, *index* y *offset* son de **2 bytes**

Fig 4-11

# Java Virtual Machine

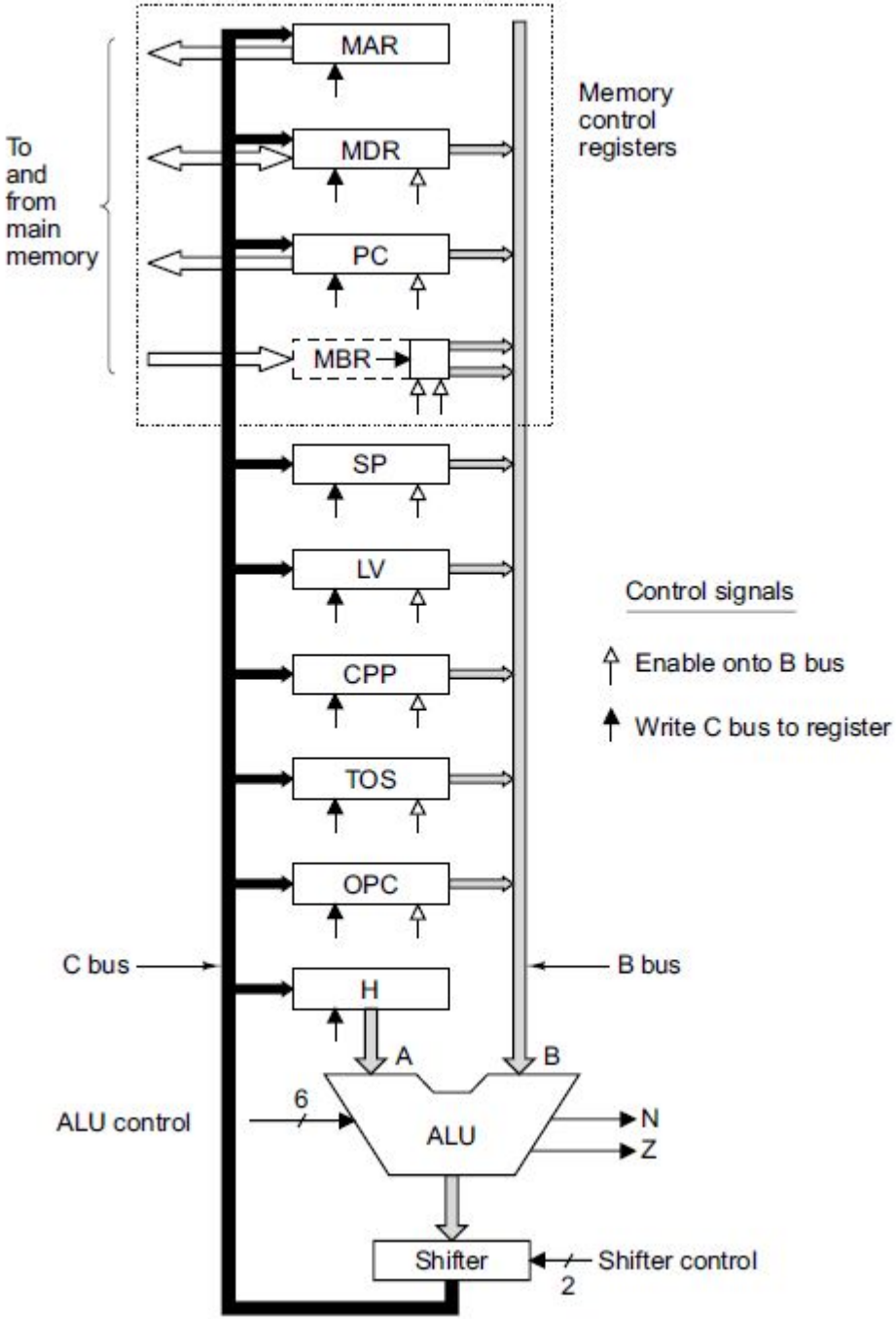
## Instrucciones Assembler IJVM (Macro-lenguaje)

| Hex | Mnemónico | Significado |
|---|---|---|
| 0x10 | BIPUSH *byte* | Pone “byte” a la pila |
| 0x59 | DUP | Copia la palabra superior en la pila y la pone en la pila |
| 0xA7 | GOTO *offset* | Salto incondicional |
| 0x60 | IADD | Saca dos palabras de la pila; pone su suma |
| 0x7E | IAND | Saca dos palabras de la pila; pone el AND booleano |
| 0x99 | IFEQ *offset* | Saca una palabra de la pila y salta si es cero |
| 0x9B | IFLT *offset* | Saca una palabra de la pila y salta si es menor que cero |
| 0x9F | IF_ICMPEQ *offset* | Saca dos palabras de la pila; salta si son iguales |
| 0x84 | IINC *varnum const* | Añade una constante a una variable local |
| 0x15 | ILOAD *varnum* | Pone la variable local a la pila |
| 0xB6 | INVOKEVIRTUAL *disp* | Invoca un método |
| 0x80 | IOR | Saca dos palabras de la pila; Pone el OR booleano |
| 0xAC | IRETURN | Retorna de un método con valor entero |
| 0x36 | ISTORE *varnum* | Saca una palabra de la pila y la almacena en una variable local |
| 0x64 | ISUB | Saca dos palabras de la pila; empuja su diferencia |
| 0x13 | LDC_W *index* | Empuja una constante del pool de constantes a la pila |
| 0x00 | NOP | No hace nada |
| 0x57 | POP | Elimina la palabra superior de la pila |
| 0x5F | SWAP | Intercambia las dos palabras superiores de la pila |
| 0xC4 | WIDE | Prefijo; la siguiente instrucción tiene un índice de 16 bits |

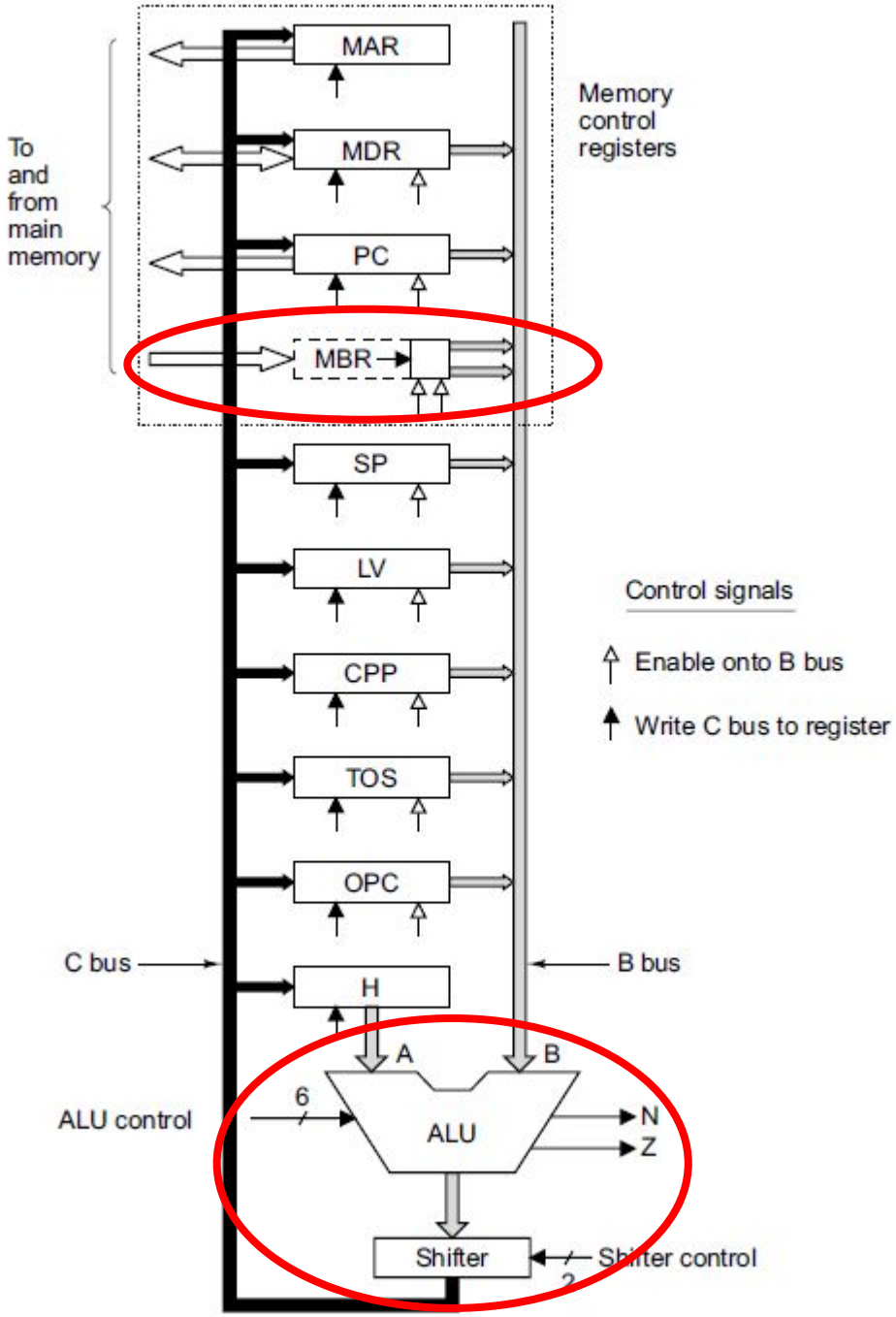
Los operandos *byte*, *const* y *varnum* son de **1 byte** y los operandos *disp*, *index* y *offset* son de **2 bytes**

Fig 4-11

# Flujo de datos de la JVM



# Flujo de datos de la JVM

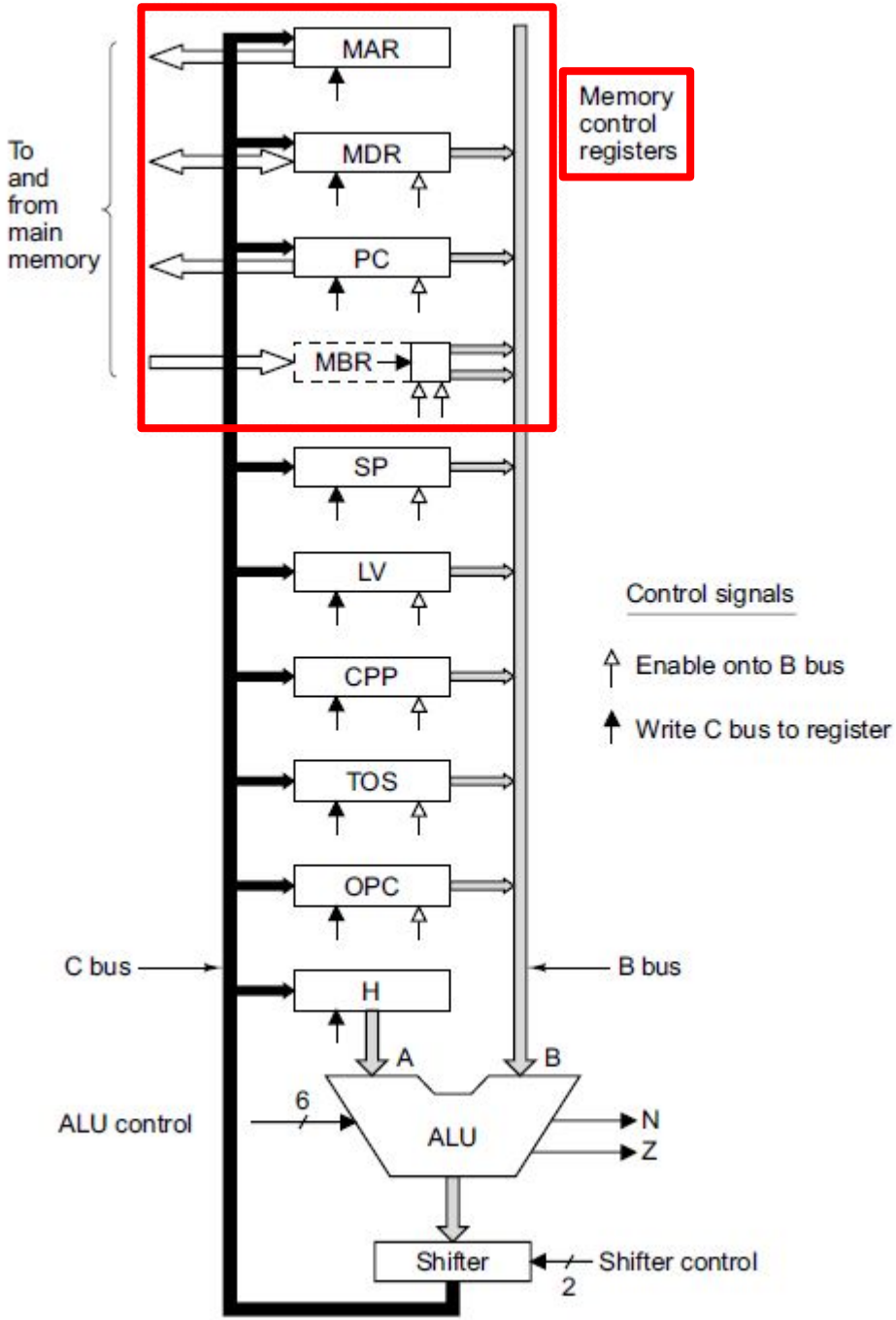


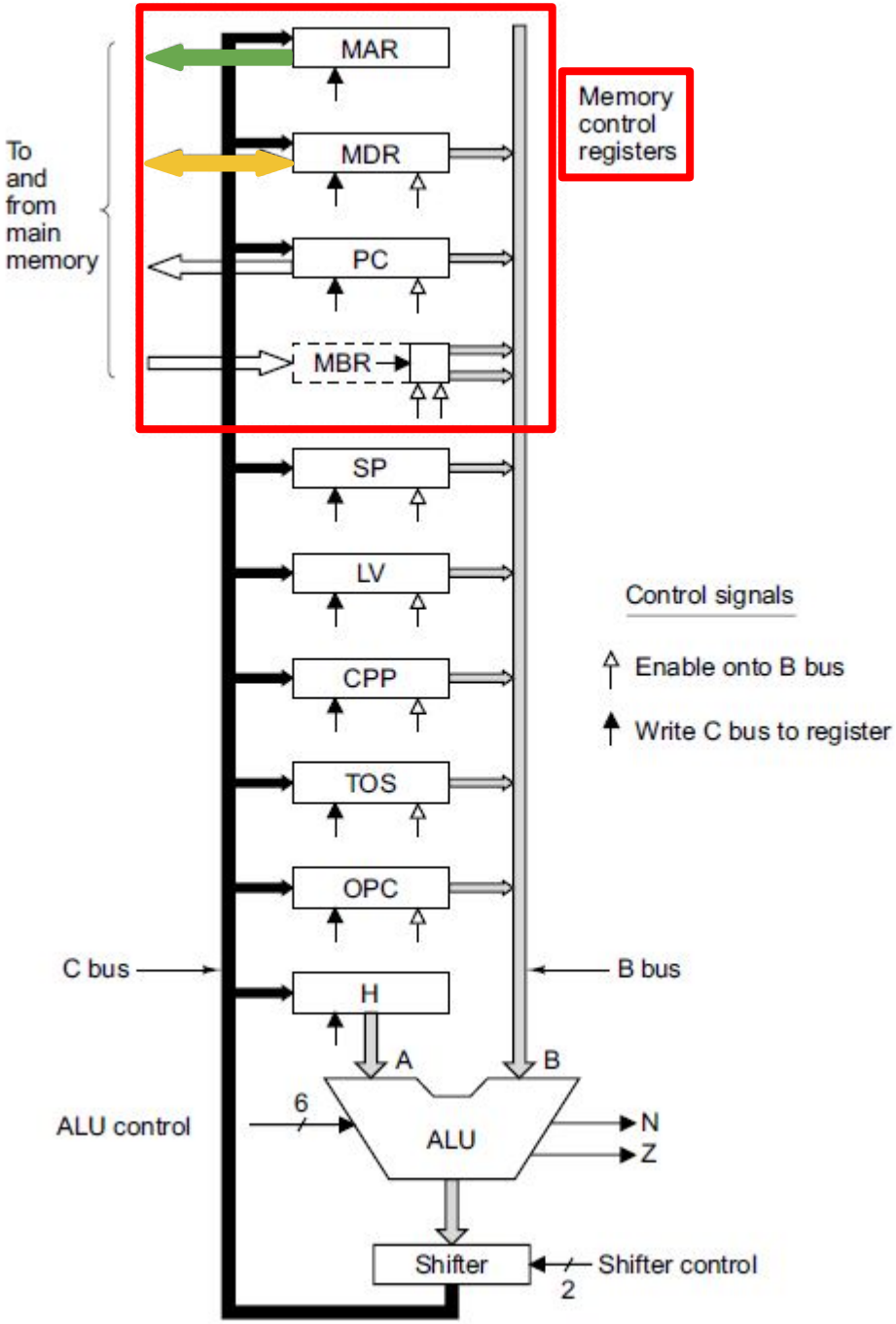

- Registros de 32 bits
- excepto **MBR** de 8 bits

- ALU y Shifter

# Flujo de datos de la JVM

- Registros que interactúan con la memoria RAM

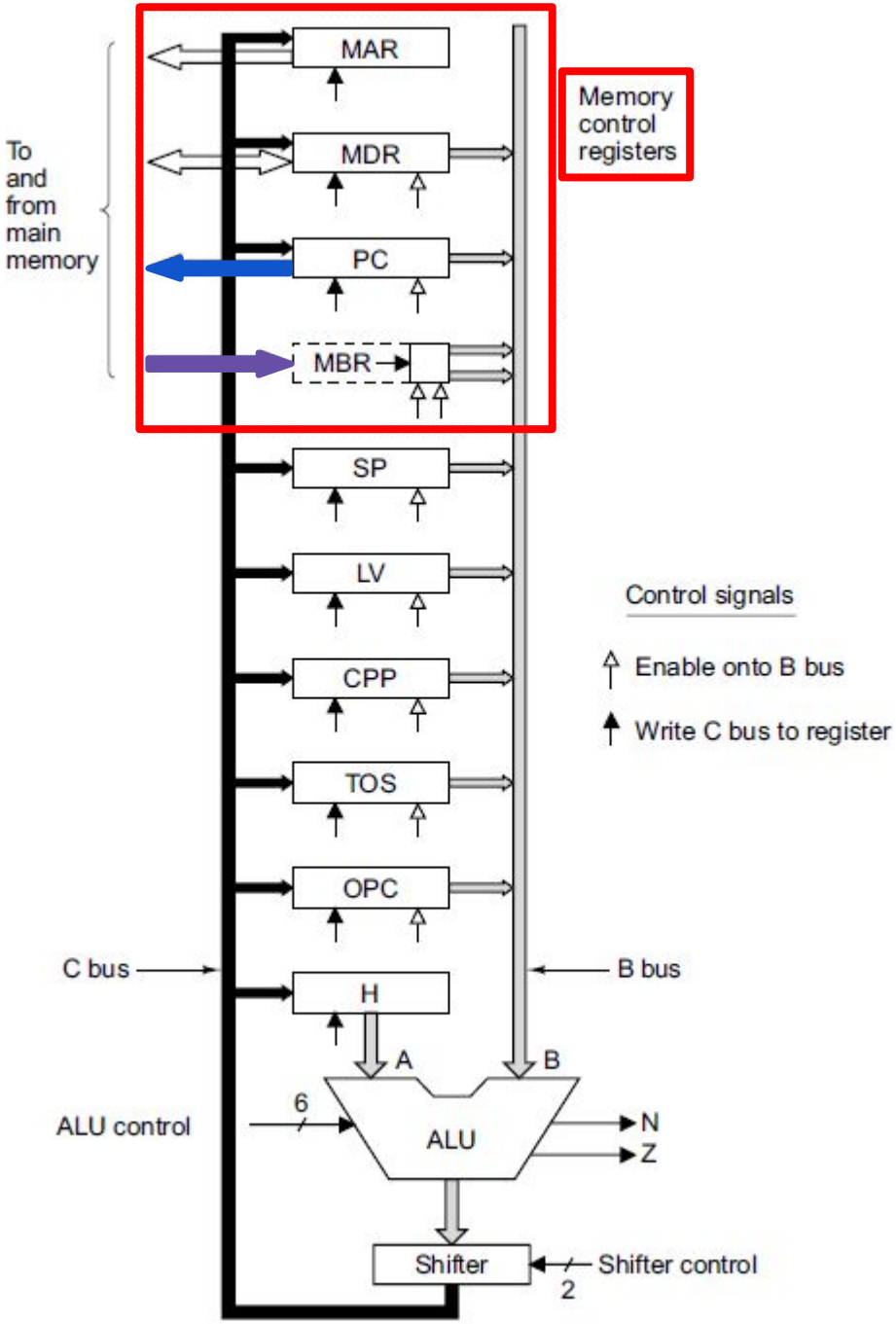





## Flujo de datos de la JVM

- MAR y MDR para R/W datos a memoria RAM

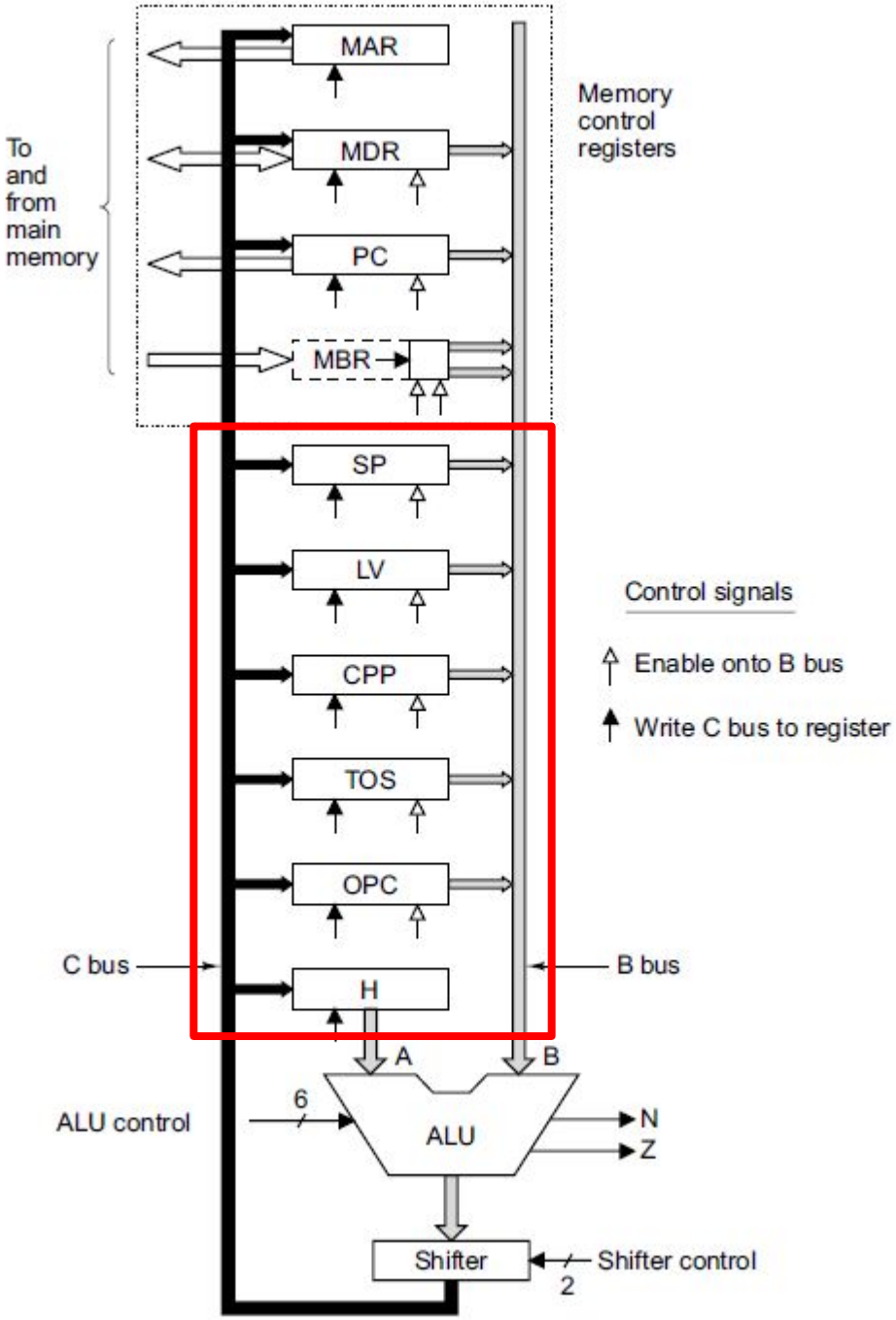
- **M**emory **A**ddress **R**egister
- **M**emory **D**ata **R**egister




## Flujo de datos de la JVM

- PC y MBR para Fetch instrucciones de la RAM

- **P**rogram **C**ounter
- **M**emory **B**uffer **R**egister




## Flujo de datos de la JVM

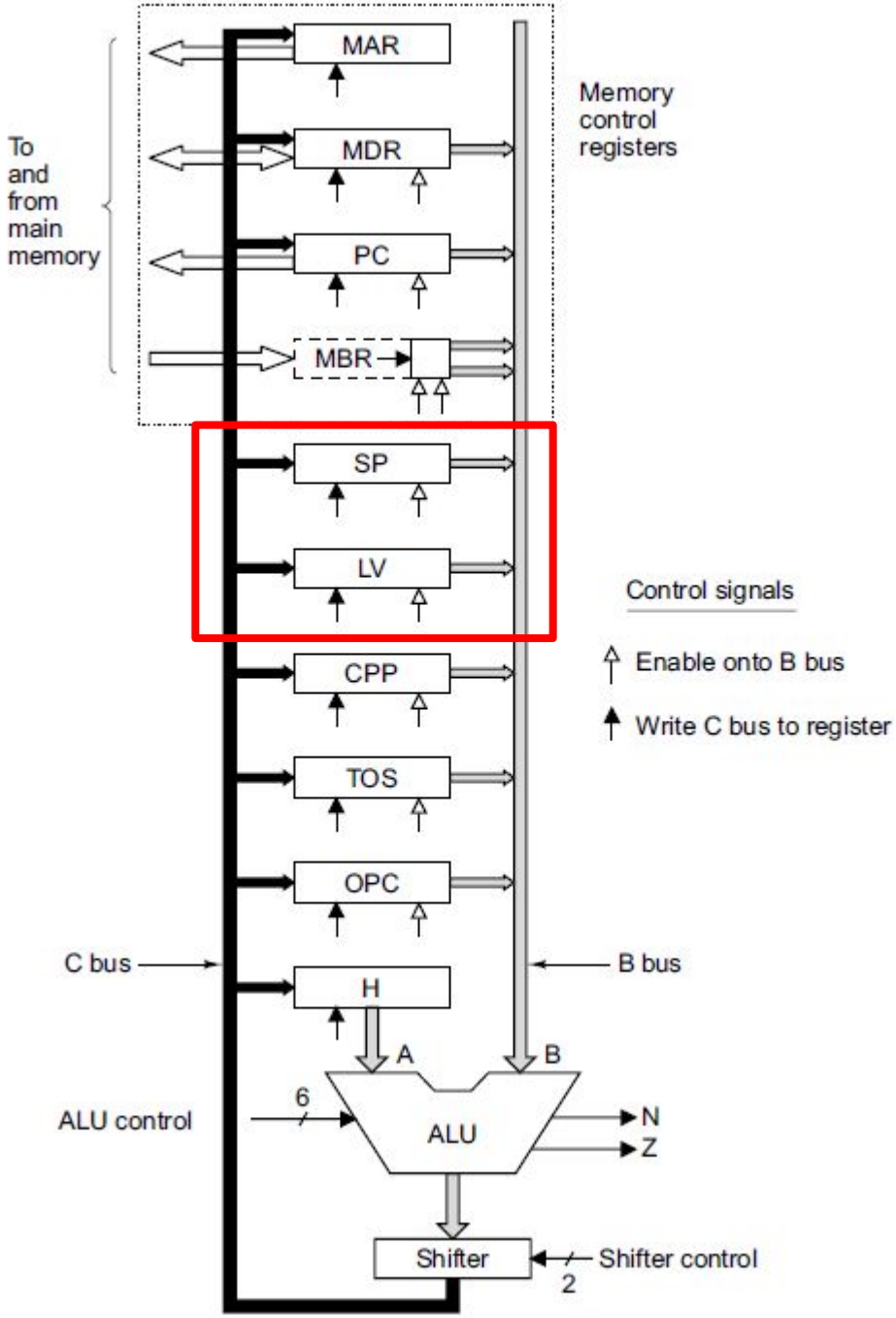
- Registros de usos específicos

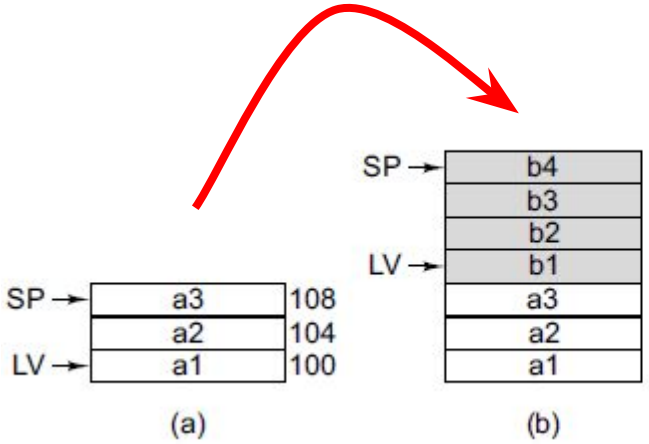
- Stack Pointer
- Local Variables
- Constant Pool Pointer
- Top Of Stack
- Old PC / Auxiliar
- Held

# Flujo de datos de la JVM

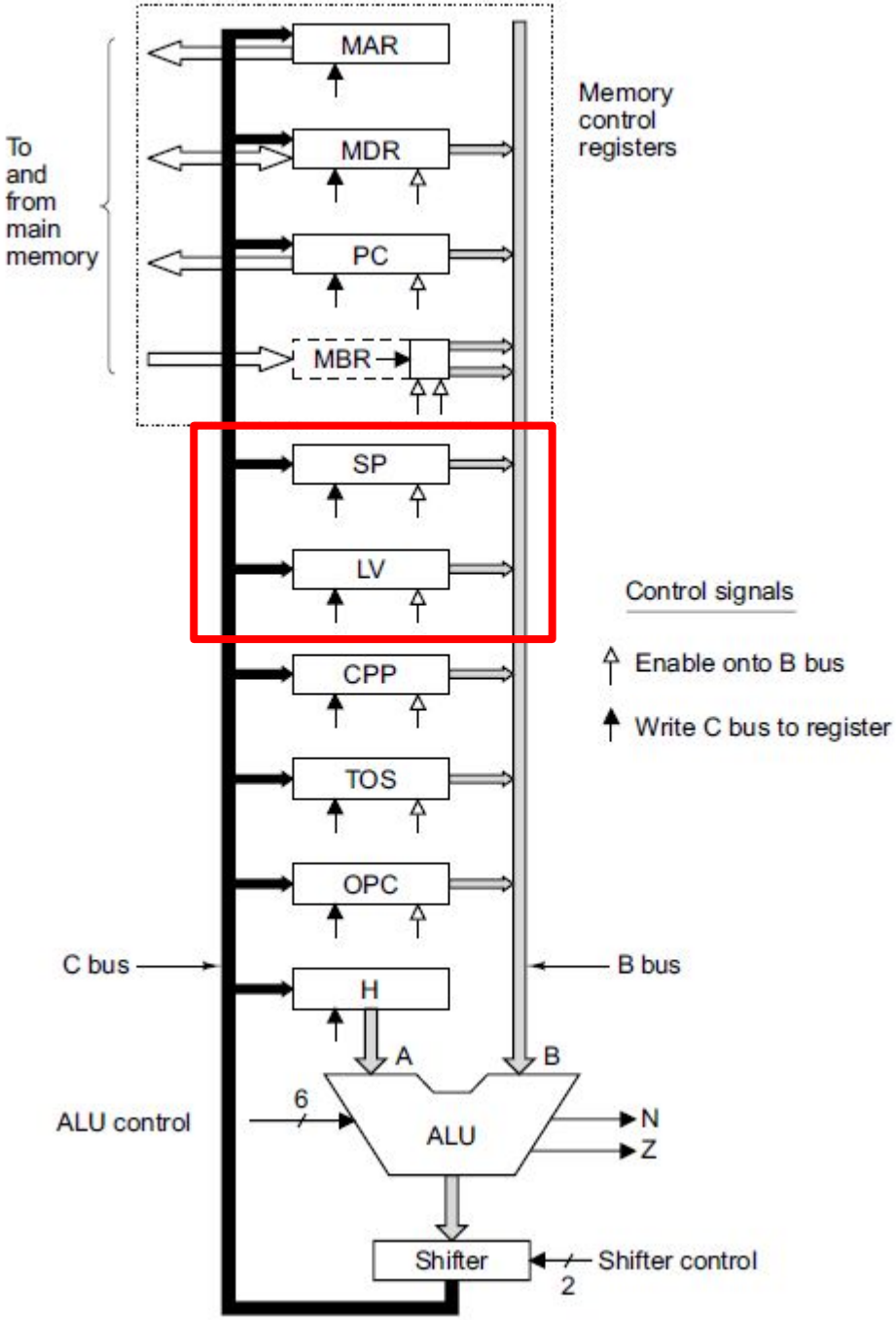
- Registros de usos específicos

- Stack Pointer
- Local Variables
- Constant Pool Pointer
- Top Of Stack
- Old PC / Auxiliar
- Held



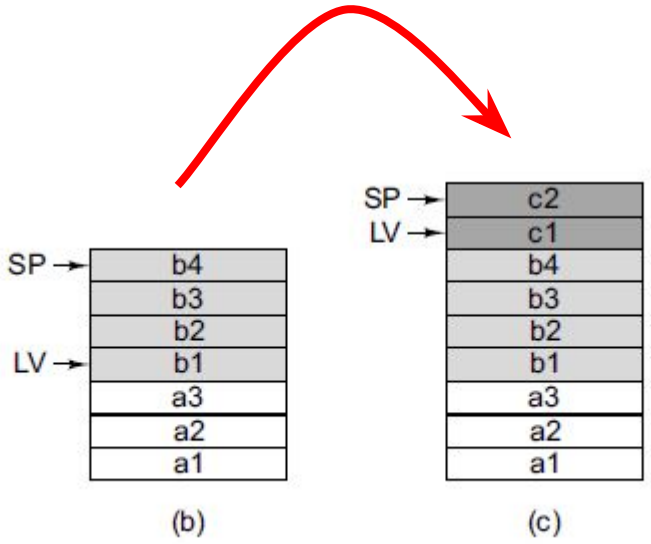

El proc. A invoca al proc. B




# Flujo de datos de la JVM

- Registros de usos específicos

- Stack Pointer
- Local Variables
- Constant Pool Pointer
- Top Of Stack
- Old PC / Auxiliar
- Held

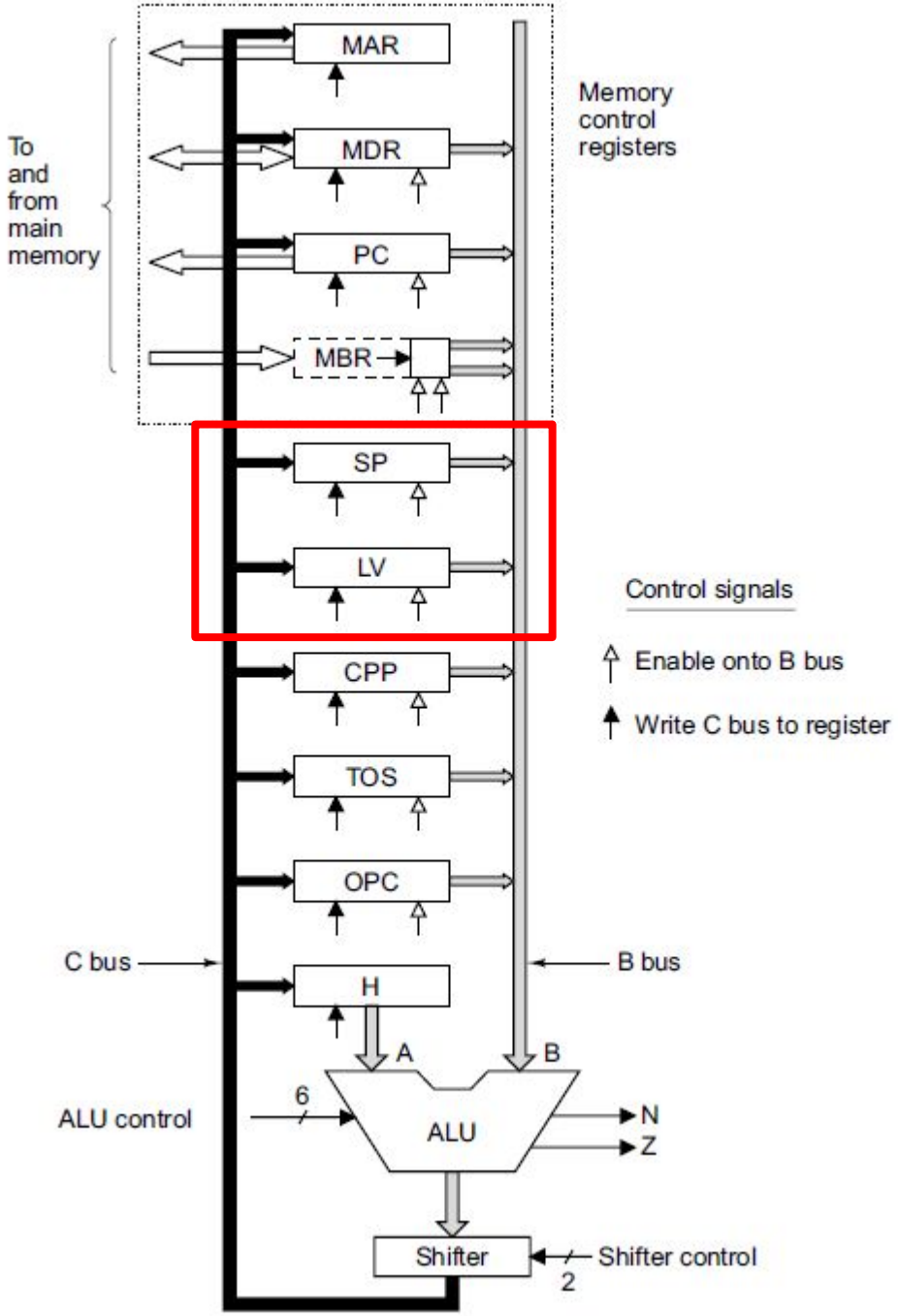


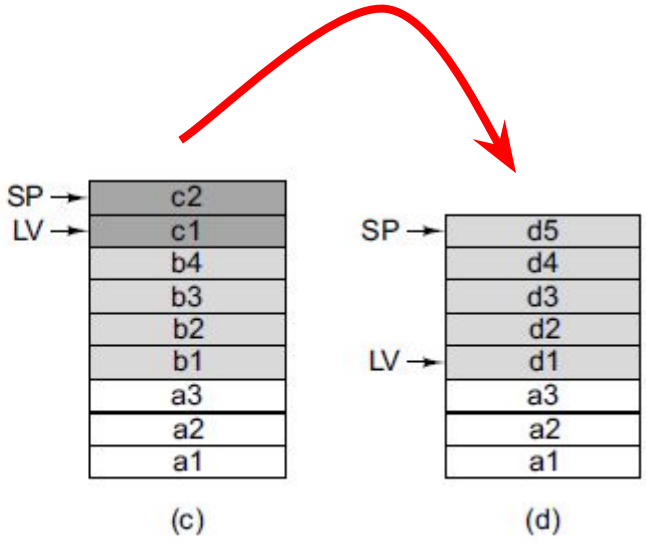

El proc. B invoca al proc. C

# Flujo de datos de la JVM

- Registros de usos específicos

- Stack Pointer
- Local Variables
- Constant Pool Pointer
- Top Of Stack
- Old PC / Auxiliar
- Held



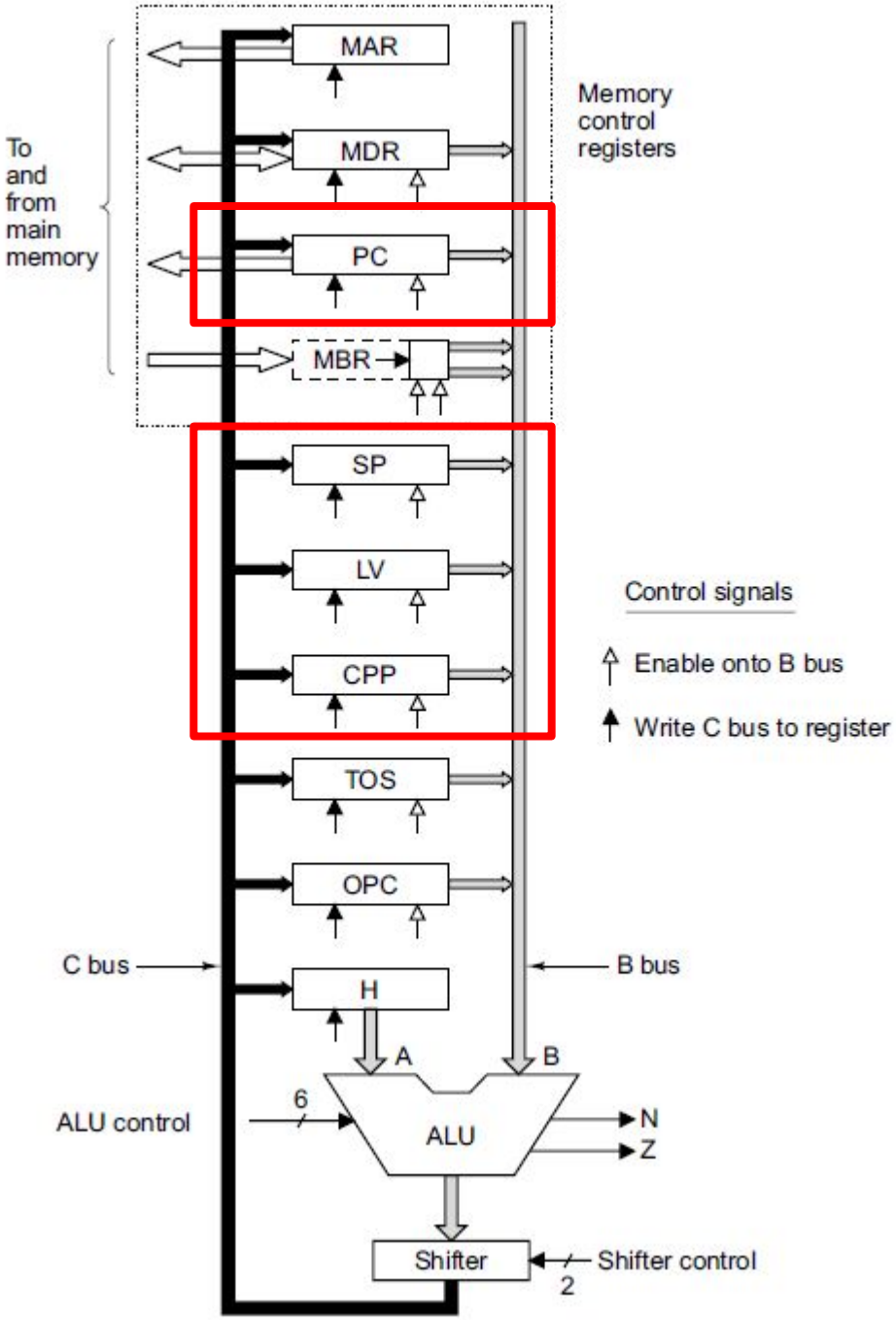



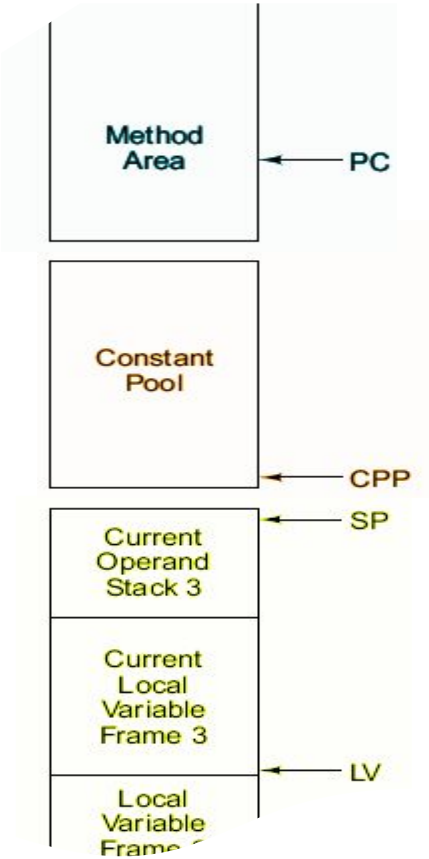

Retorna al proc. A e invoca al proc. D

# Flujo de datos de la JVM

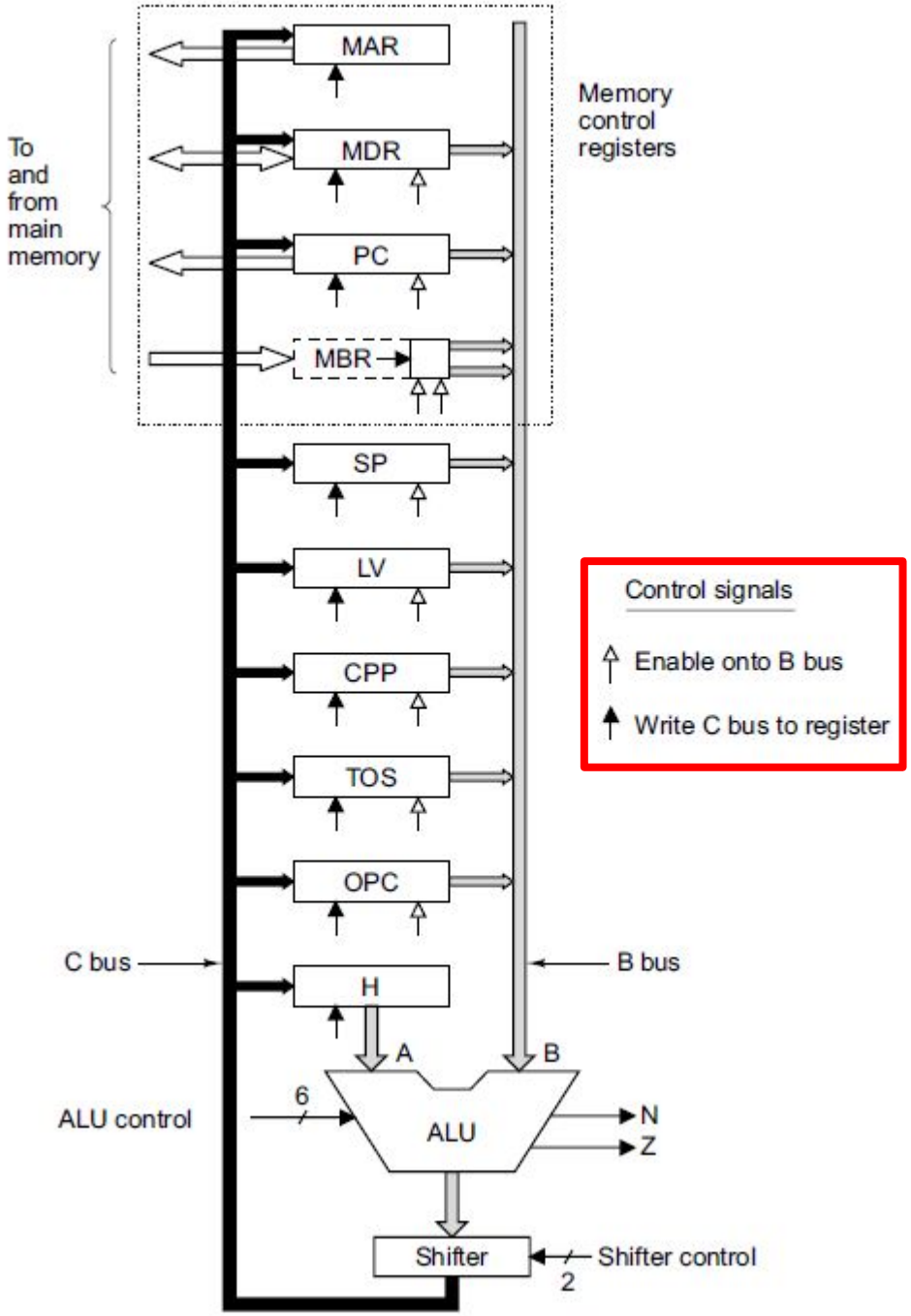
- Registros de usos específicos

- Stack Pointer
- Local Variables
- Constant Pool Pointer
- Top Of Stack
- Old PC / Auxiliar
- Held

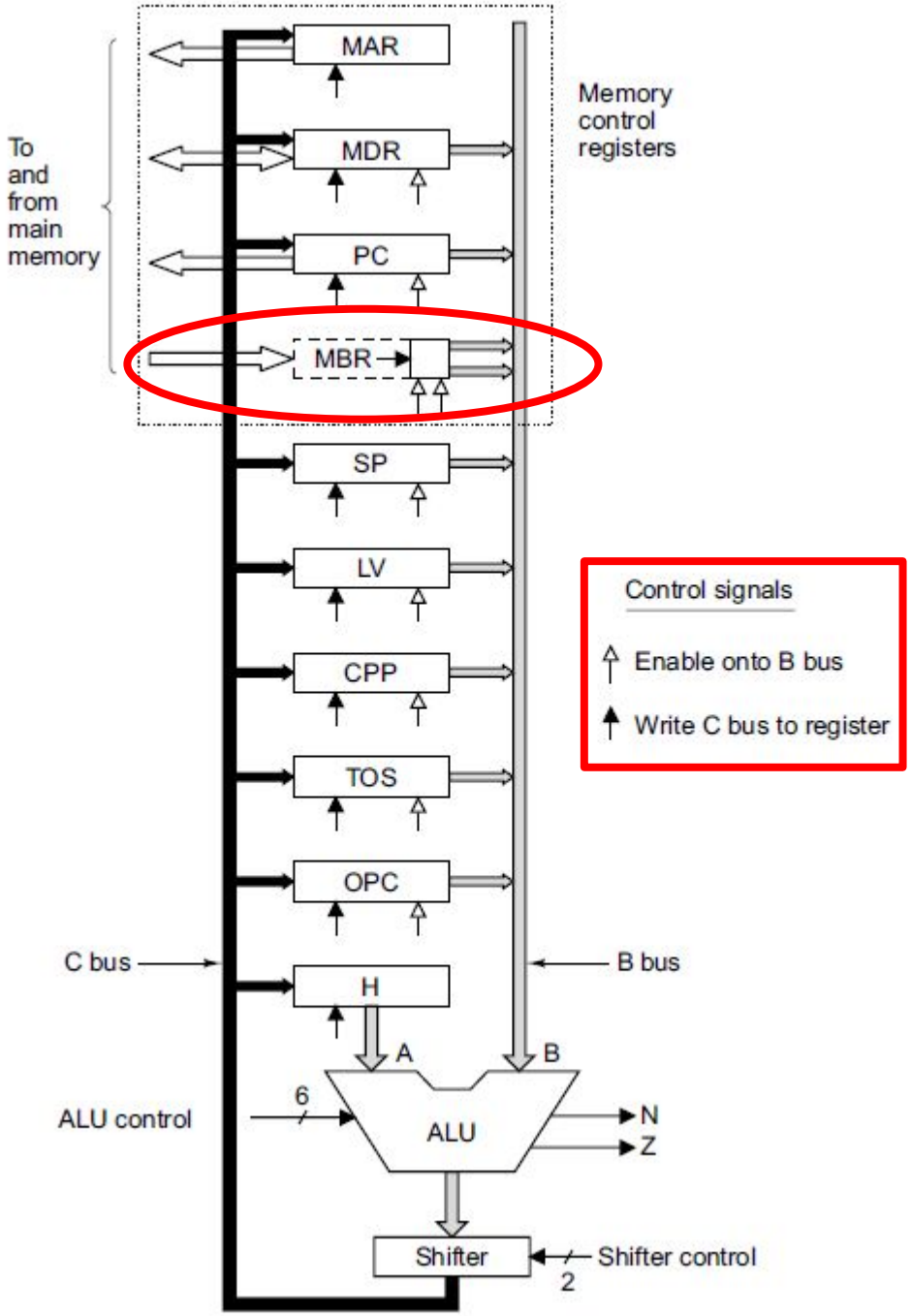


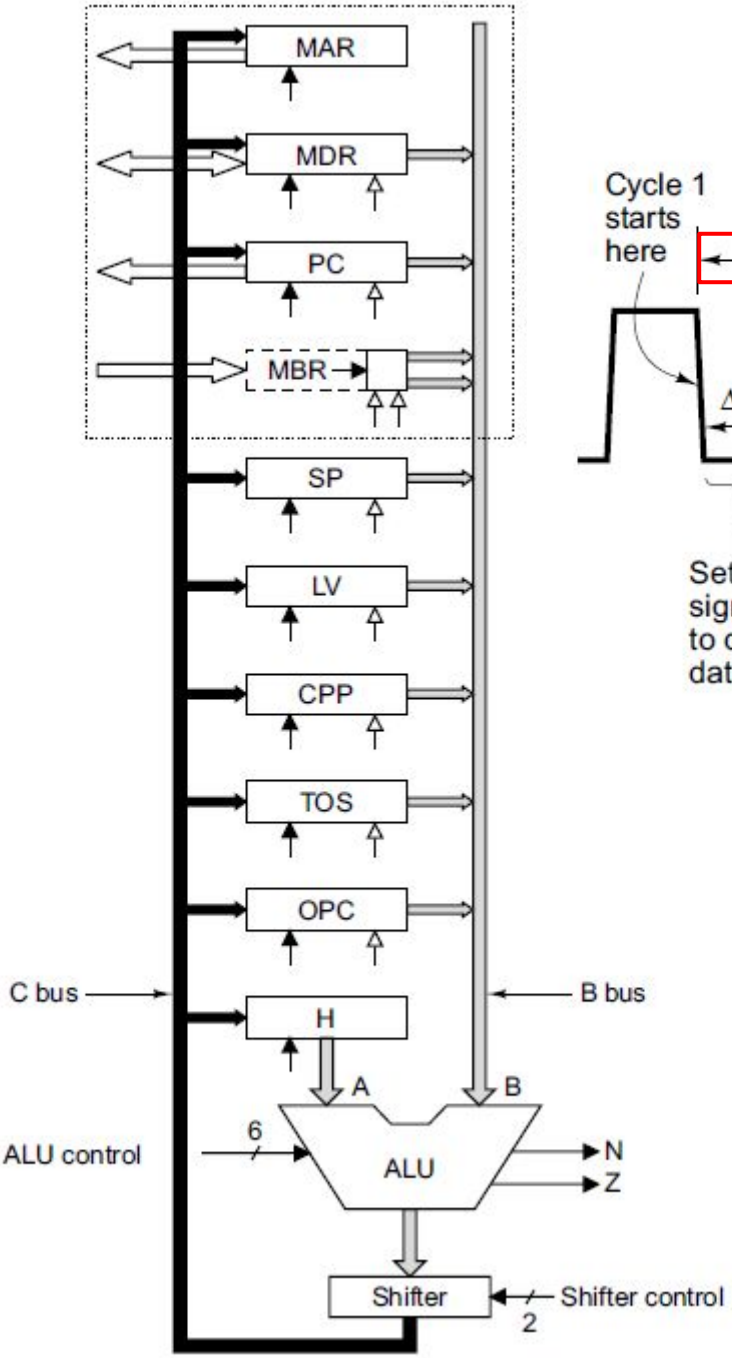
# Flujo de datos de la JVM

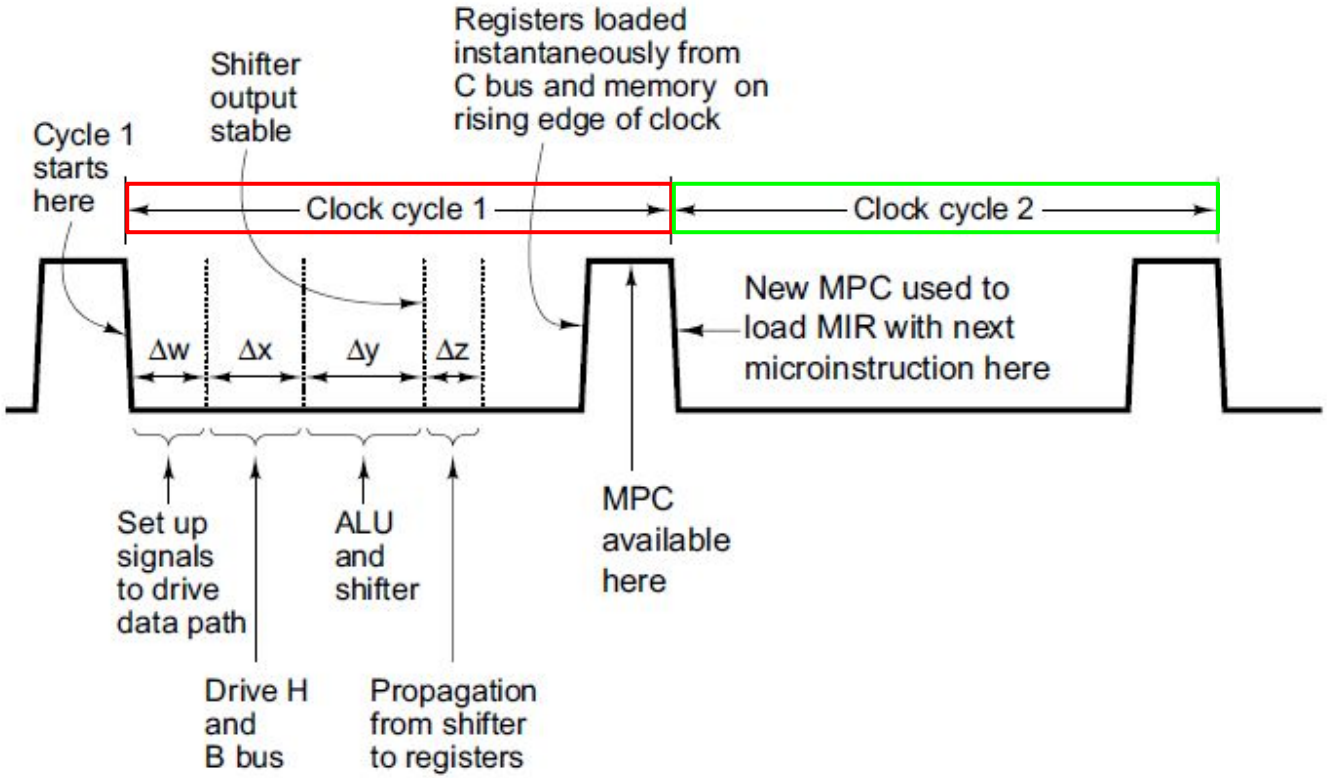


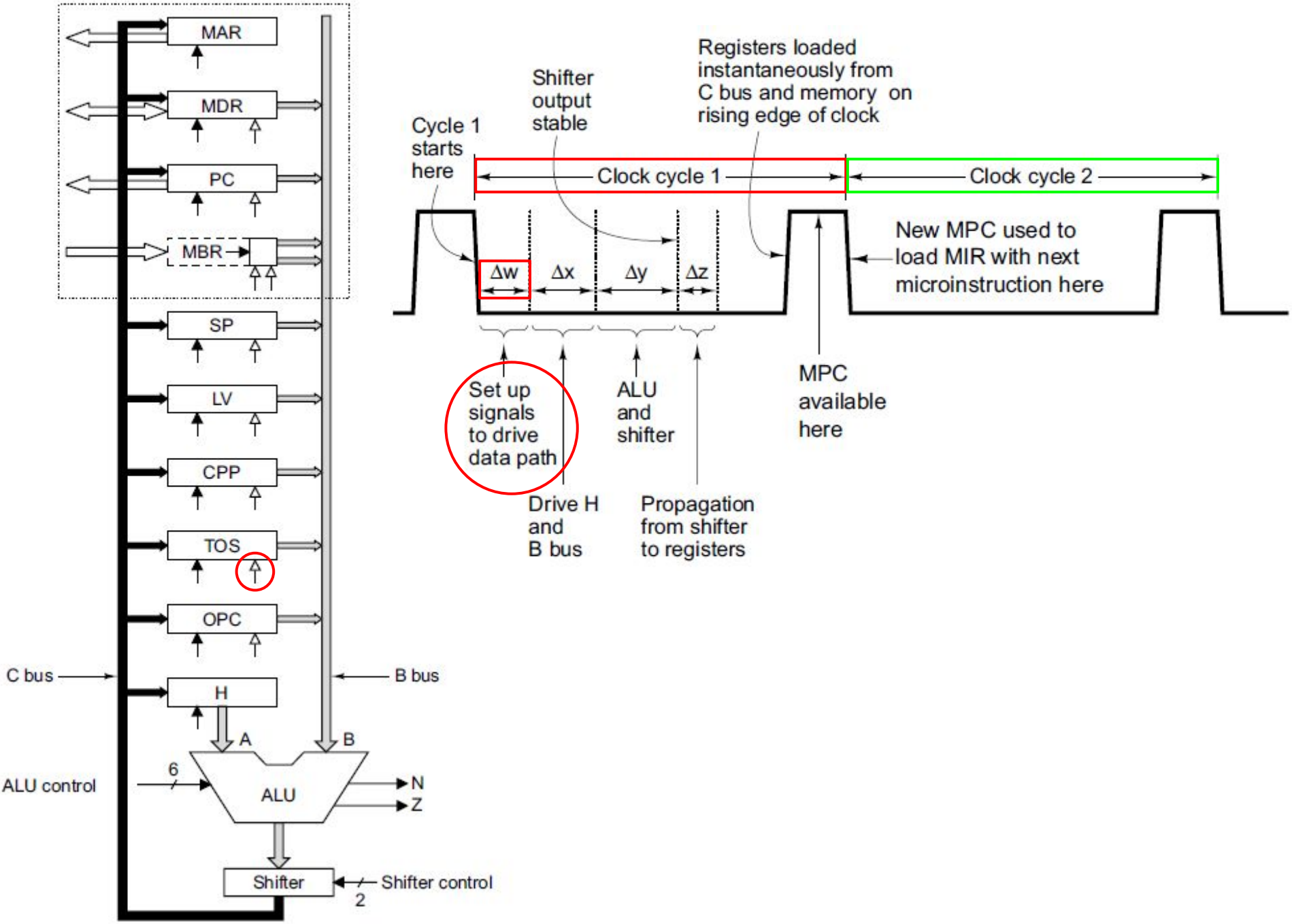
# Flujo de datos de la JVM

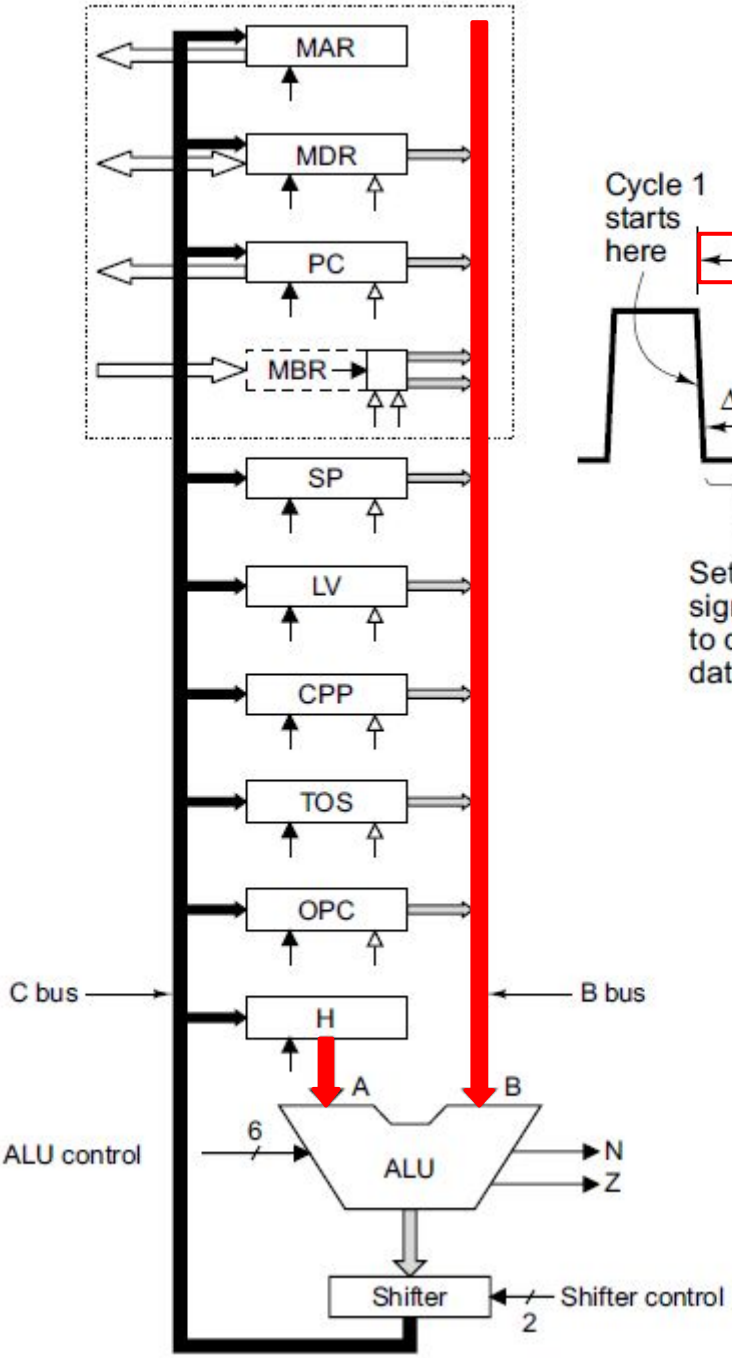


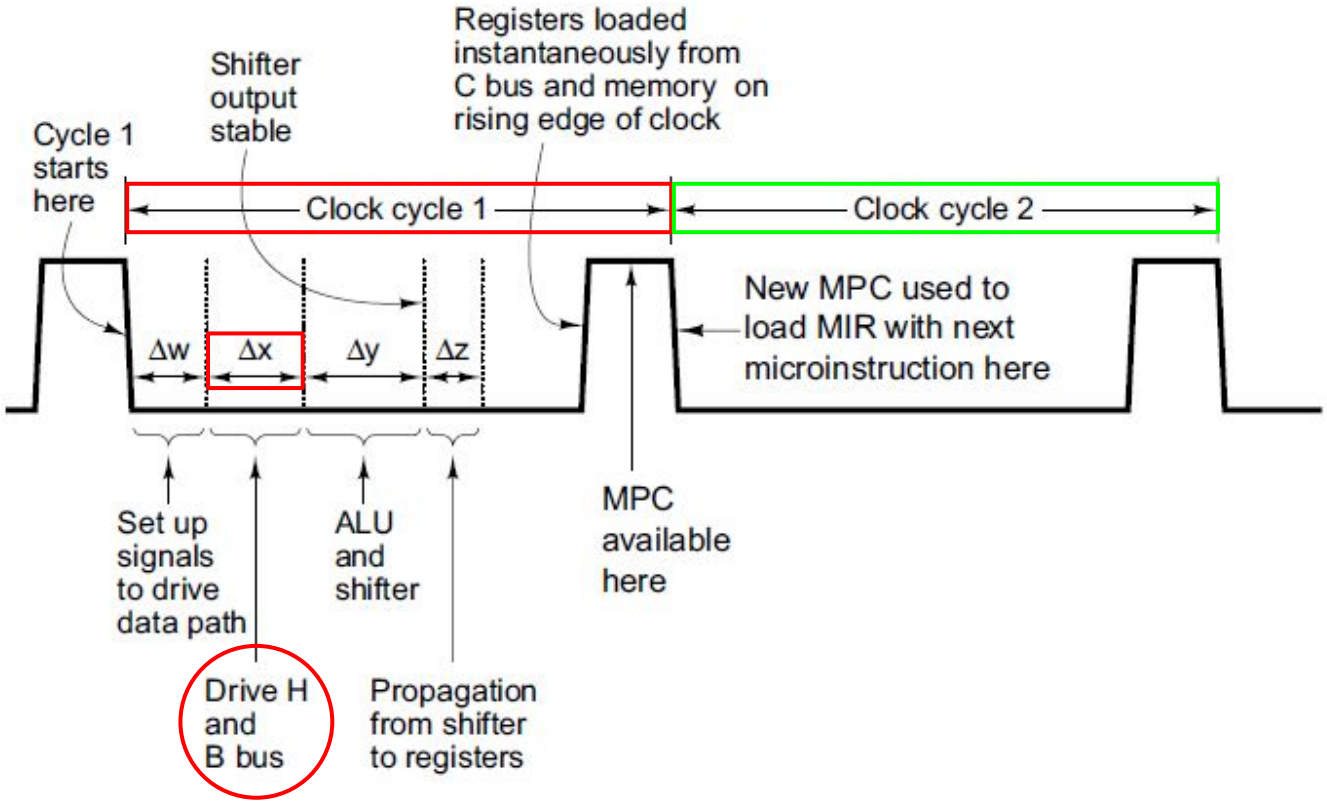

- MBR tiene 2 salidas al bus B y 2 líneas de control
- Unsigned y Signed

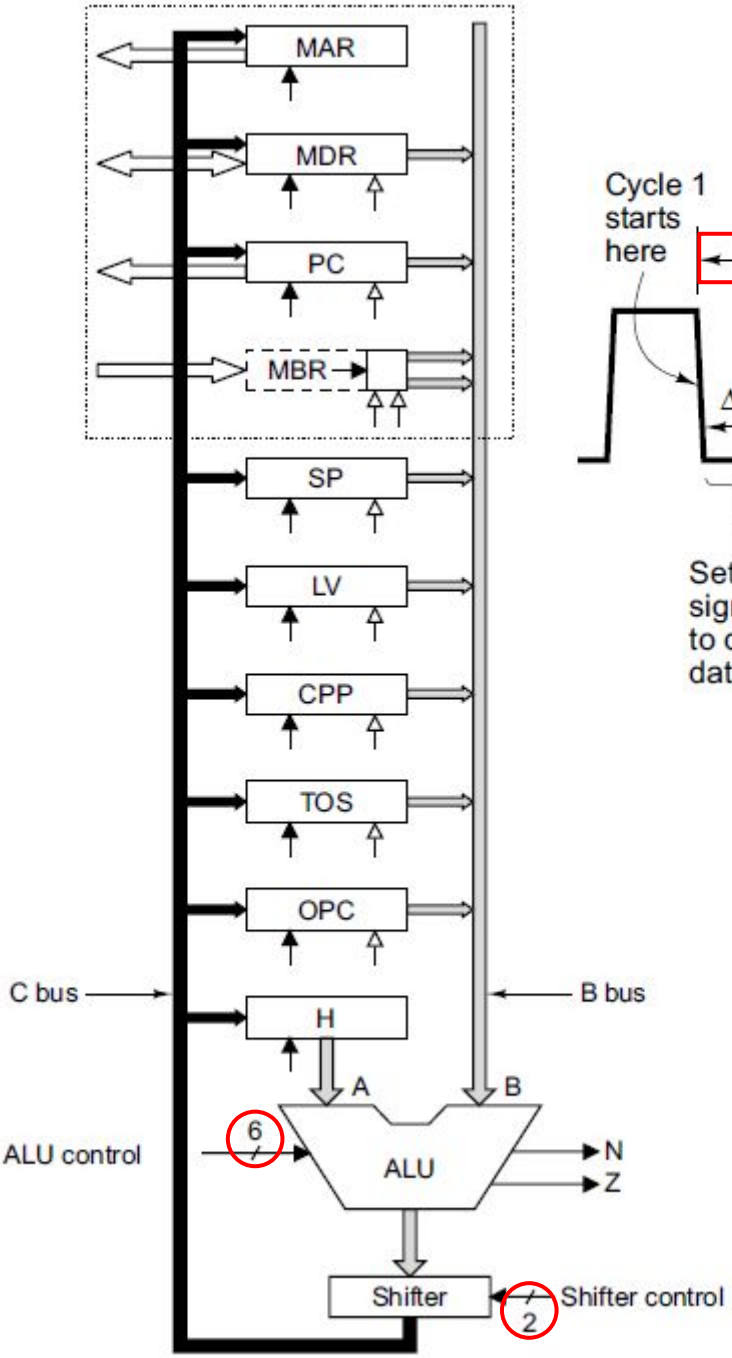

MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
C bus
B bus
H
A
B
6
ALU control
ALU
N
Z
Shifter
Shifter control
2

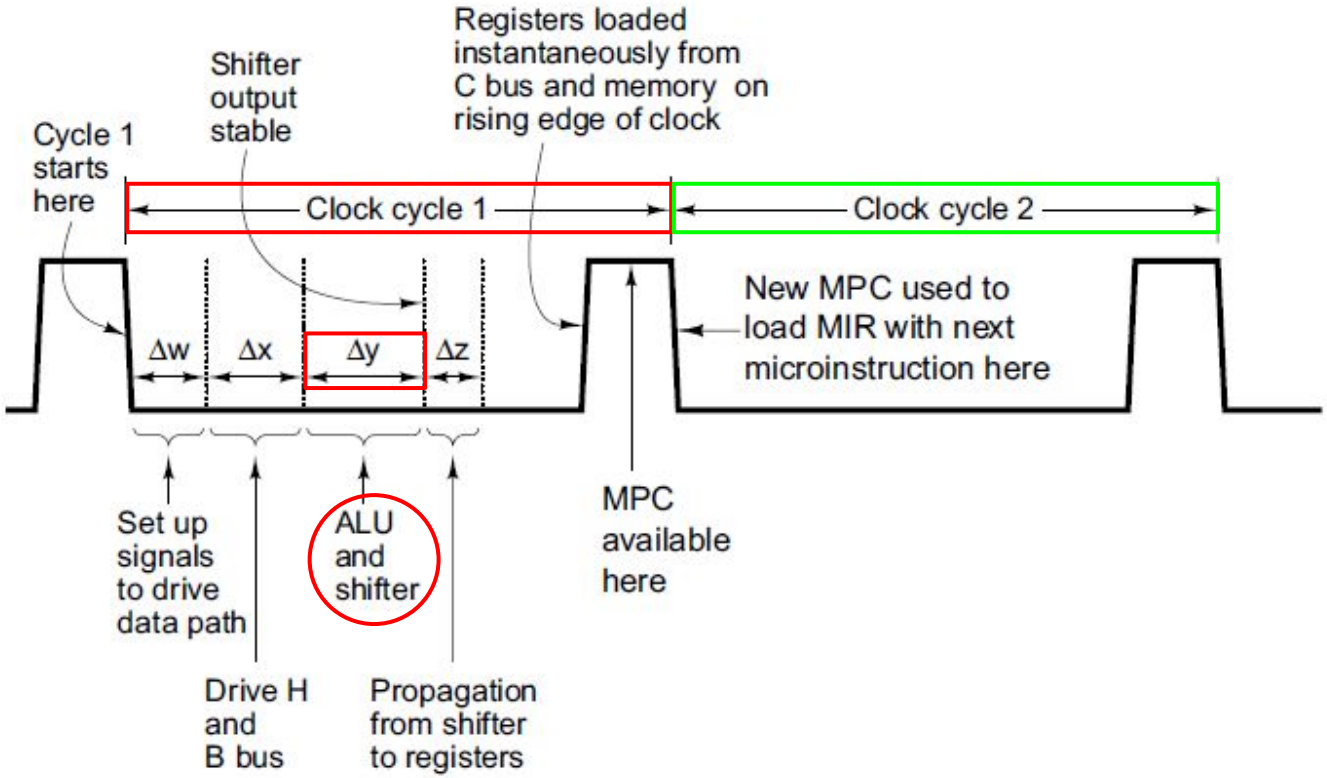

Registers loaded instantaneously from C bus and memory on rising edge of clock
Shifter output stable
Cycle 1 starts here
Clock cycle 1
Clock cycle 2
New MPC used to load MIR with next microinstruction here
Δw
Δx
Δy
Δz
Set up signals to drive data path
Drive H and B bus
ALU and shifter
Propagation from shifter to registers
MPC available here

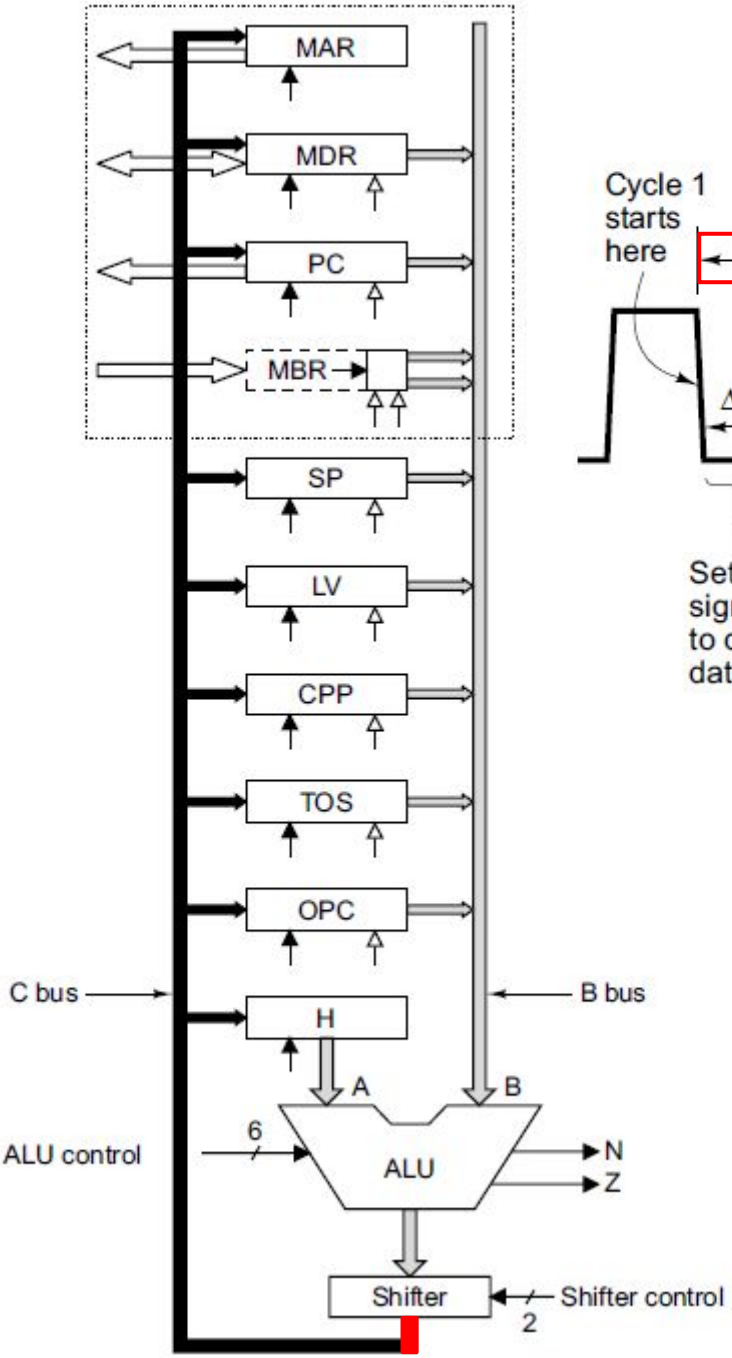

MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
C bus
H
B bus
A
B
6
ALU control
ALU
N
Z
Shifter
Shifter control
2
Registers loaded instantaneously from C bus and memory on rising edge of clock
Shifter output stable
Cycle 1 starts here
Clock cycle 1
Clock cycle 2
New MPC used to load MIR with next microinstruction here
Δw
Δx
Δy
Δz
Set up signals to drive data path
Drive H and B bus
ALU and shifter
Propagation from shifter to registers
MPC available here

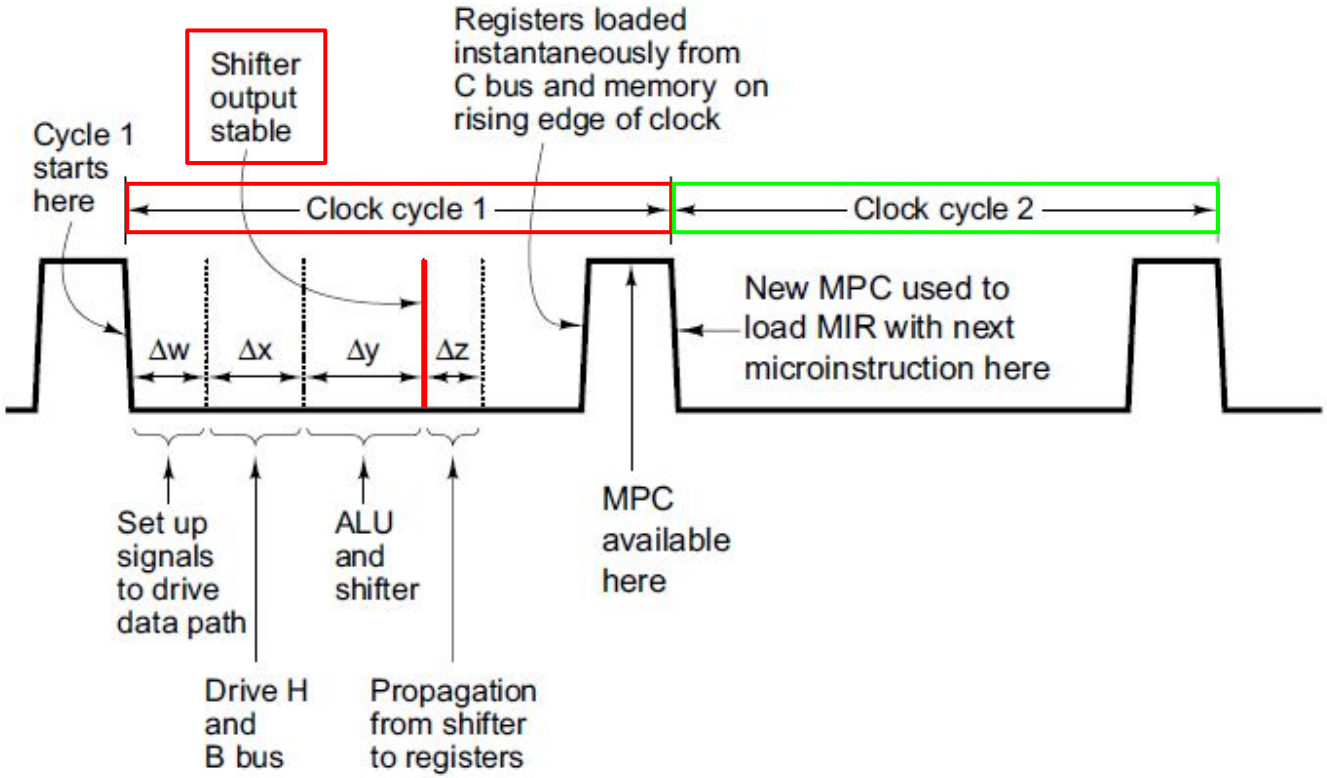

MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
C bus
B bus
H
A
B
6
ALU control
ALU
N
Z
Shifter
Shifter control
2

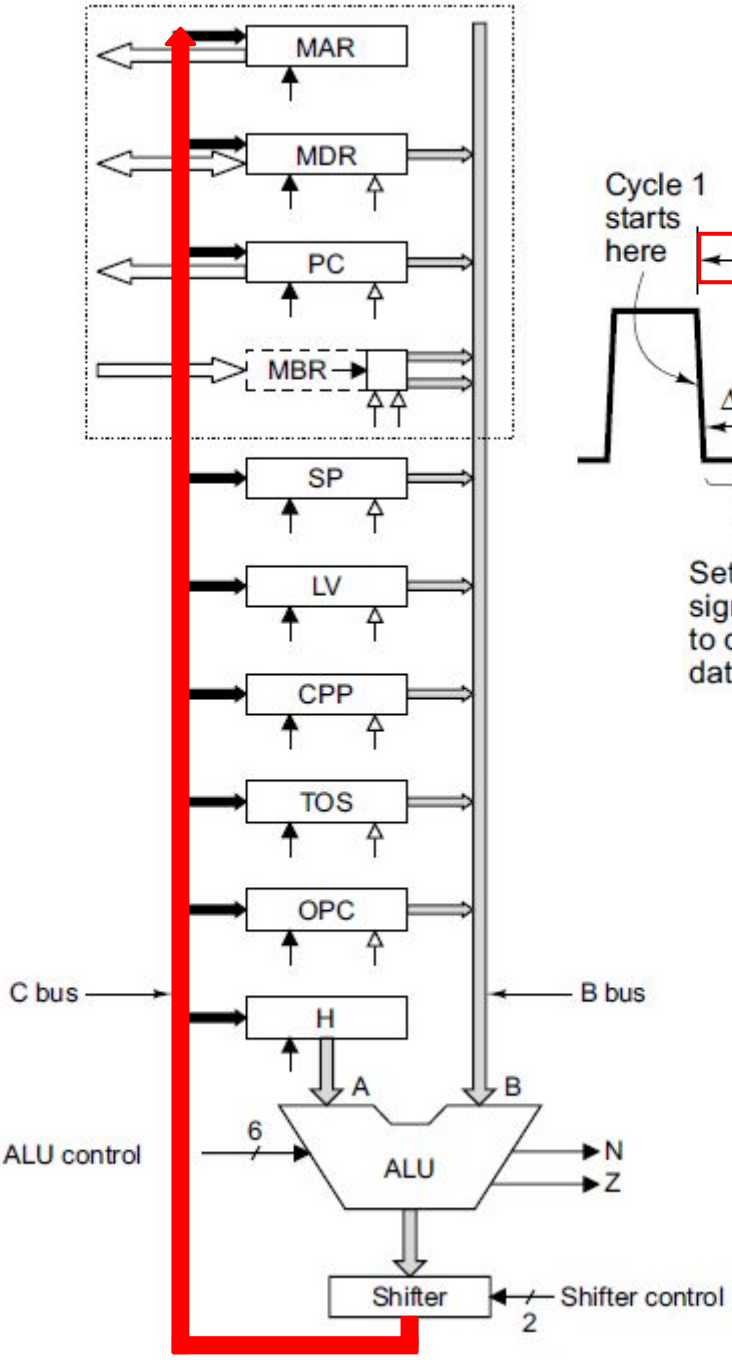

Registers loaded instantaneously from C bus and memory on rising edge of clock
Shifter output stable
Cycle 1 starts here
Clock cycle 1
Clock cycle 2
New MPC used to load MIR with next microinstruction here
Δw
Δx
Δy
Δz
Set up signals to drive data path
Drive H and B bus
ALU and shifter
Propagation from shifter to registers
MPC available here

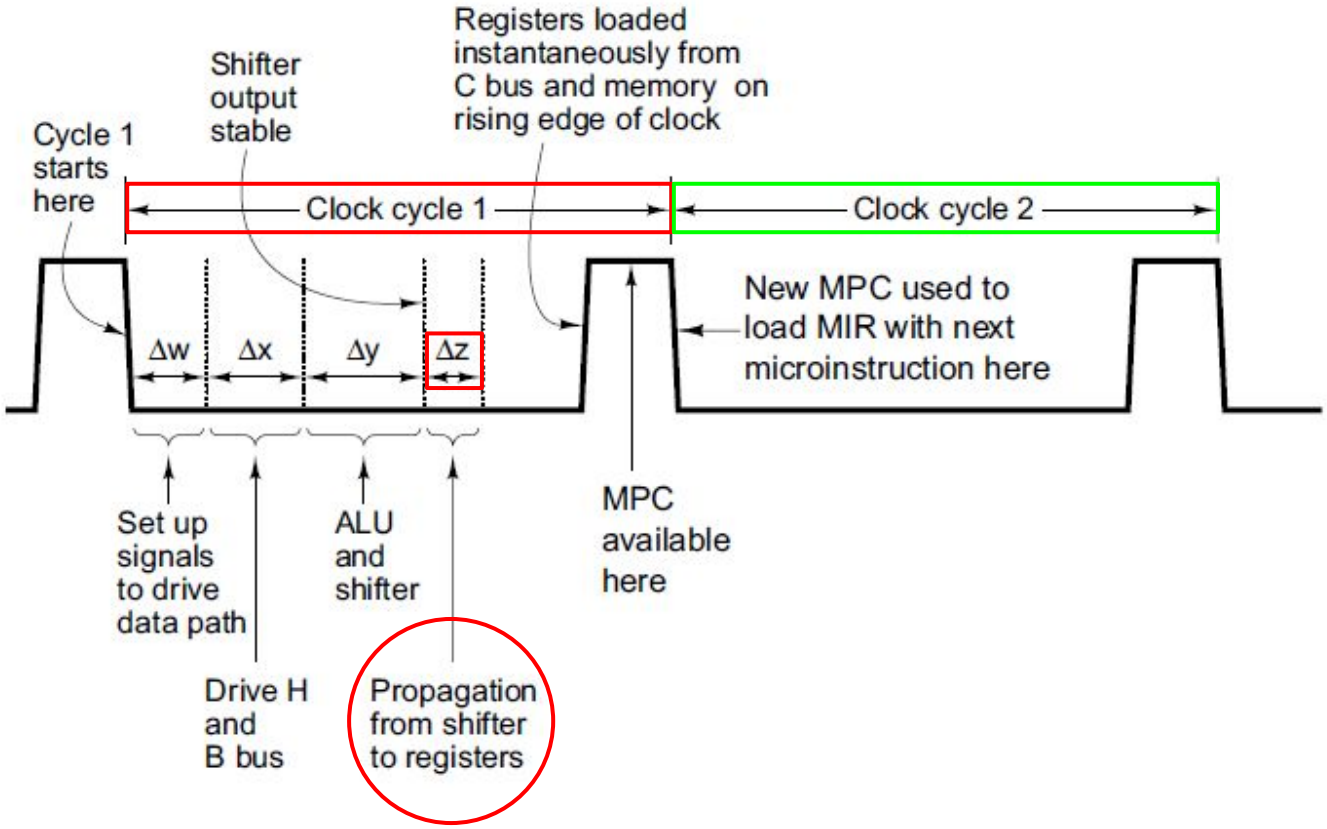

MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
C bus
B bus
H
A
B
6
ALU control
ALU
N
Z
Shifter
Shifter control
2

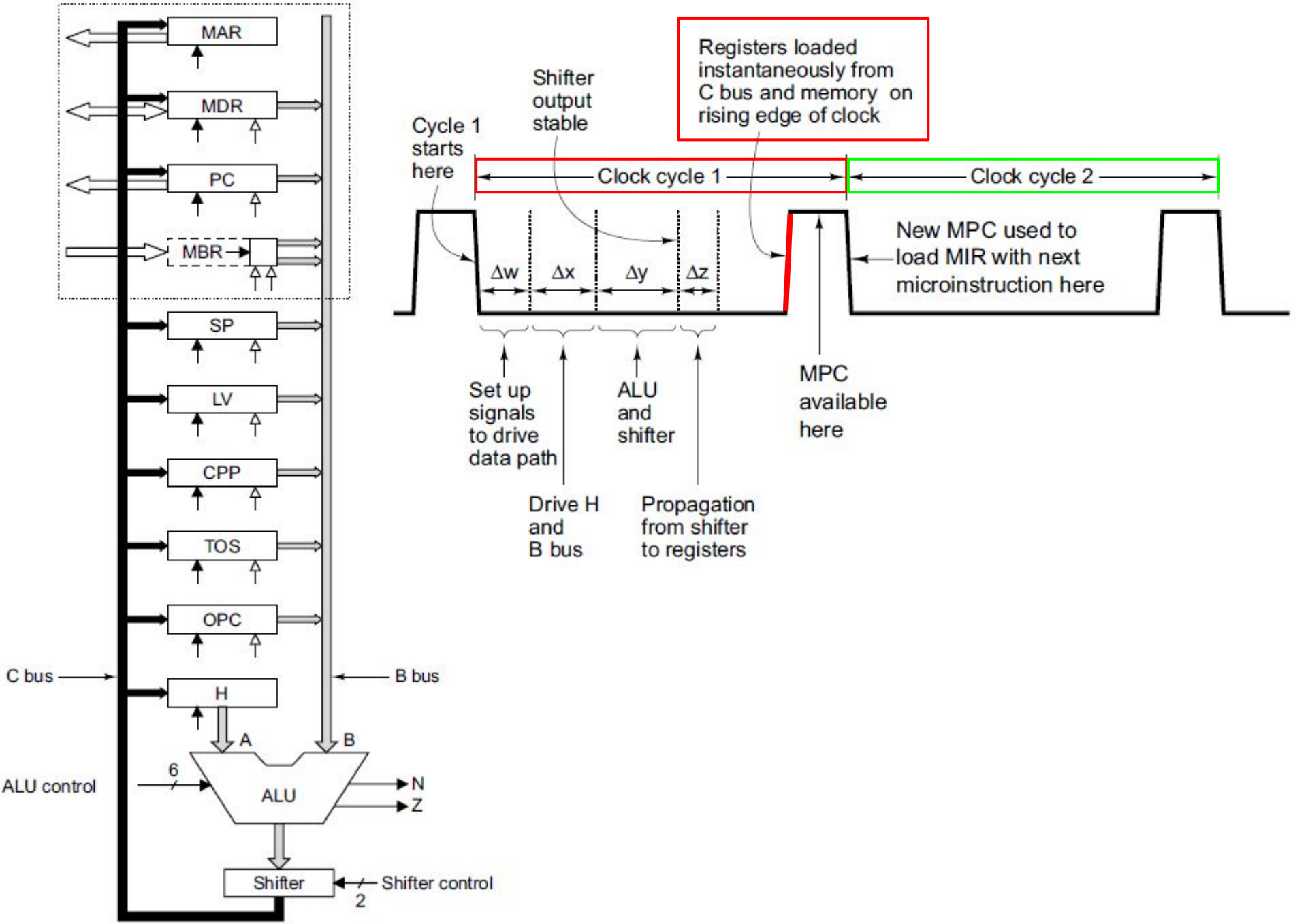

Registers loaded instantaneously from C bus and memory on rising edge of clock
Shifter output stable
Cycle 1 starts here
Clock cycle 1
Clock cycle 2
Δw
Δx
Δy
Δz
New MPC used to load MIR with next microinstruction here
Set up signals to drive data path
ALU and shifter
MPC available here
Drive H and B bus
Propagation from shifter to registers

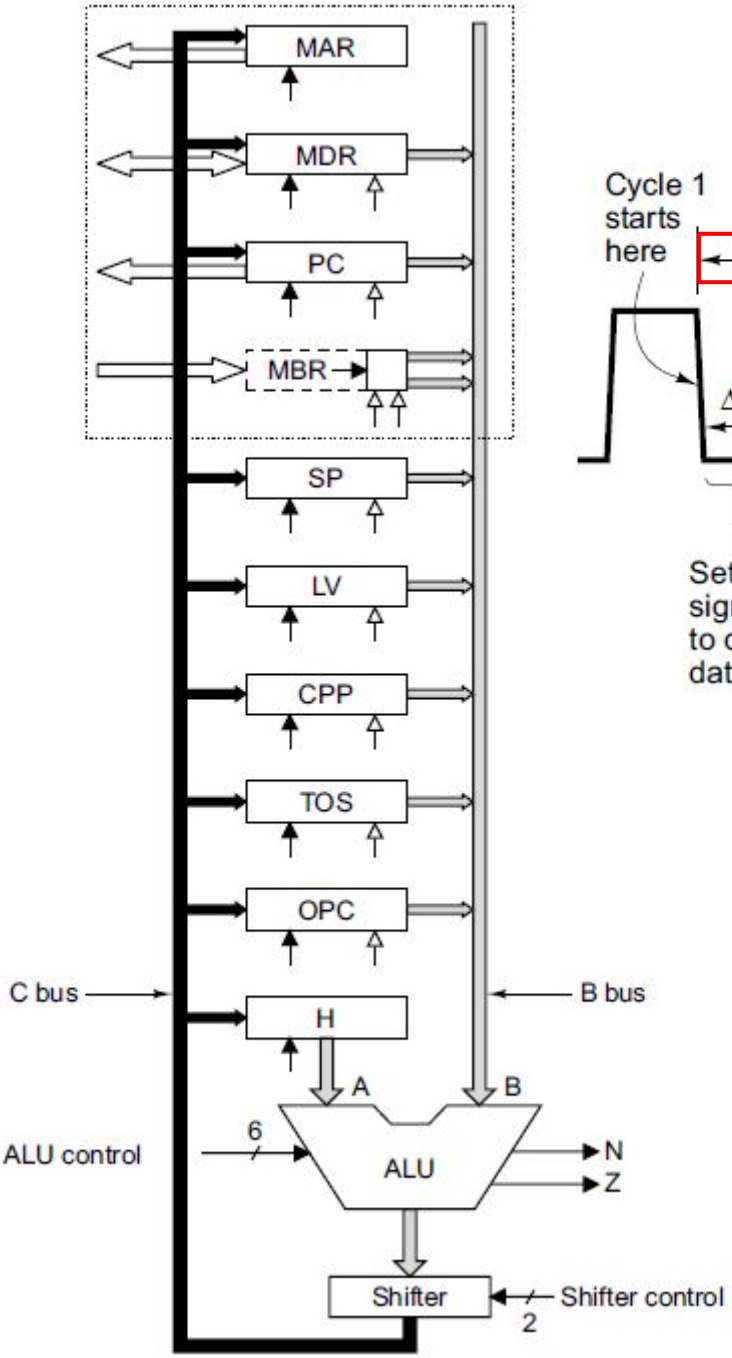

MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
C bus
B bus
H
A
B
6
ALU control
ALU
N
Z
Shifter
Shifter control
2

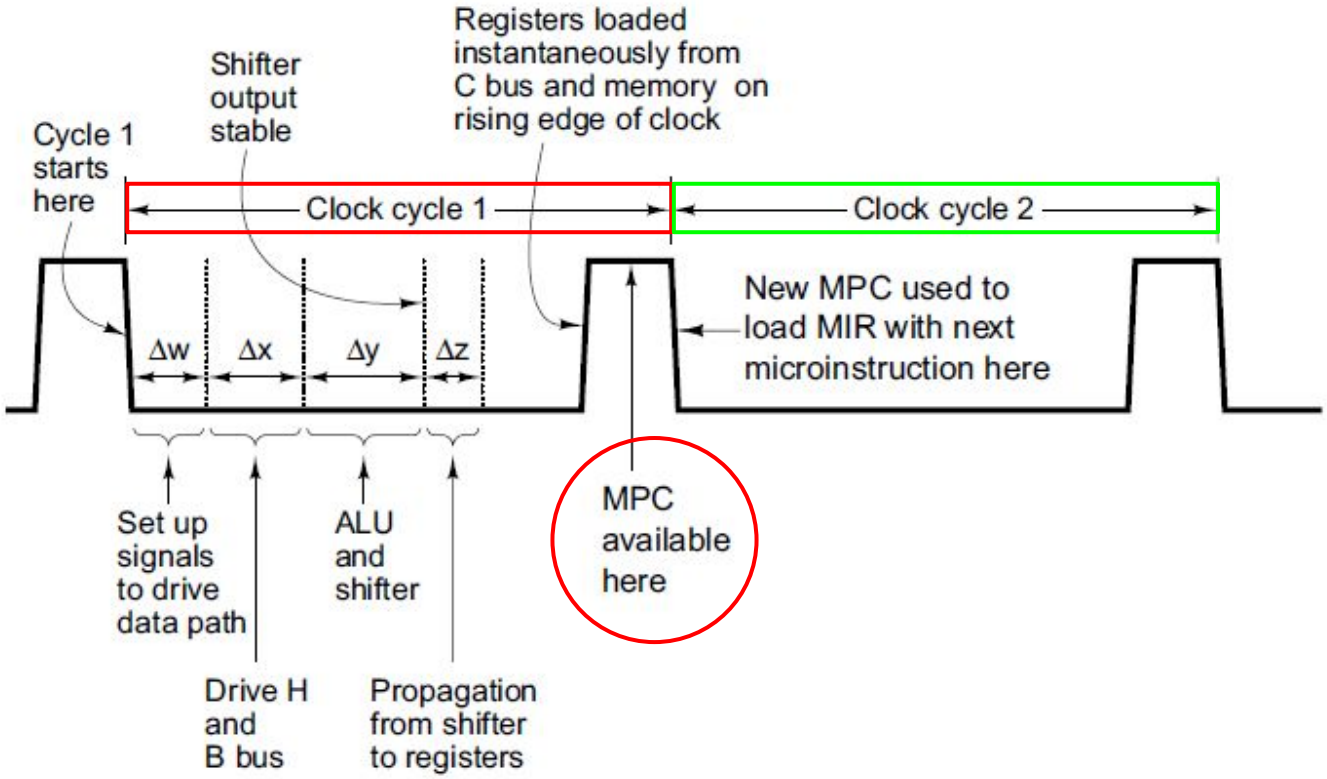

Shifter output stable
Registers loaded instantaneously from C bus and memory on rising edge of clock
Cycle 1 starts here
Clock cycle 1
Clock cycle 2
New MPC used to load MIR with next microinstruction here
Δw
Δx
Δy
Δz
Set up signals to drive data path
Drive H and B bus
ALU and shifter
Propagation from shifter to registers
MPC available here

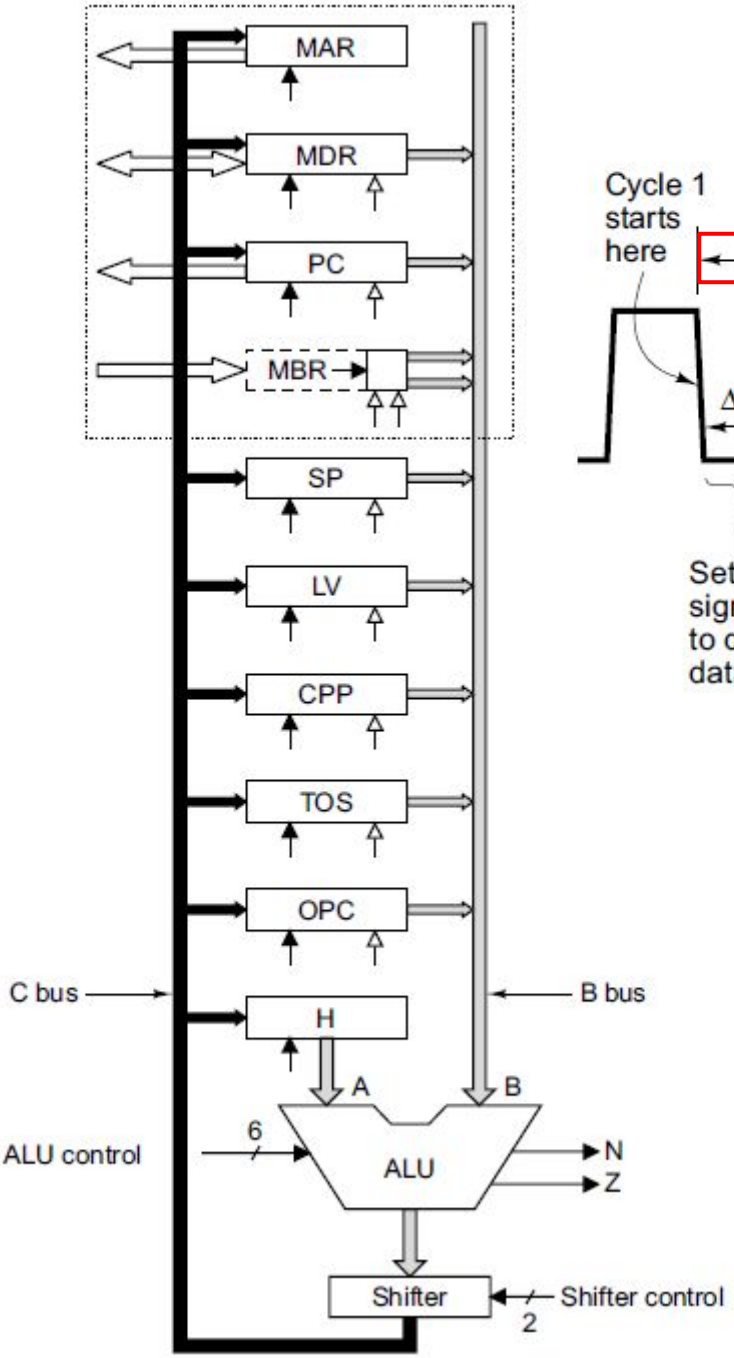

MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
C bus
B bus
H
A
B
6
ALU control
ALU
N
Z
Shifter
Shifter control
2

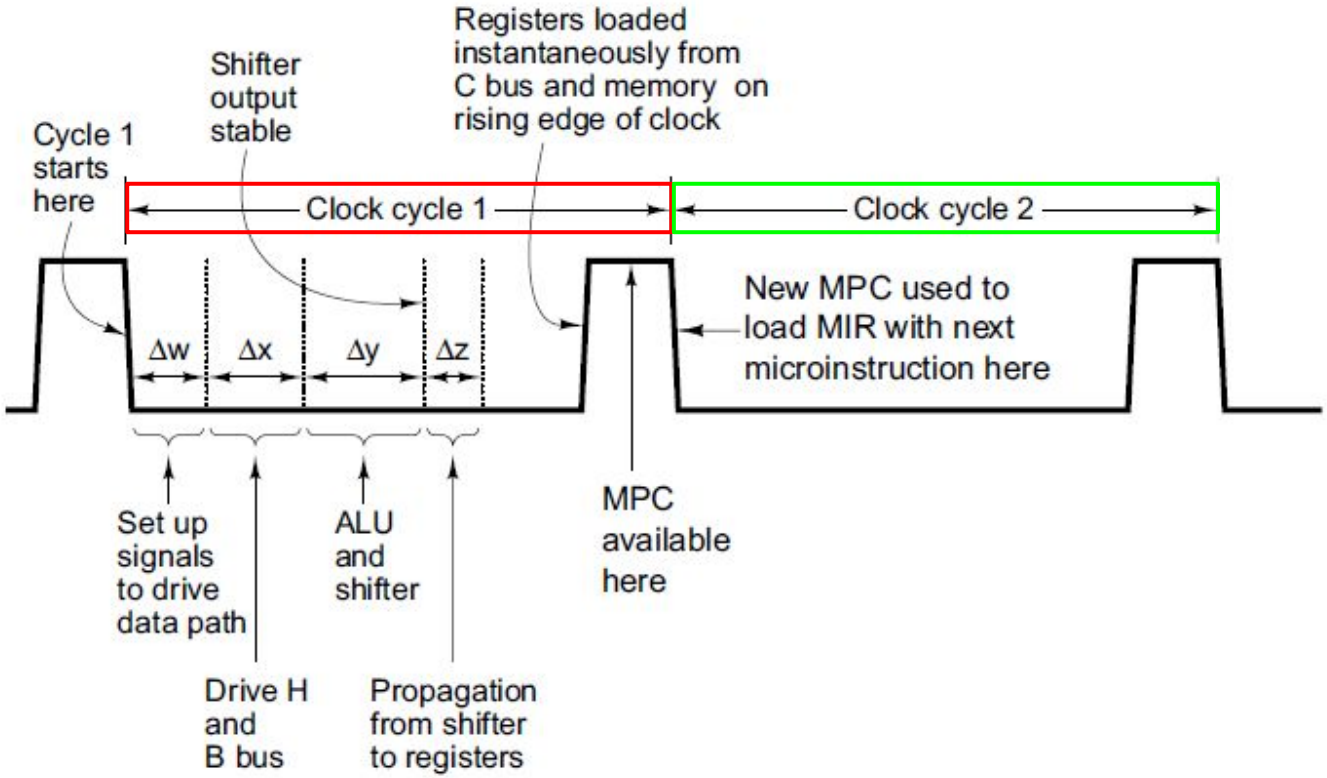

Registers loaded instantaneously from C bus and memory on rising edge of clock
Shifter output stable
Cycle 1 starts here
Clock cycle 1
Clock cycle 2
New MPC used to load MIR with next microinstruction here
Δw
Δx
Δy
Δz
Set up signals to drive data path
Drive H and B bus
ALU and shifter
Propagation from shifter to registers
MPC available here


MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
C bus
B bus
H
A
B
6
ALU control
ALU
N
Z
Shifter
Shifter control
2
Registers loaded instantaneously from C bus and memory on rising edge of clock
Shifter output stable
Cycle 1 starts here
Clock cycle 1
Clock cycle 2
New MPC used to load MIR with next microinstruction here
Δw
Δx
Δy
Δz
Set up signals to drive data path
Drive H and B bus
ALU and shifter
Propagation from shifter to registers
MPC available here


MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
C bus
B bus
H
A
B
6
ALU control
ALU
N
Z
Shifter
Shifter control
2

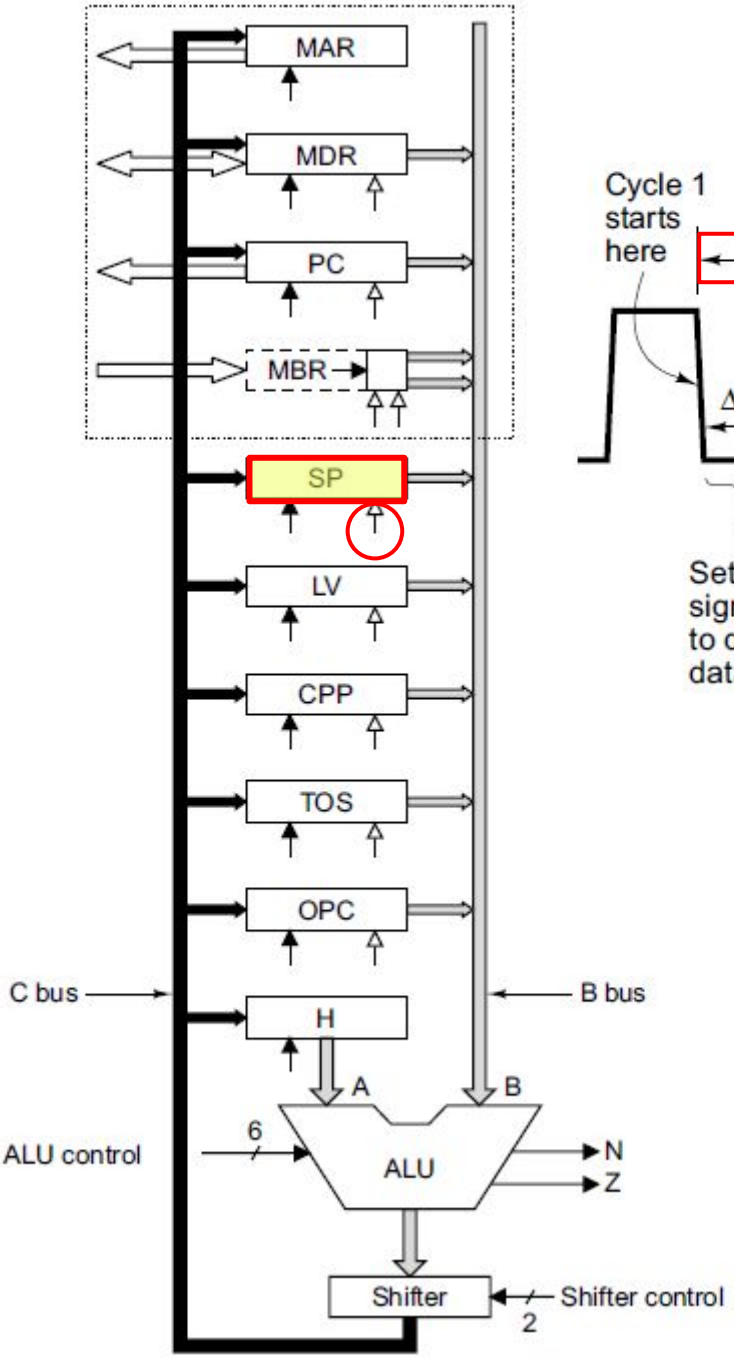

Registers loaded instantaneously from C bus and memory on rising edge of clock
Shifter output stable
Cycle 1 starts here
Clock cycle 1
Clock cycle 2
New MPC used to load MIR with next microinstruction here
Δw
Δx
Δy
Δz
Set up signals to drive data path
Drive H and B bus
ALU and shifter
Propagation from shifter to registers
MPC available here

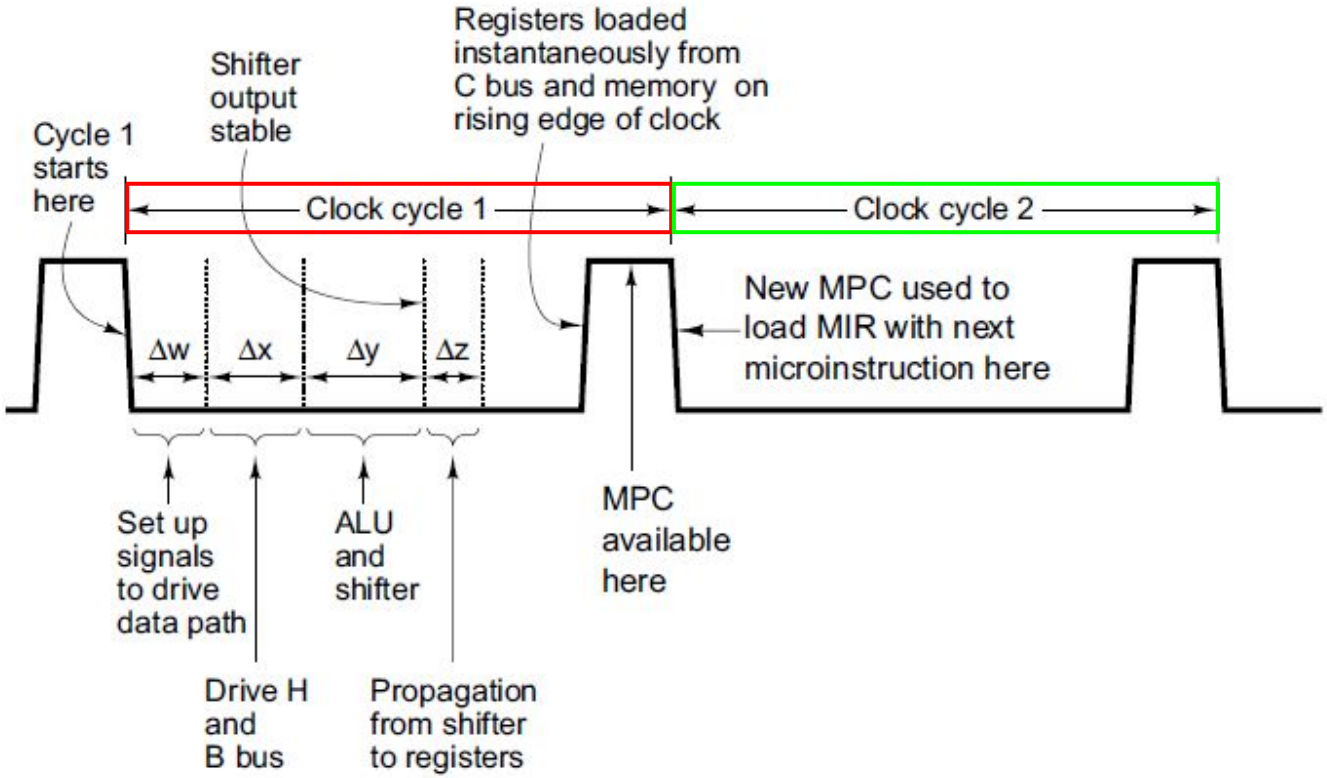



Acciones para resolver la instrucción **IADD**

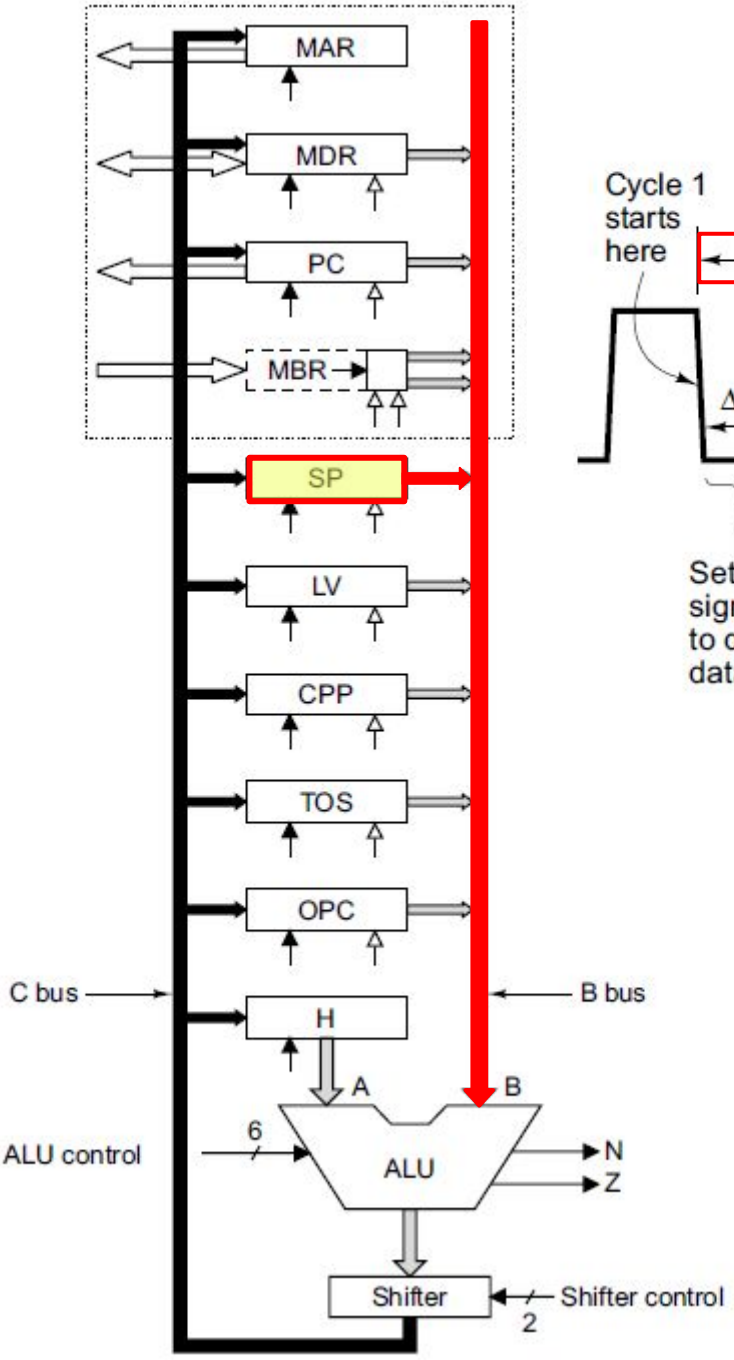
**MAR = SP = SP - 1; rd;**
**H = TOS;**
**MDR = TOS = H + MDR; wr;**

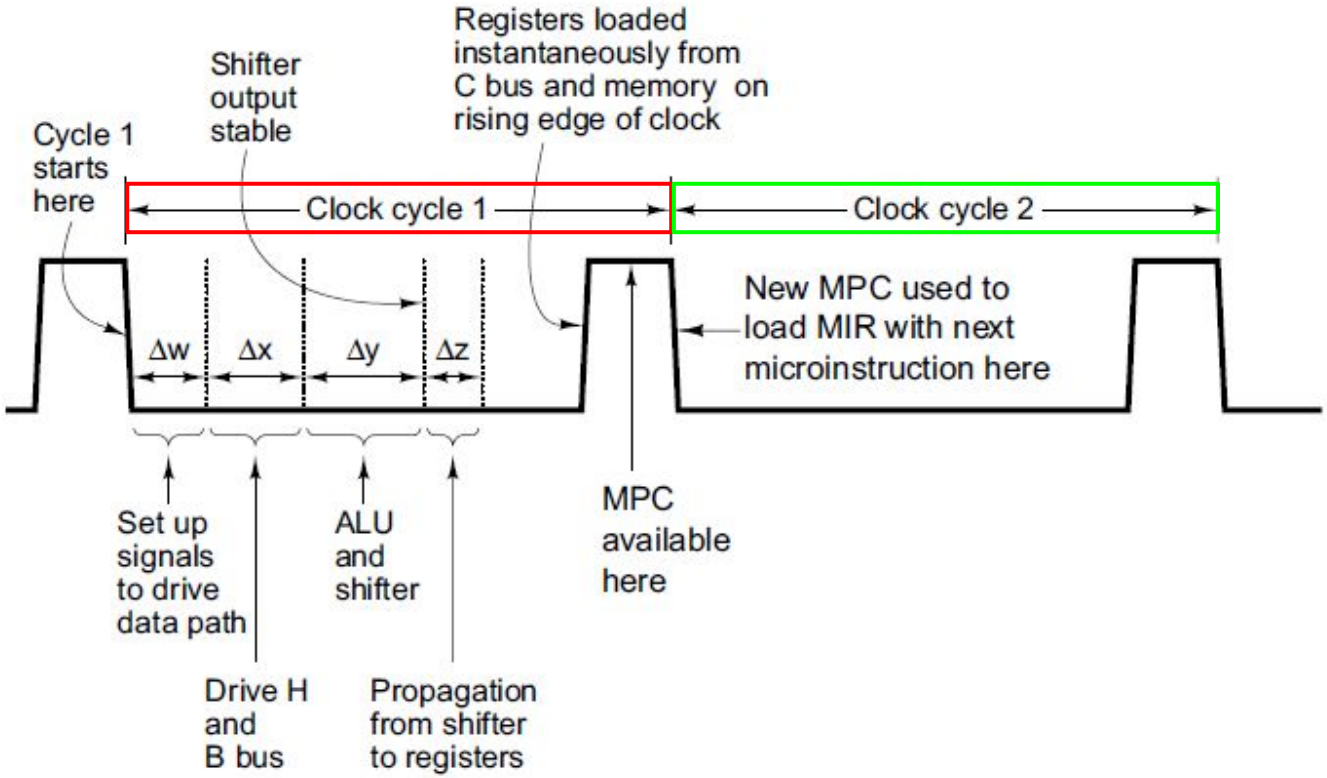



Acciones para resolver la instrucción **IADD**

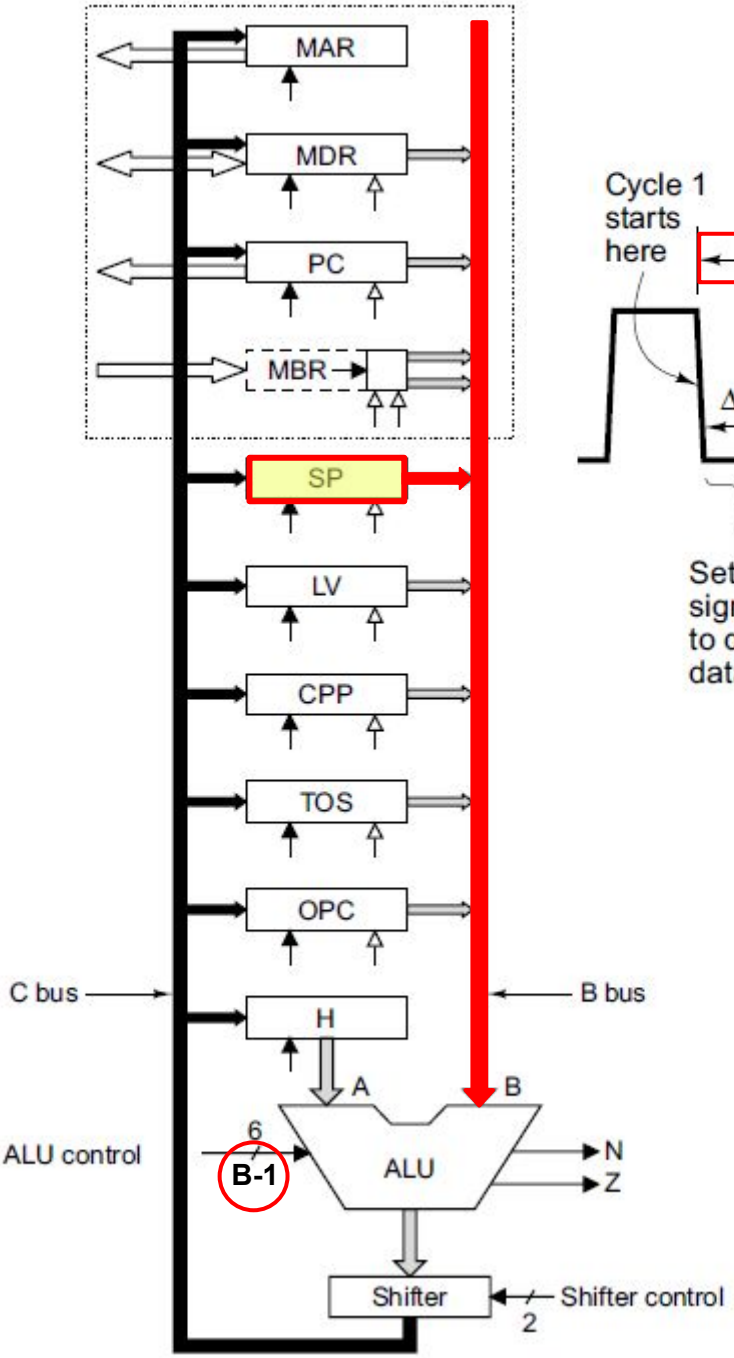
**MAR = SP = SP - 1; rd;**
**H = TOS;**
**MDR = TOS = H + MDR; wr;**

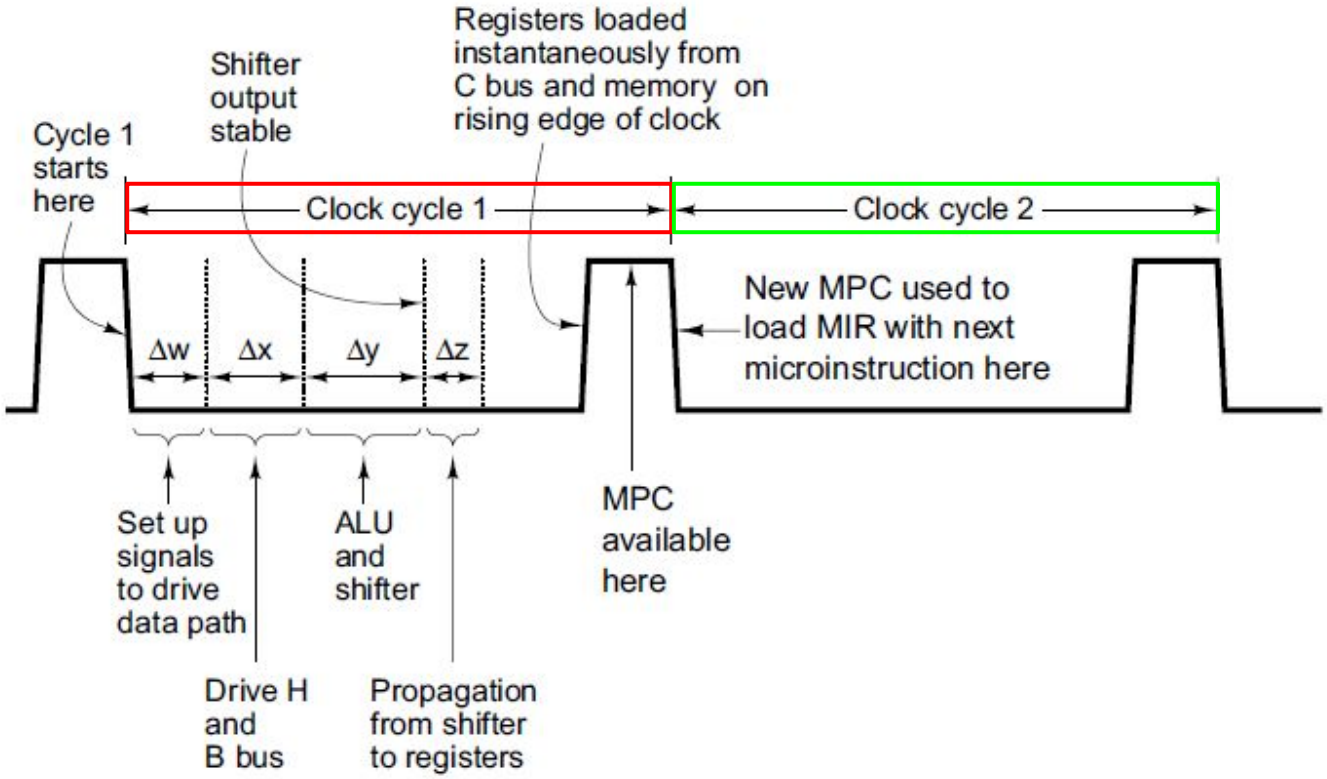



Acciones para resolver la instrucción **IADD**

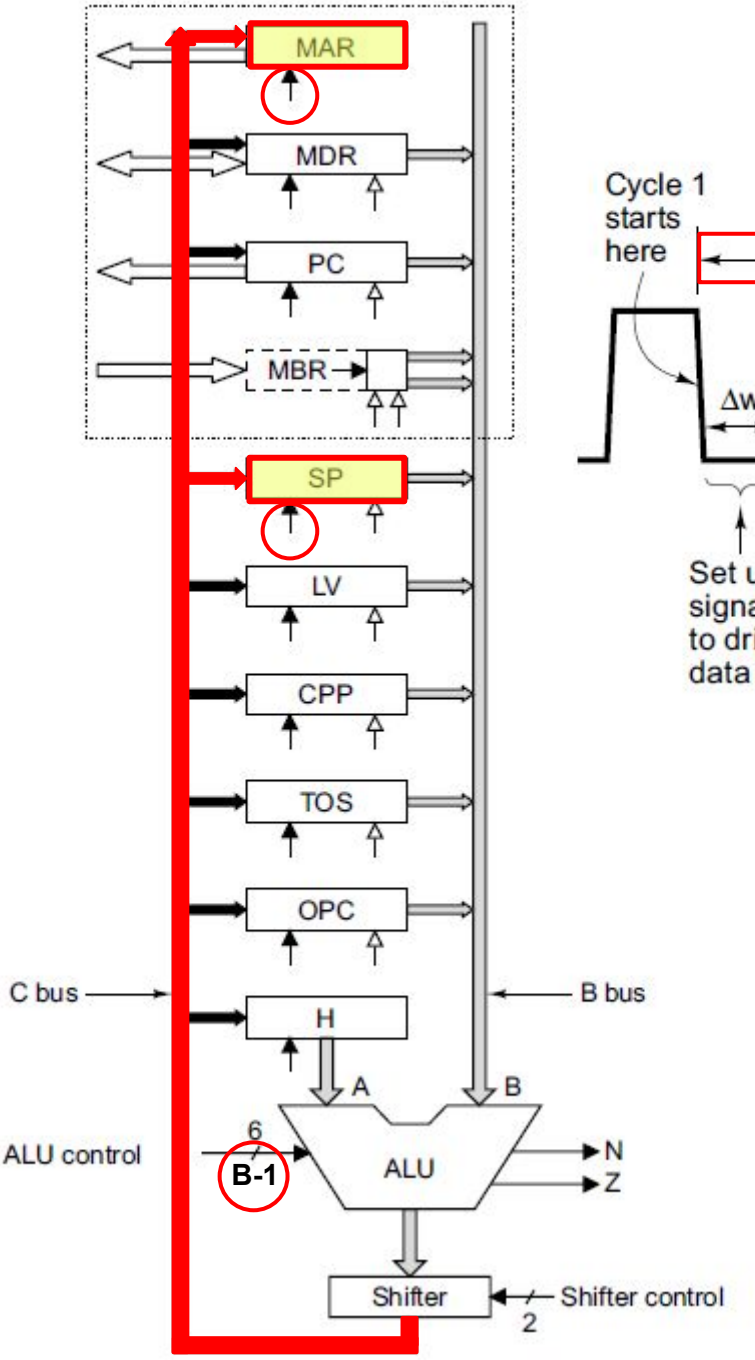
**MAR = SP = SP - 1; rd;**
**H = TOS;**
**MDR = TOS = H + MDR; wr;**

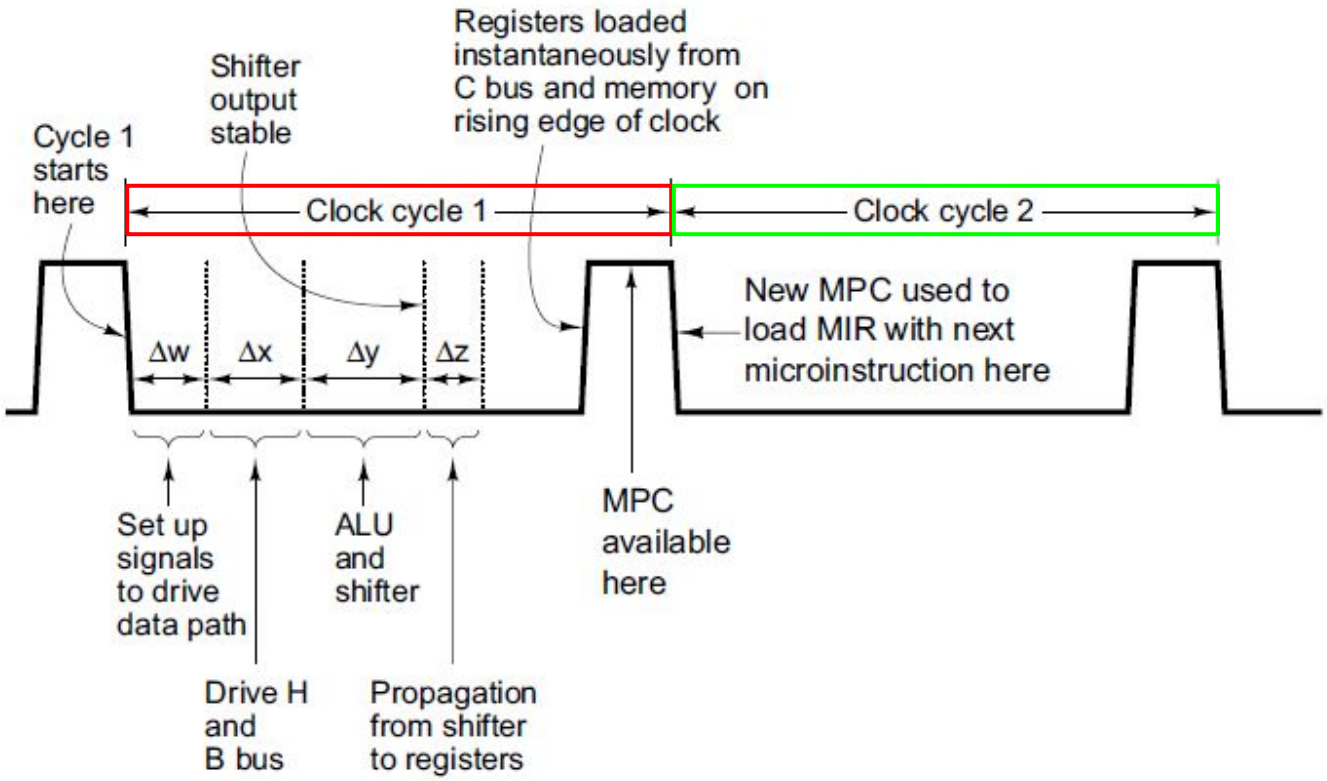



Acciones para resolver la instrucción **IADD**

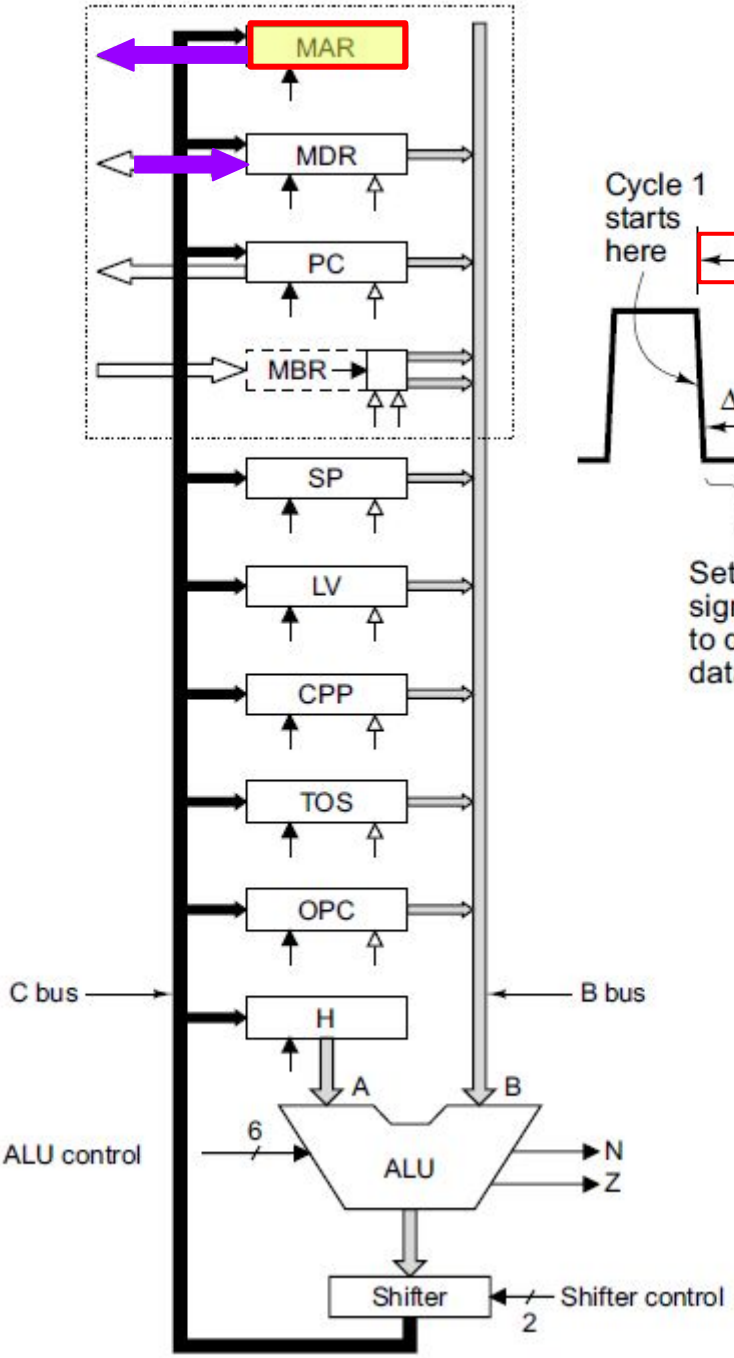
**MAR = SP = SP - 1; rd;**
**H = TOS;**
**MDR = TOS = H + MDR; wr;**

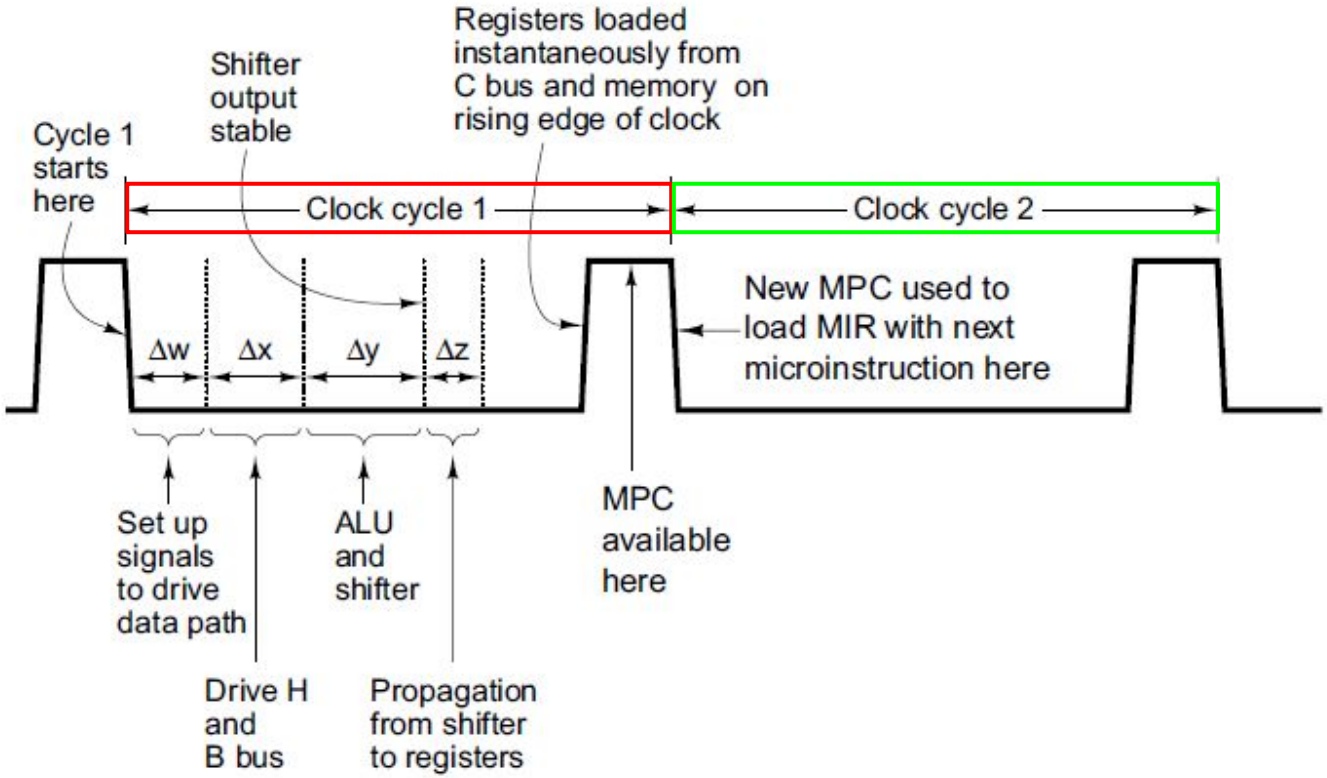



Acciones para resolver la instrucción **IADD**

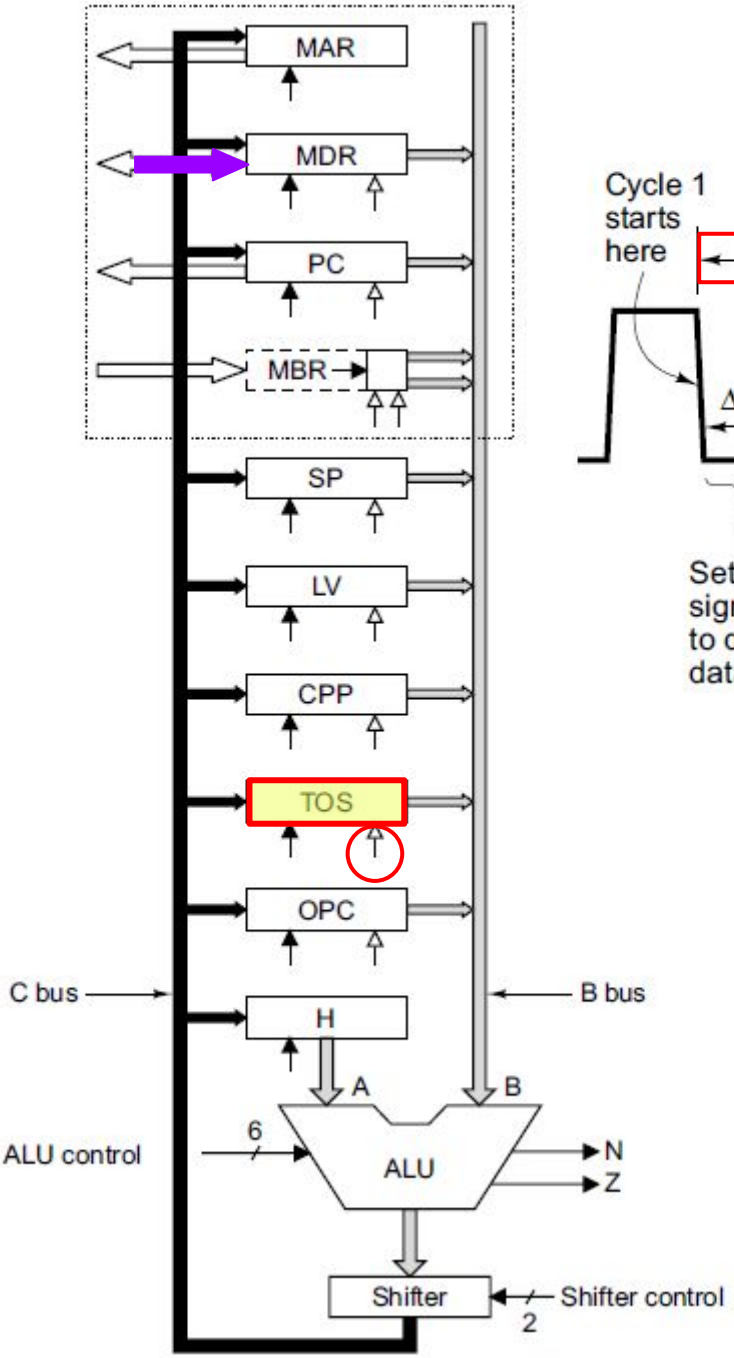
**MAR = SP = SP - 1; rd;**
**H = TOS;**
**MDR = TOS = H + MDR; wr;**

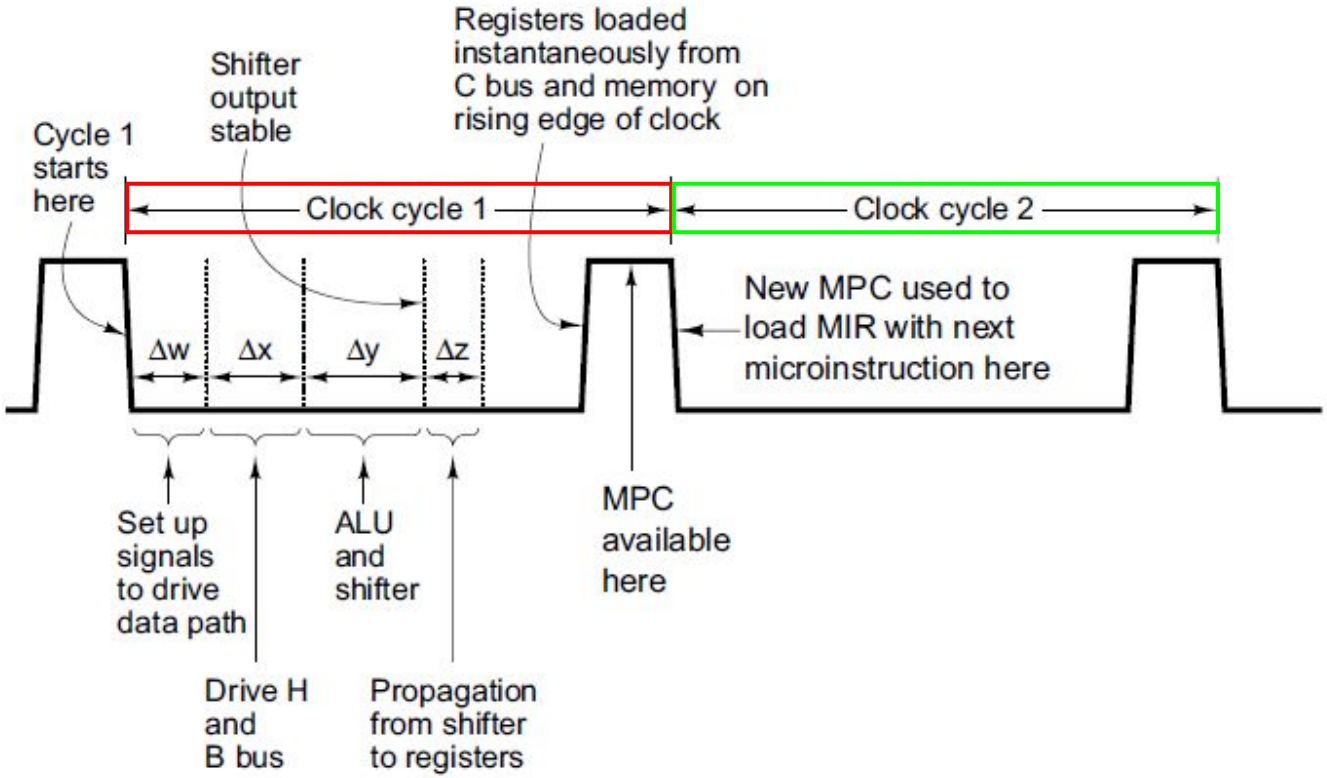



Acciones para resolver la instrucción **IADD**

**MAR = SP = SP - 1; rd;**
**H = TOS;**
**MDR = TOS = H + MDR; wr;**



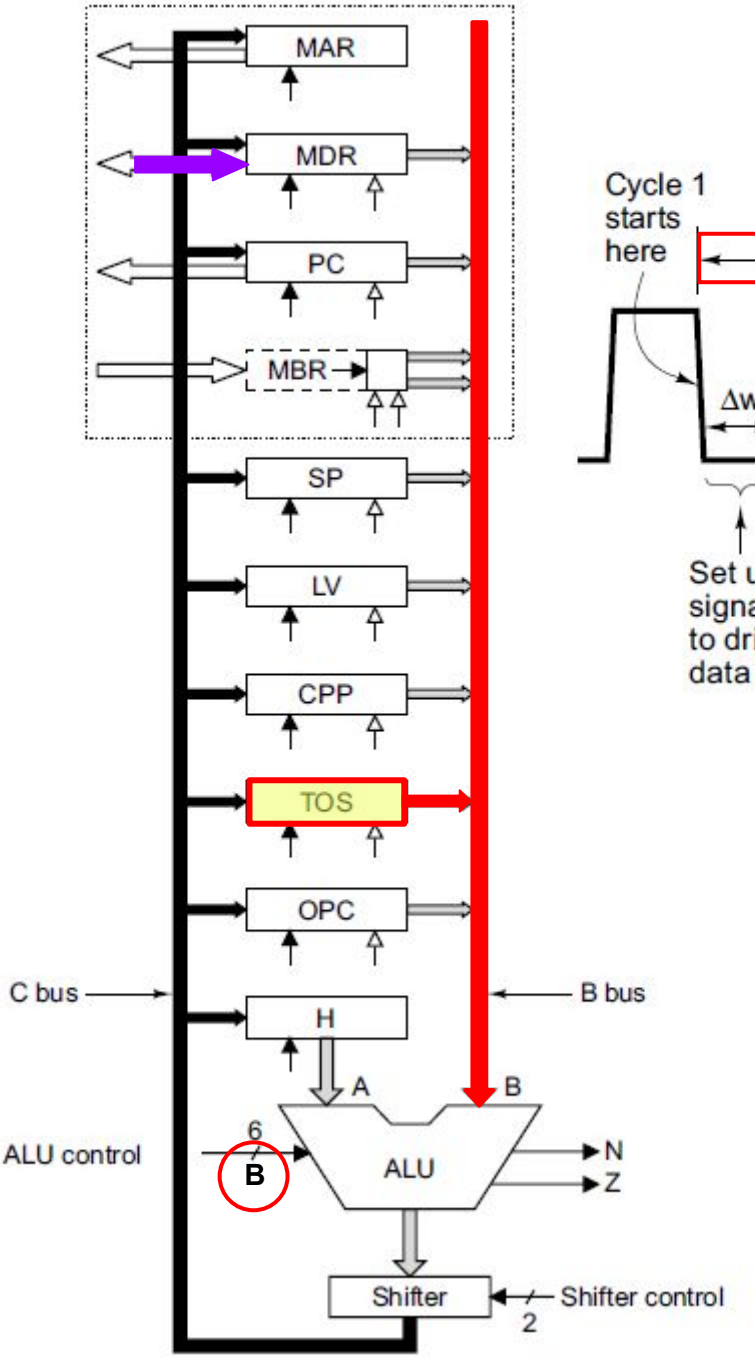



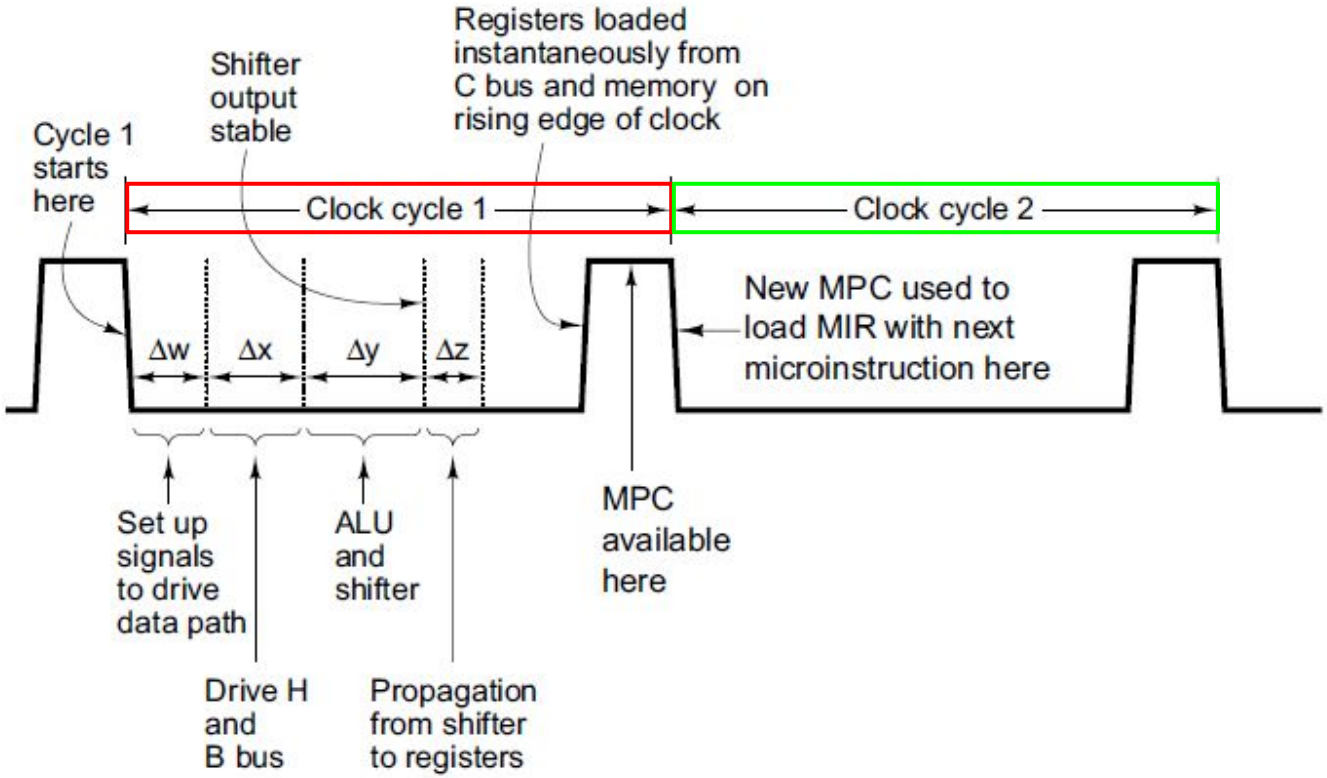

Acciones para resolver la instrucción **IADD**

**MAR = SP = SP - 1; rd;**

**H = TOS;**

**MDR = TOS = H + MDR; wr;**



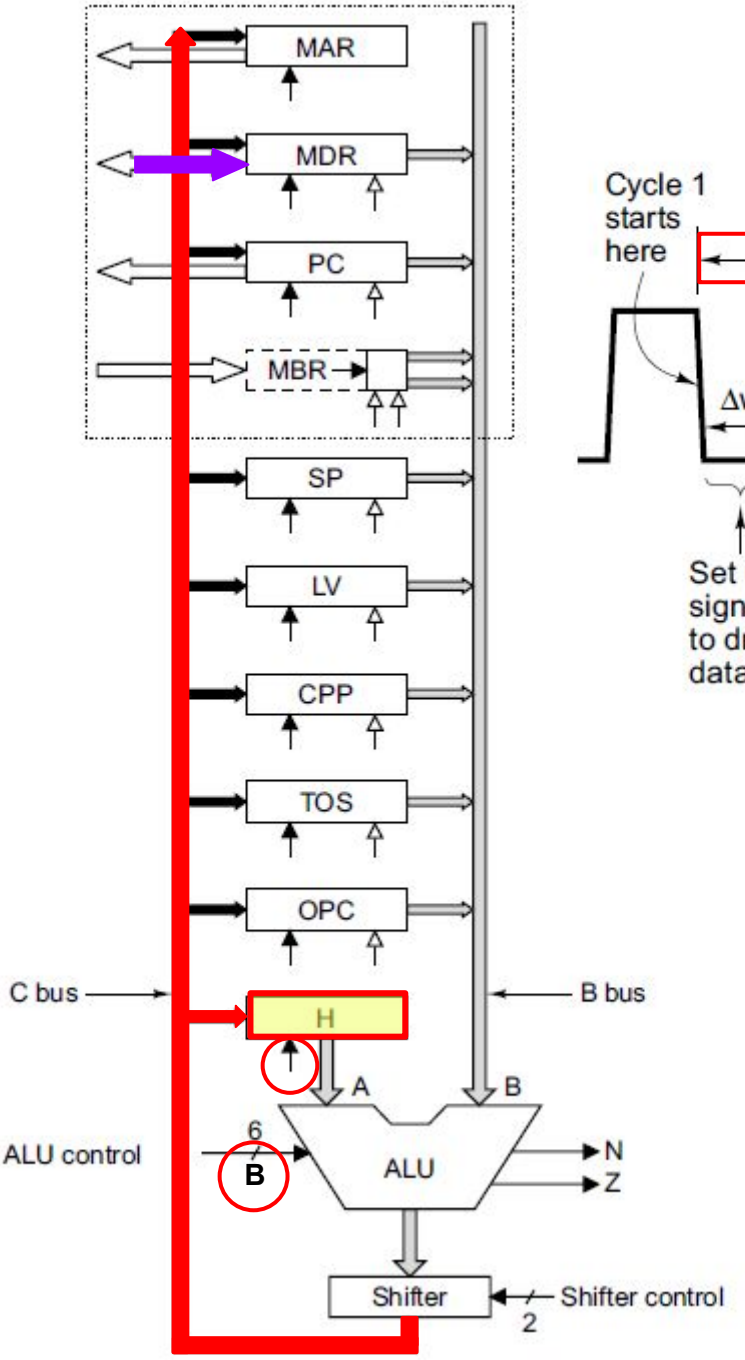



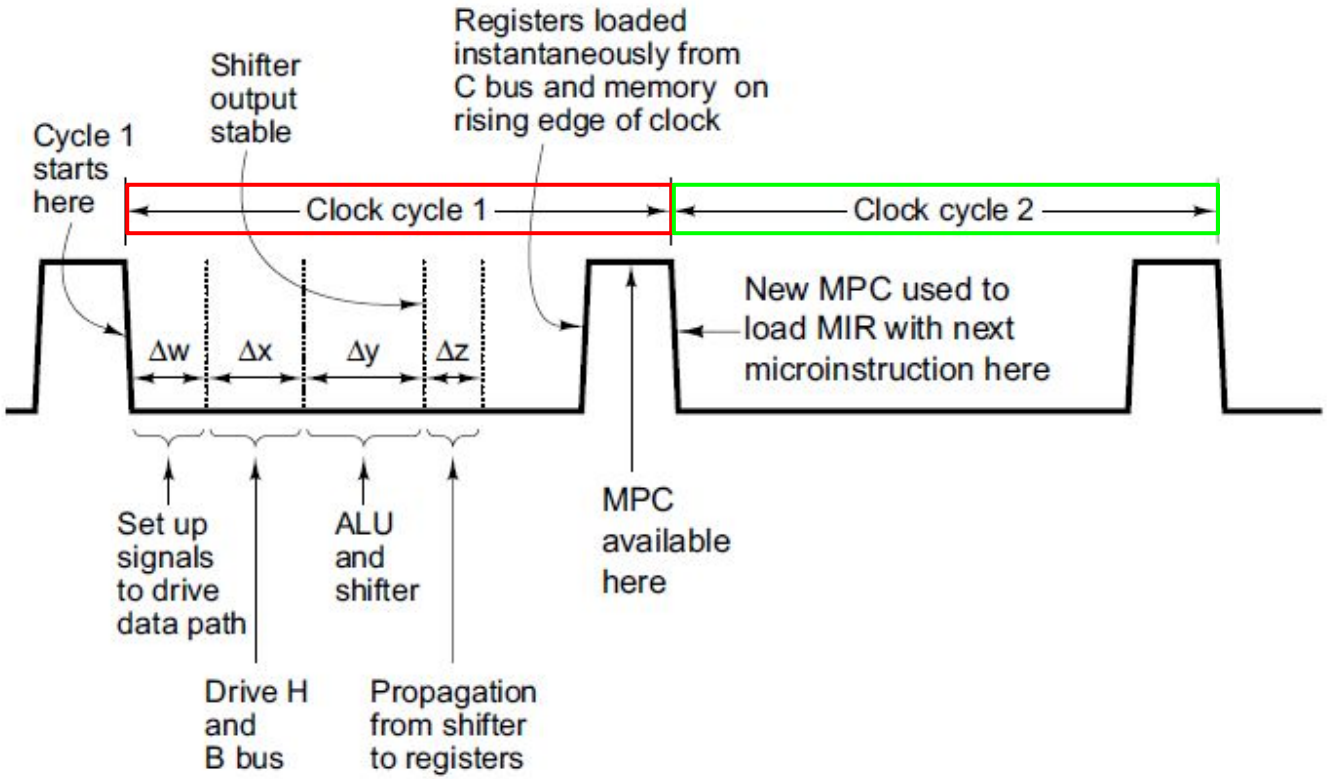

Acciones para resolver la instrucción **IADD**

**MAR = SP = SP - 1; rd;**

**H = TOS;**

**MDR = TOS = H + MDR; wr;**



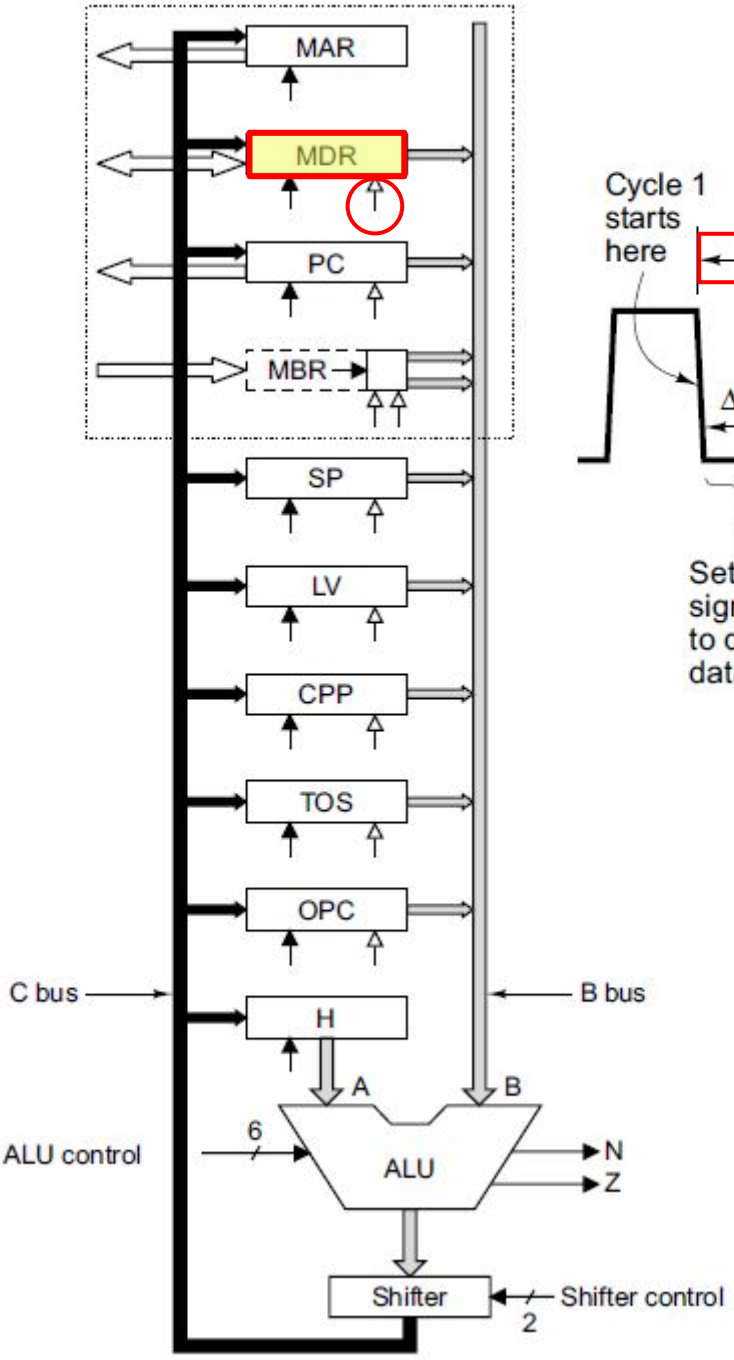



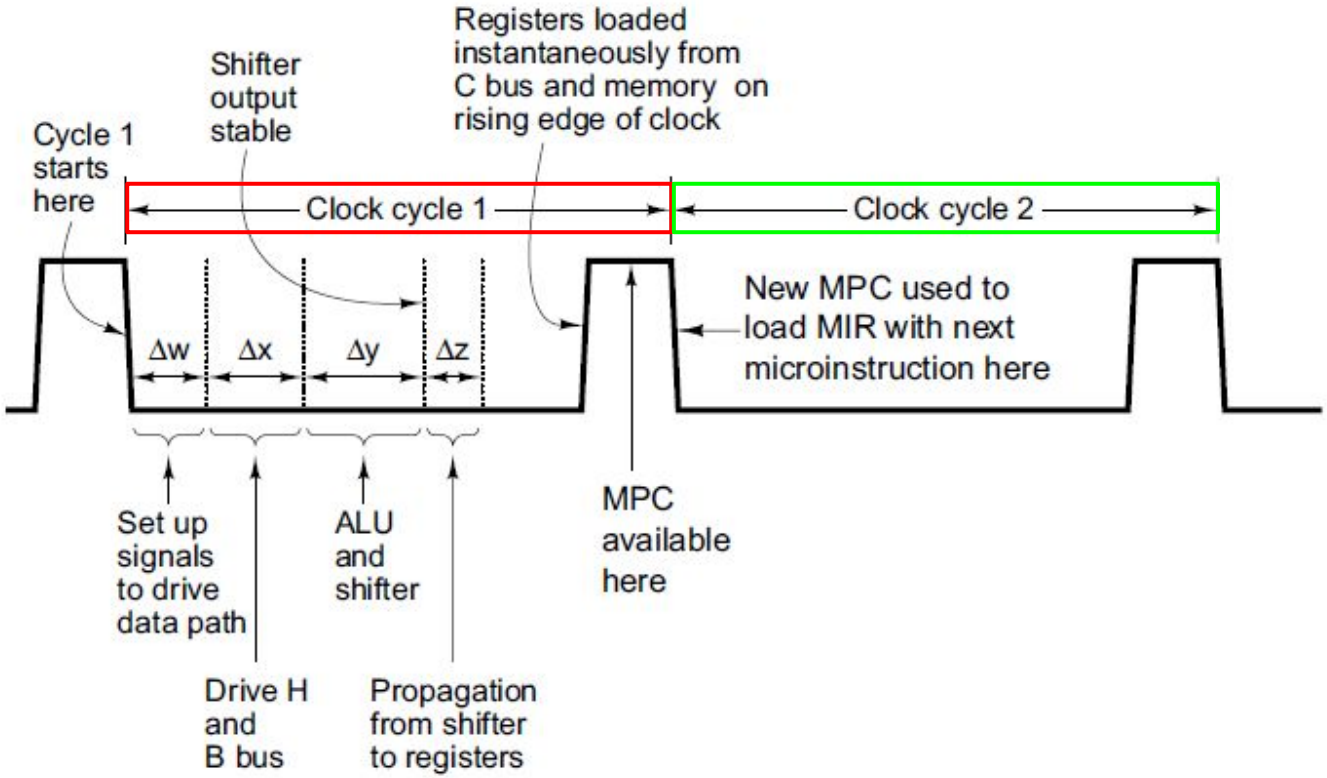

Acciones para resolver la instrucción **IADD**

**MAR = SP = SP - 1; rd;**

**H = TOS;**

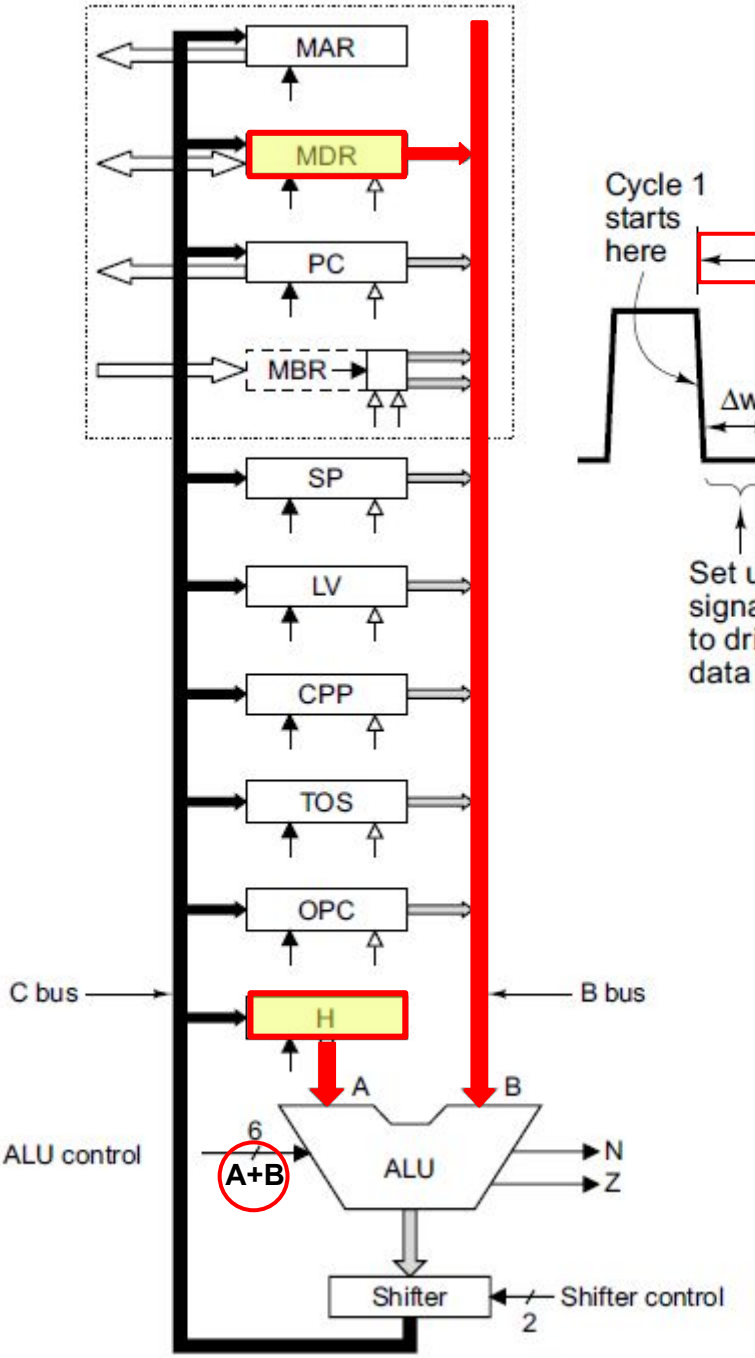
**MDR = TOS = H + MDR; wr;**

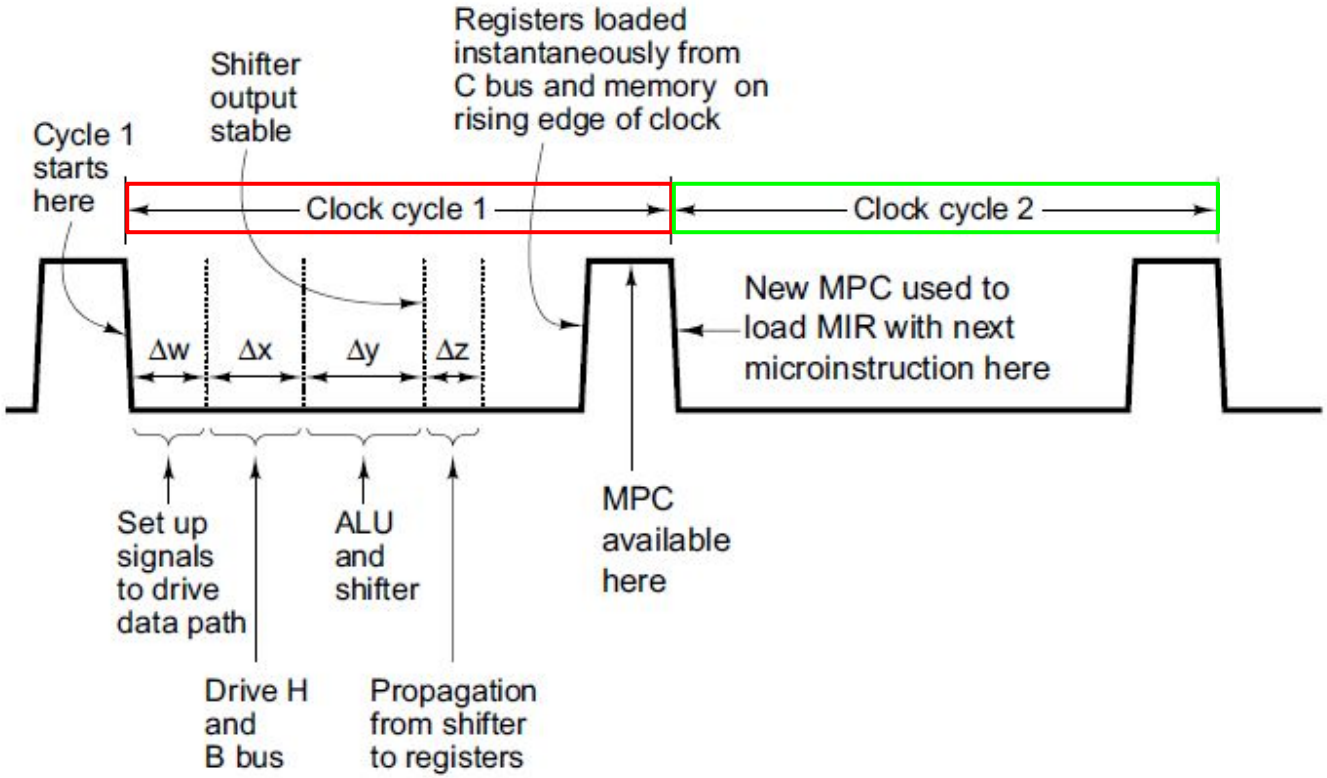



Acciones para resolver la instrucción **IADD**

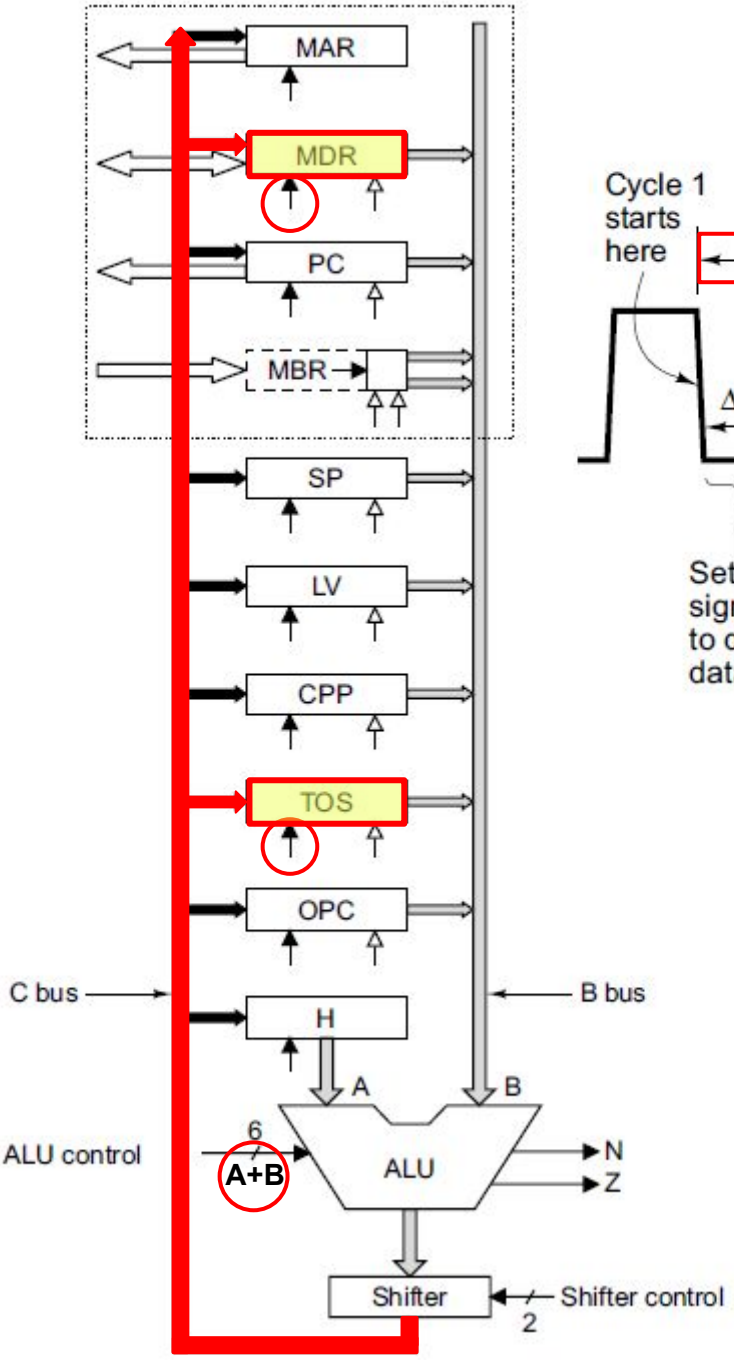
**MAR = SP = SP - 1; rd;**
**H = TOS;**
**MDR = TOS = H + MDR; wr;**

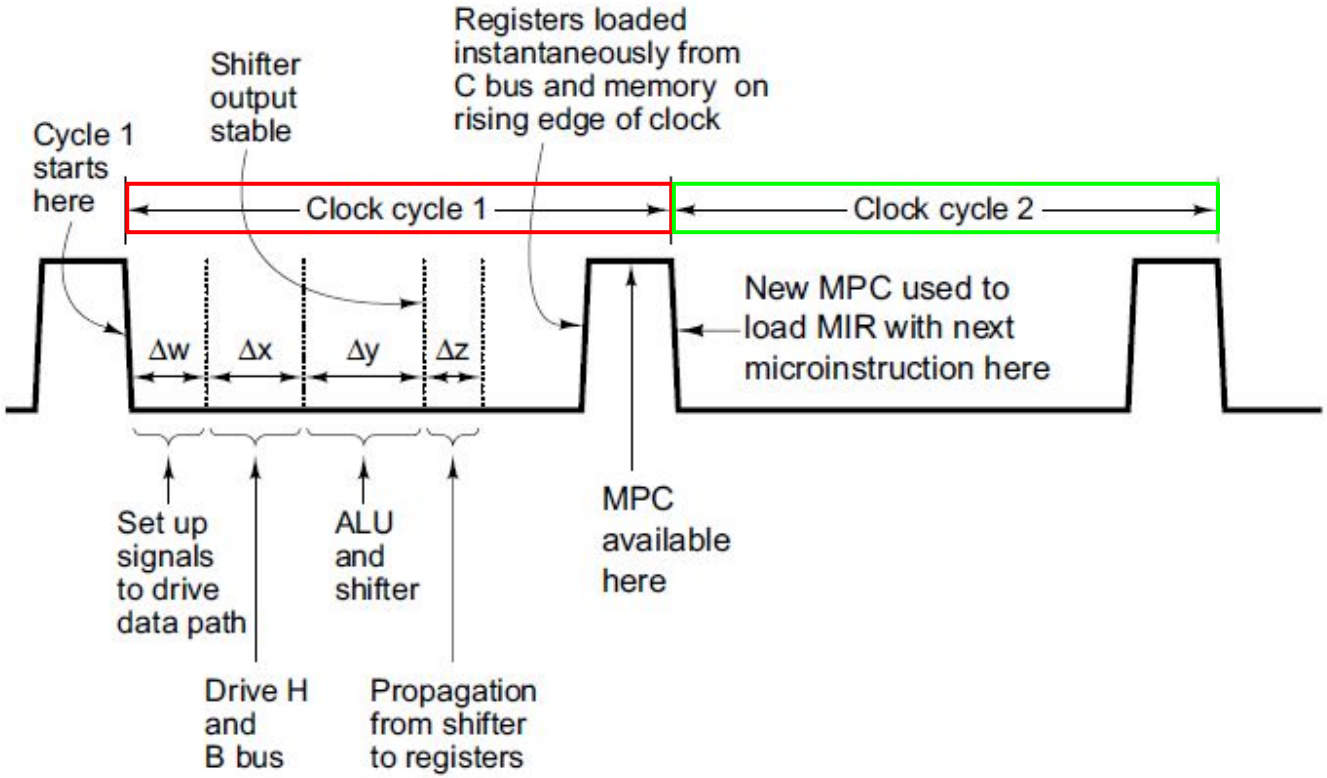



Acciones para resolver la instrucción **IADD**

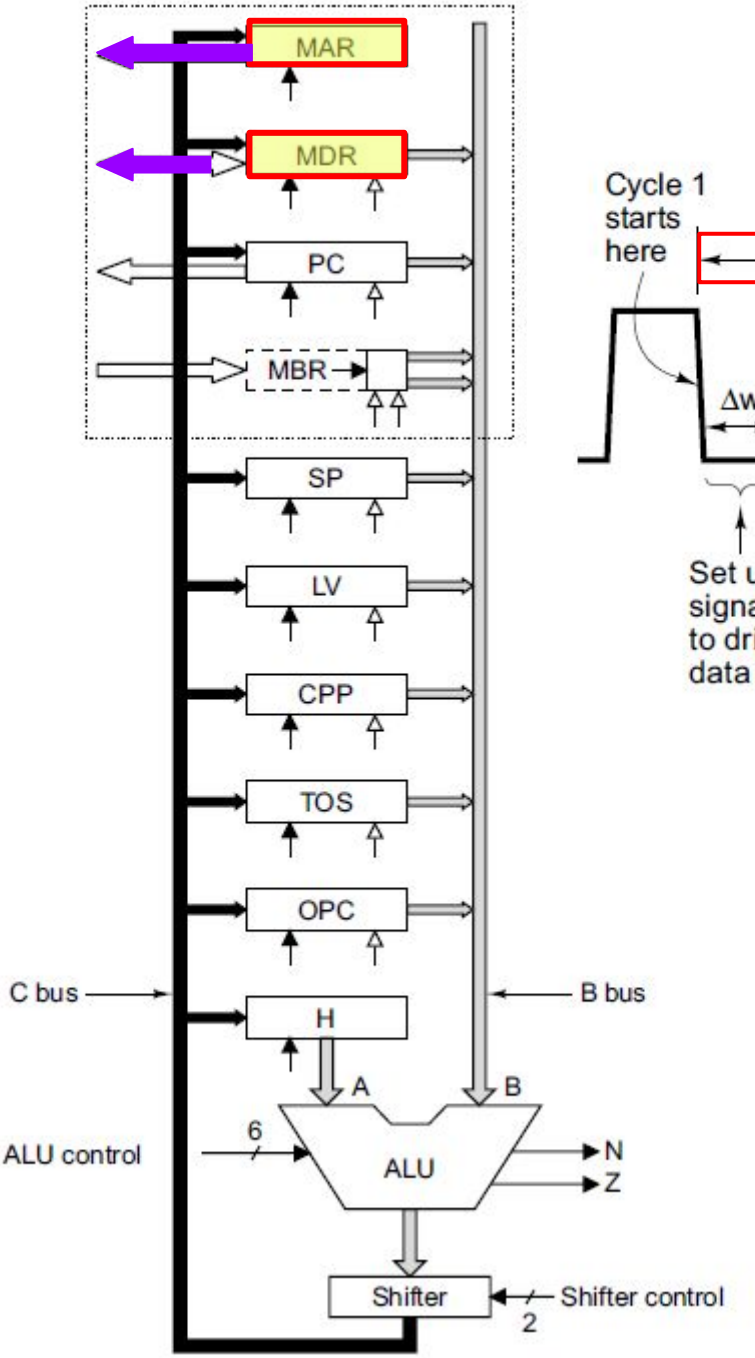
**MAR = SP = SP - 1; rd;**
**H = TOS;**
**MDR = TOS = H + MDR; wr;**

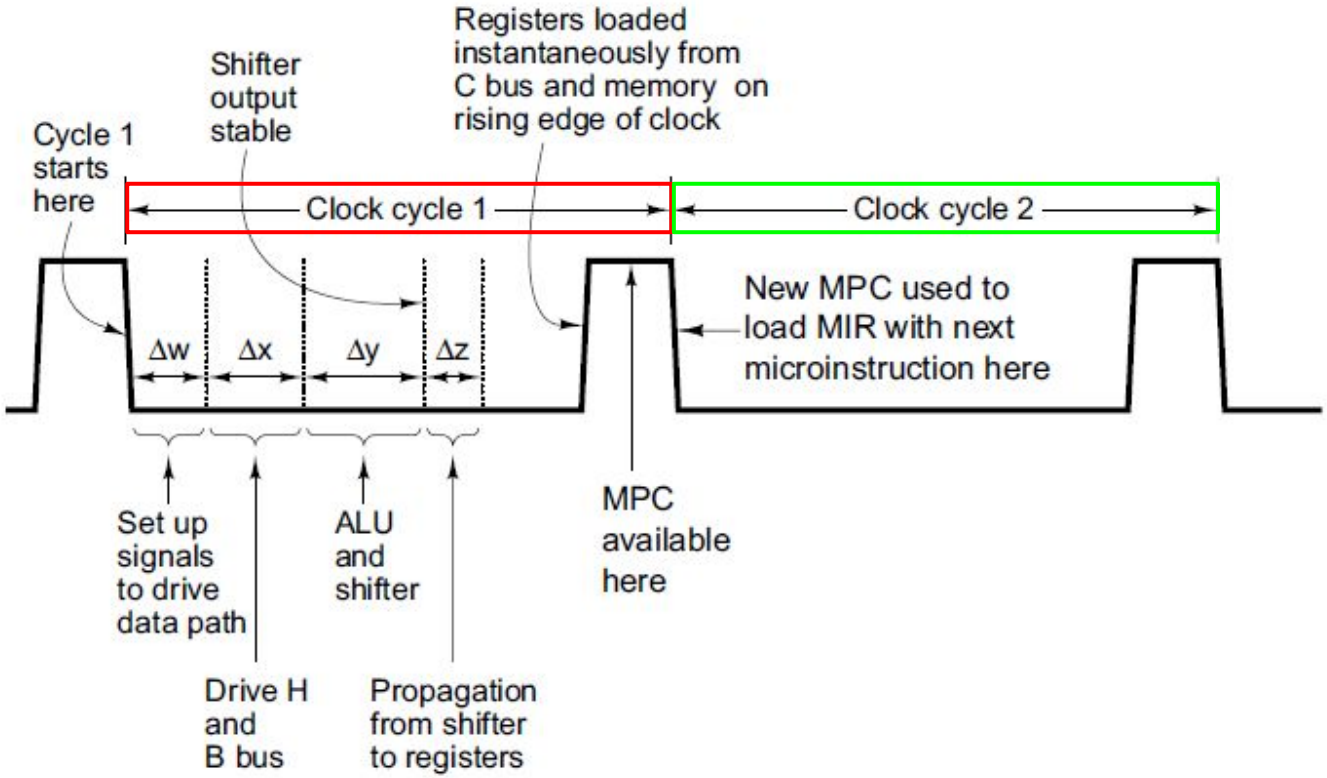



Acciones para resolver la instrucción **IADD**

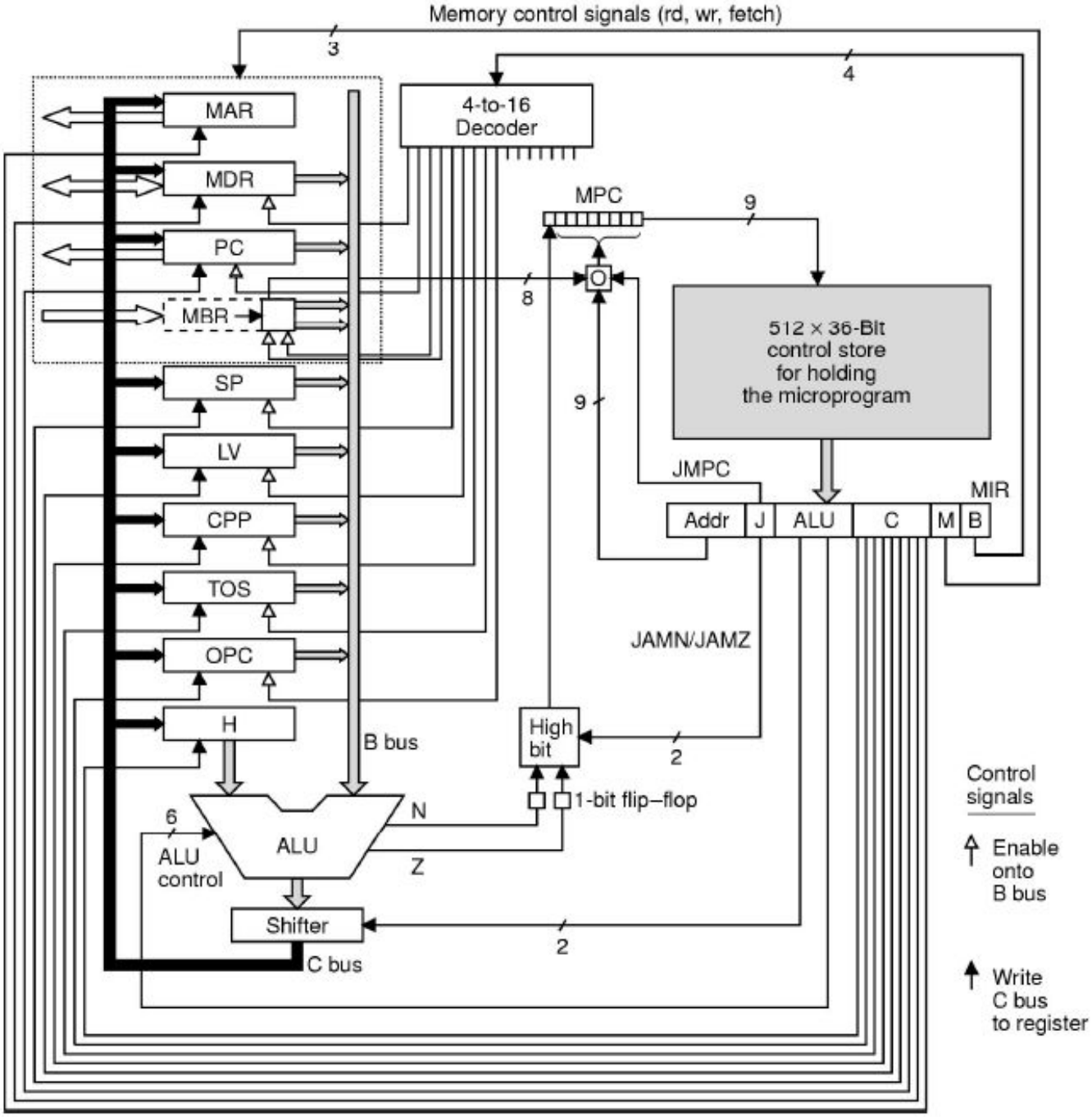
**MAR = SP = SP - 1; rd;**
**H = TOS;**
**MDR = TOS = H + MDR; wr;**

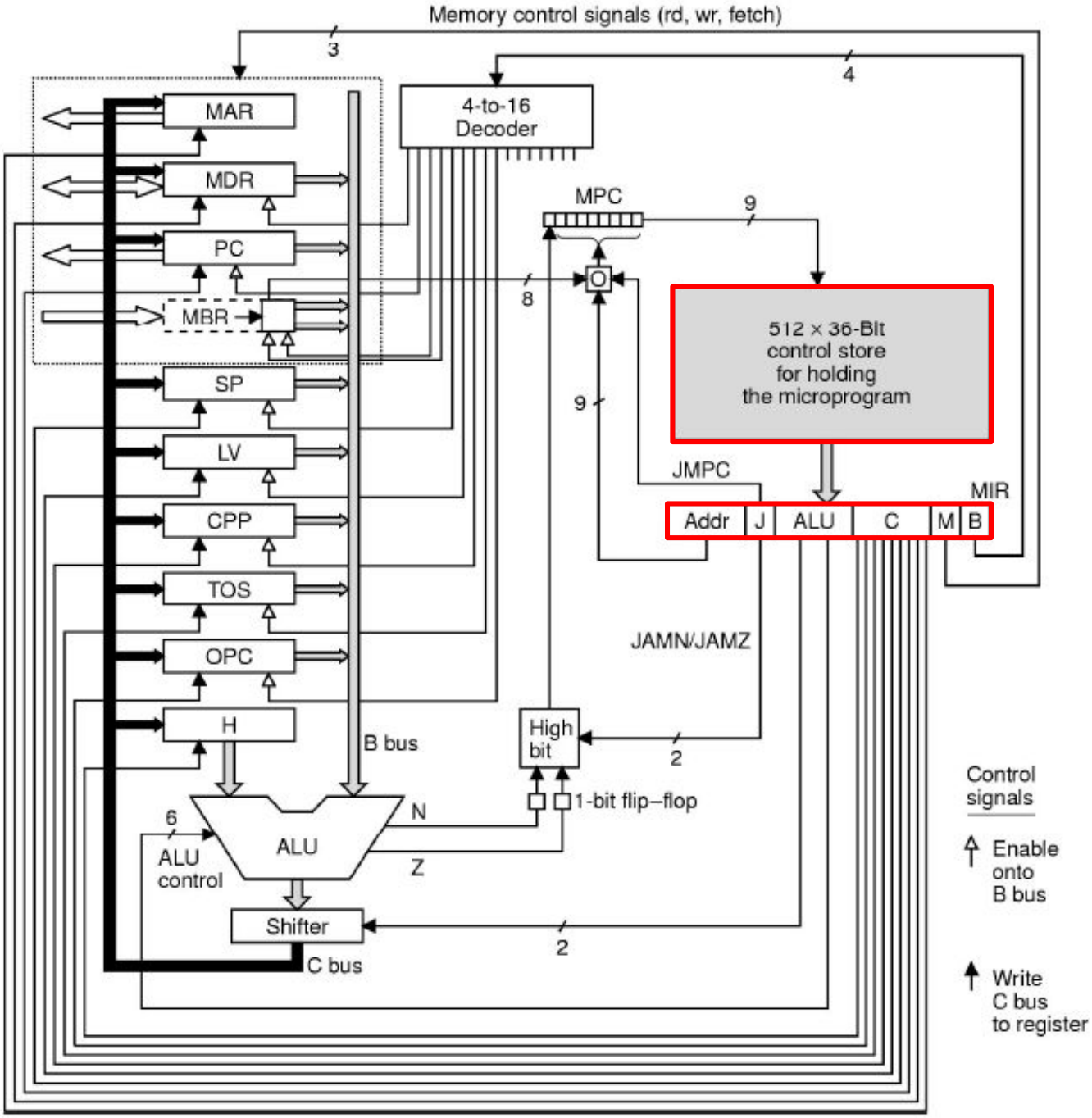


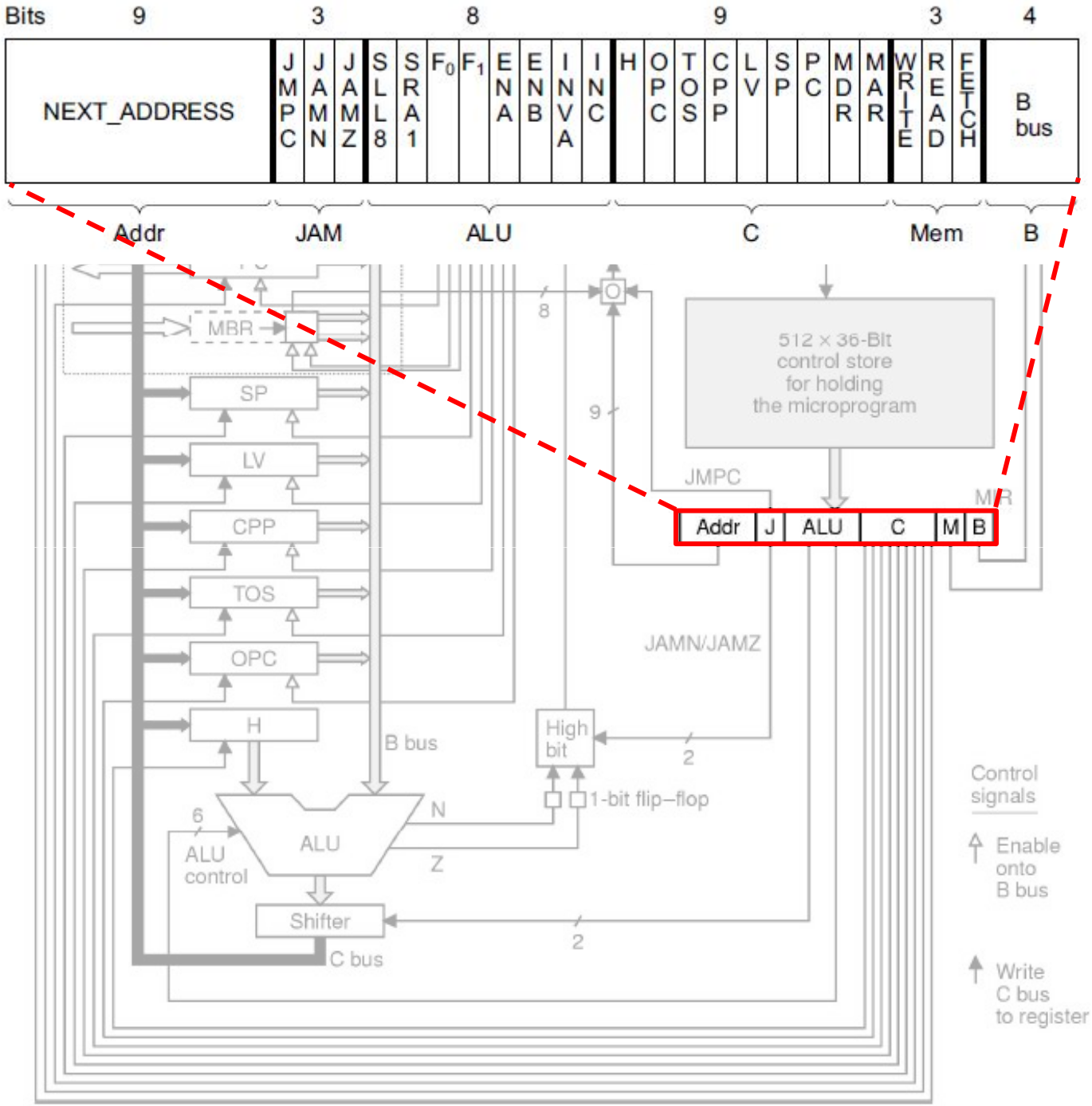



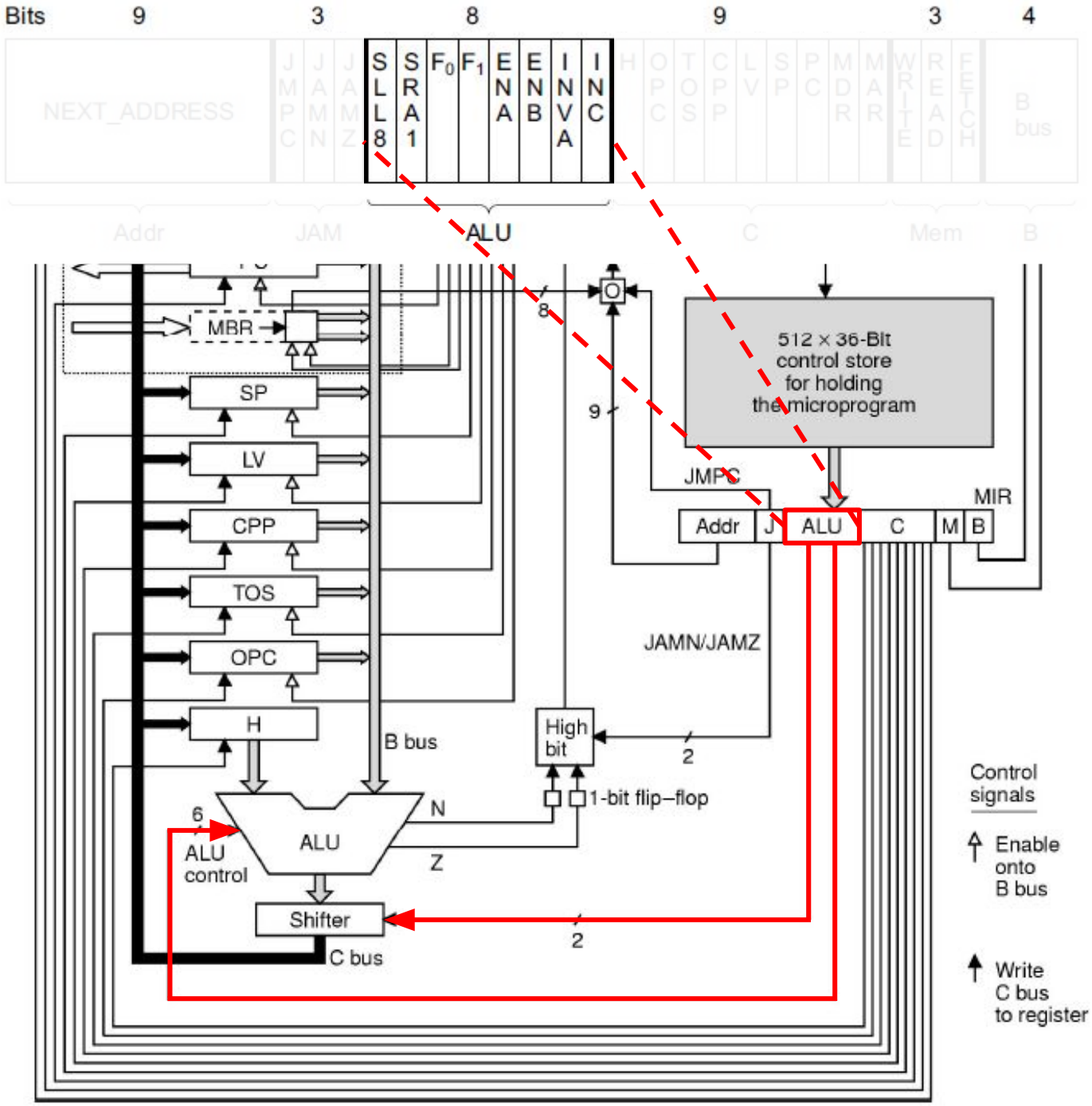

Acciones para resolver la instrucción **IADD**

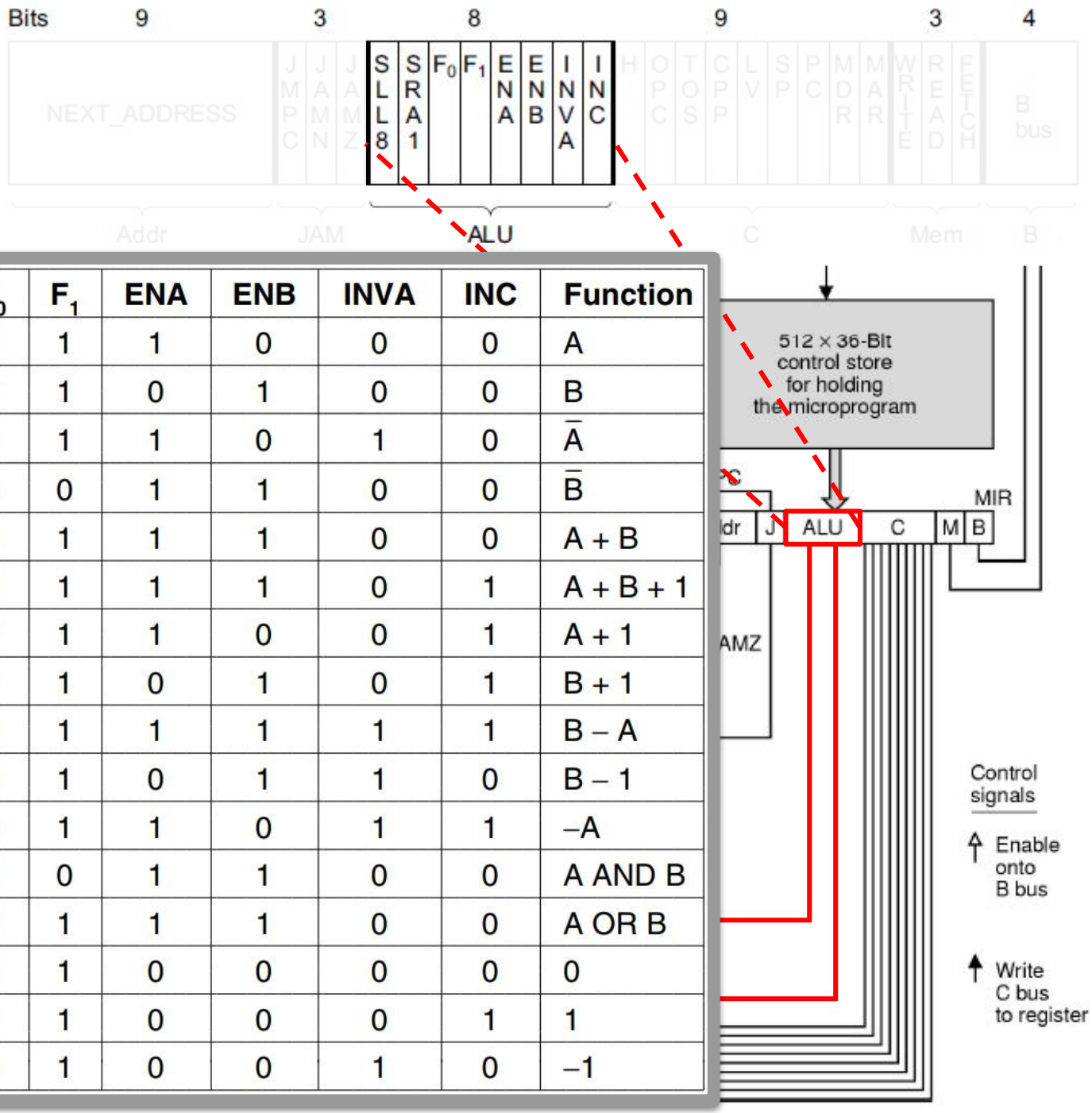
**MAR = SP = SP - 1; rd;**
**H = TOS;**
**MDR = TOS = H + MDR; wr;**


Memory control signals (rd, wr, fetch)
3
4
MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
H
4-to-16
Decoder
MPC
9
8
512 × 36-Bit
control store
for holding
the microprogram
9
JMPC
MIR
Addr
J
ALU
C
M
B
JAMN/JAMZ
High
bit
2
1-bit flip–flop
B bus
6
ALU
control
ALU
N
Z
Shifter
2
C bus
Control
signals
Enable
onto
B bus
Write
C bus
to register

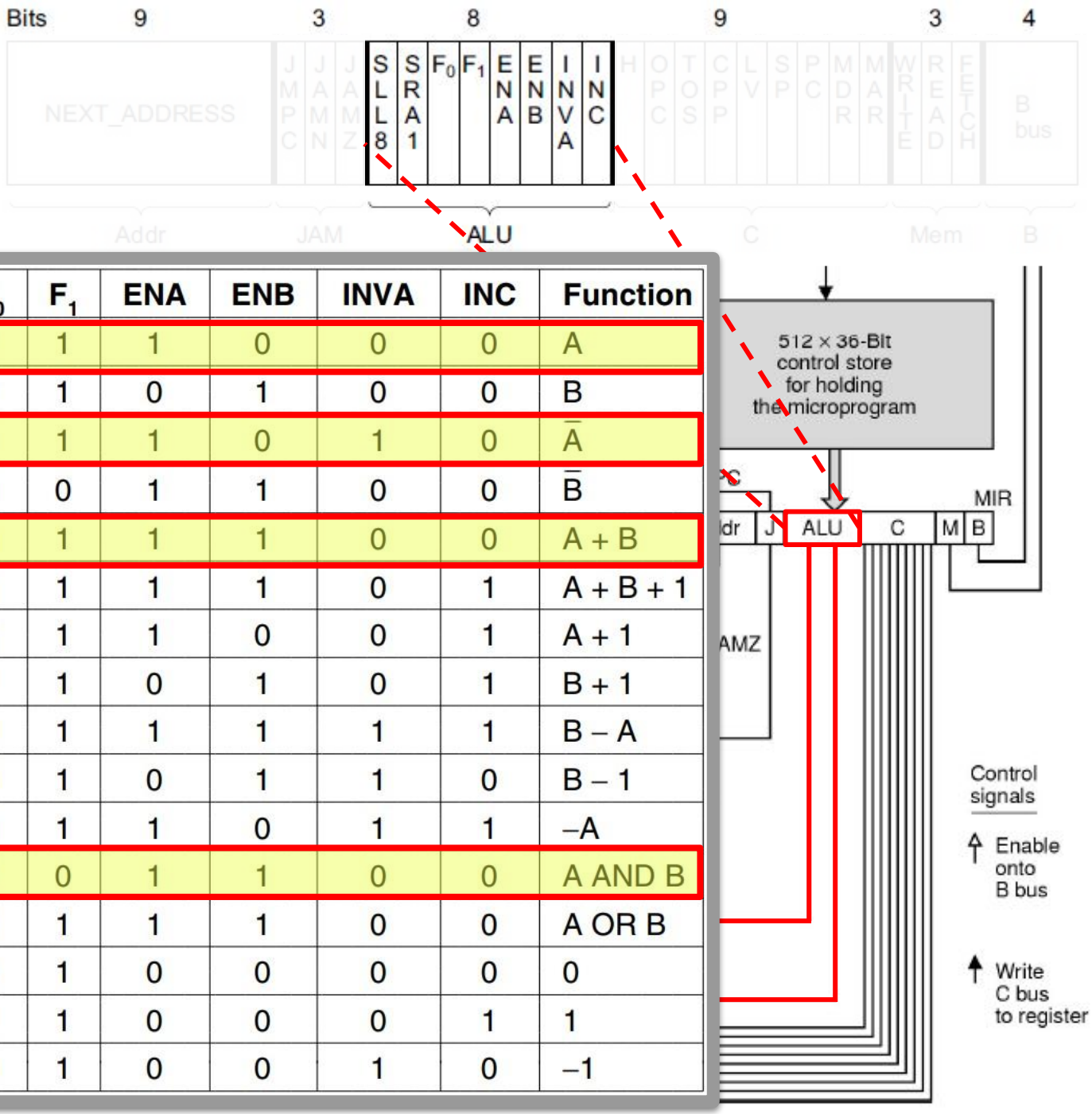

Memory control signals (rd, wr, fetch)
3
4
MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
H
4-to-16
Decoder
MPC
9
8
512 × 36-Bit
control store
for holding
the microprogram
9
JMPC
MIR
Addr
J
ALU
C
M
B
JAMN/JAMZ
High
bit
2
1-bit flip–flop
B bus
6
ALU
control
ALU
N
Z
Shifter
2
C bus
Control
signals
Enable
onto
B bus
Write
C bus
to register


Bits 9 3 8 9 3 4
NEXT_ADDRESS
JMPC JAMN JAMZ SLL8 SRA1 F0 F1 ENA ENB INVA INC H OPC TOS CPP LV SP PC MDR MAR WRITE READ FETCH
B bus
Addr JAM ALU C Mem B
MBR
SP
LV
CPP
TOS
OPC
H
B bus
8
9
512 × 36-Bit control store for holding the microprogram
JMPC
MIR
Addr J ALU C M B
JAMN/JAMZ
High bit
2
1-bit flip–flop
N
Z
6
ALU control
ALU
Shifter
2
C bus
Control signals
Enable onto B bus
Write C bus to register

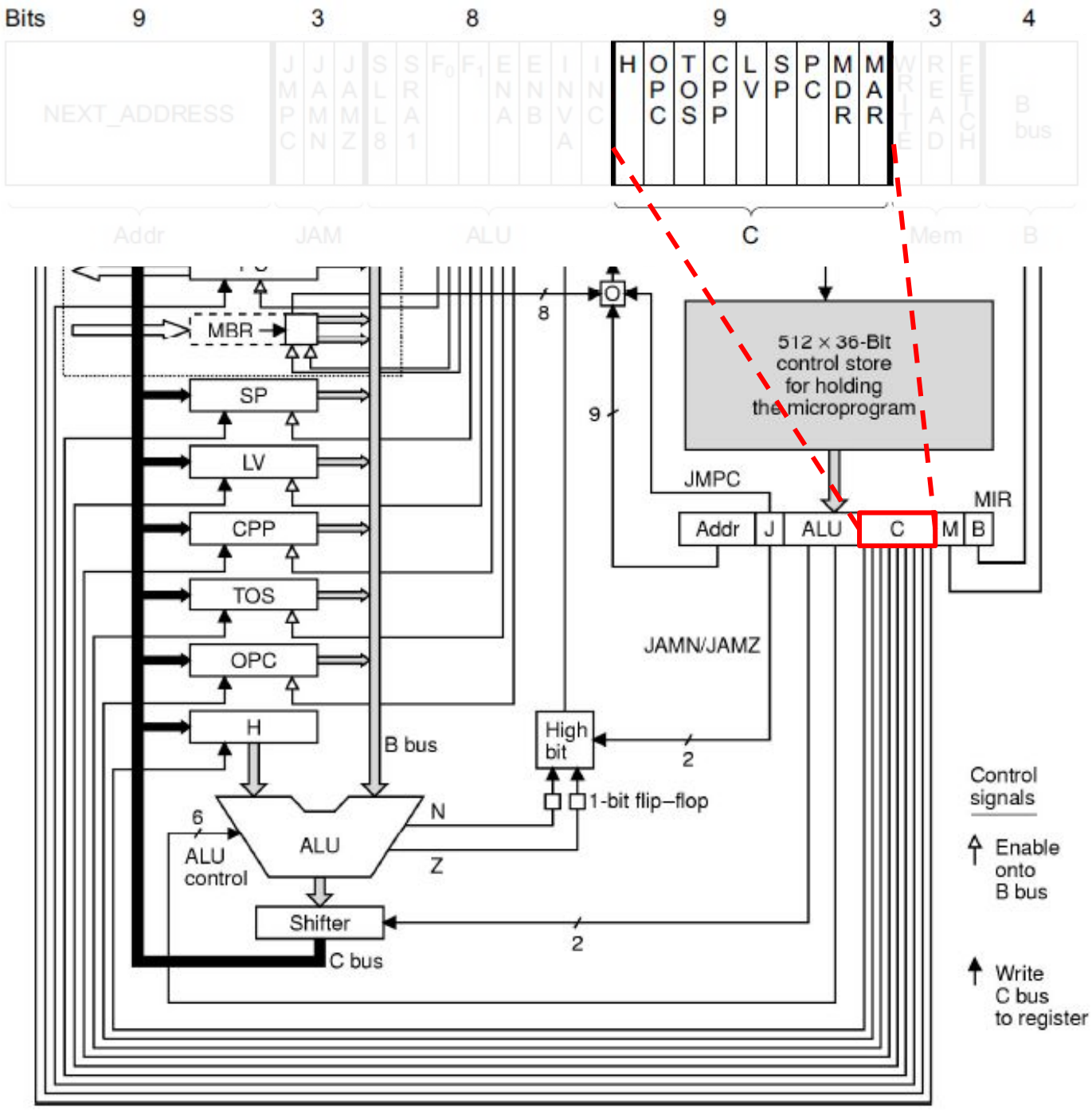

Bits 9 3 8 9 3 4
NEXT_ADDRESS
JMPC JAMN JAMZ
SLL8 SRA1 F0 F1 ENA ENB INVA INC
H OPC TOS CPP LV SP PC MDR MAR
WRITE READ FETCH
B bus
Addr JAM ALU C Mem B
MBR
SP
LV
CPP
TOS
OPC
H
8
9
512 × 36-Bit control store for holding the microprogram
JMPC
MIR
Addr J ALU C M B
JAMN/JAMZ
B bus
High bit
2
1-bit flip–flop
6
ALU control
ALU
N
Z
Shifter
2
C bus
Control signals
Enable onto B bus
Write C bus to register

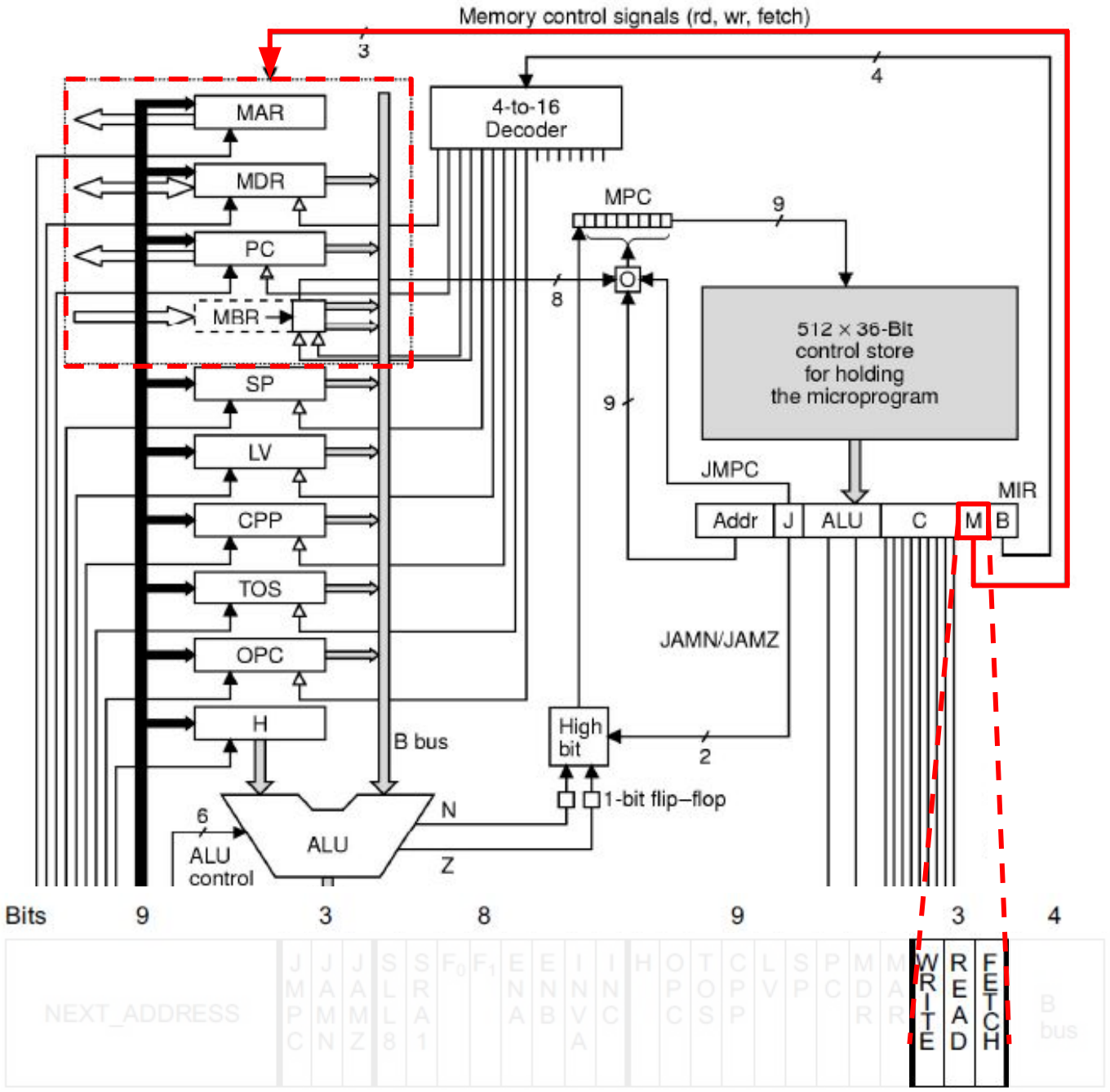


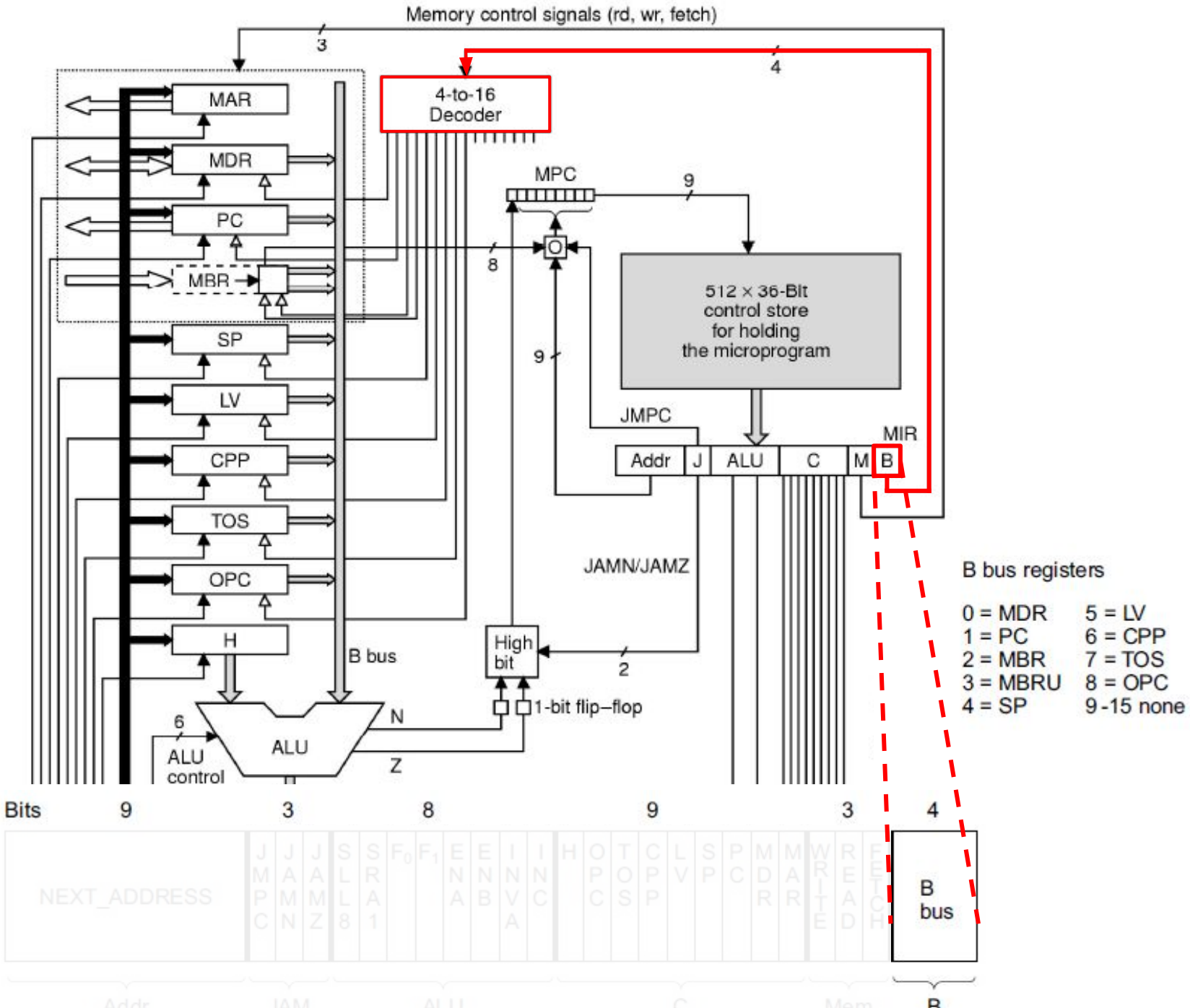

| $F_0$ | $F_1$ | ENA | ENB | INVA | INC | Function |
|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 0 | A |
| 0 | 1 | 0 | 1 | 0 | 0 | B |
| 0 | 1 | 1 | 0 | 1 | 0 | $\overline{A}$ |
| 1 | 0 | 1 | 1 | 0 | 0 | $\overline{B}$ |
| 1 | 1 | 1 | 1 | 0 | 0 | A + B |
| 1 | 1 | 1 | 1 | 0 | 1 | A + B + 1 |
| 1 | 1 | 1 | 0 | 0 | 1 | A + 1 |
| 1 | 1 | 0 | 1 | 0 | 1 | B + 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | B – A |
| 1 | 1 | 0 | 1 | 1 | 0 | B – 1 |
| 1 | 1 | 1 | 0 | 1 | 1 | –A |
| 0 | 0 | 1 | 1 | 0 | 0 | A AND B |
| 0 | 1 | 1 | 1 | 0 | 0 | A OR B |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 | 0 | –1 |

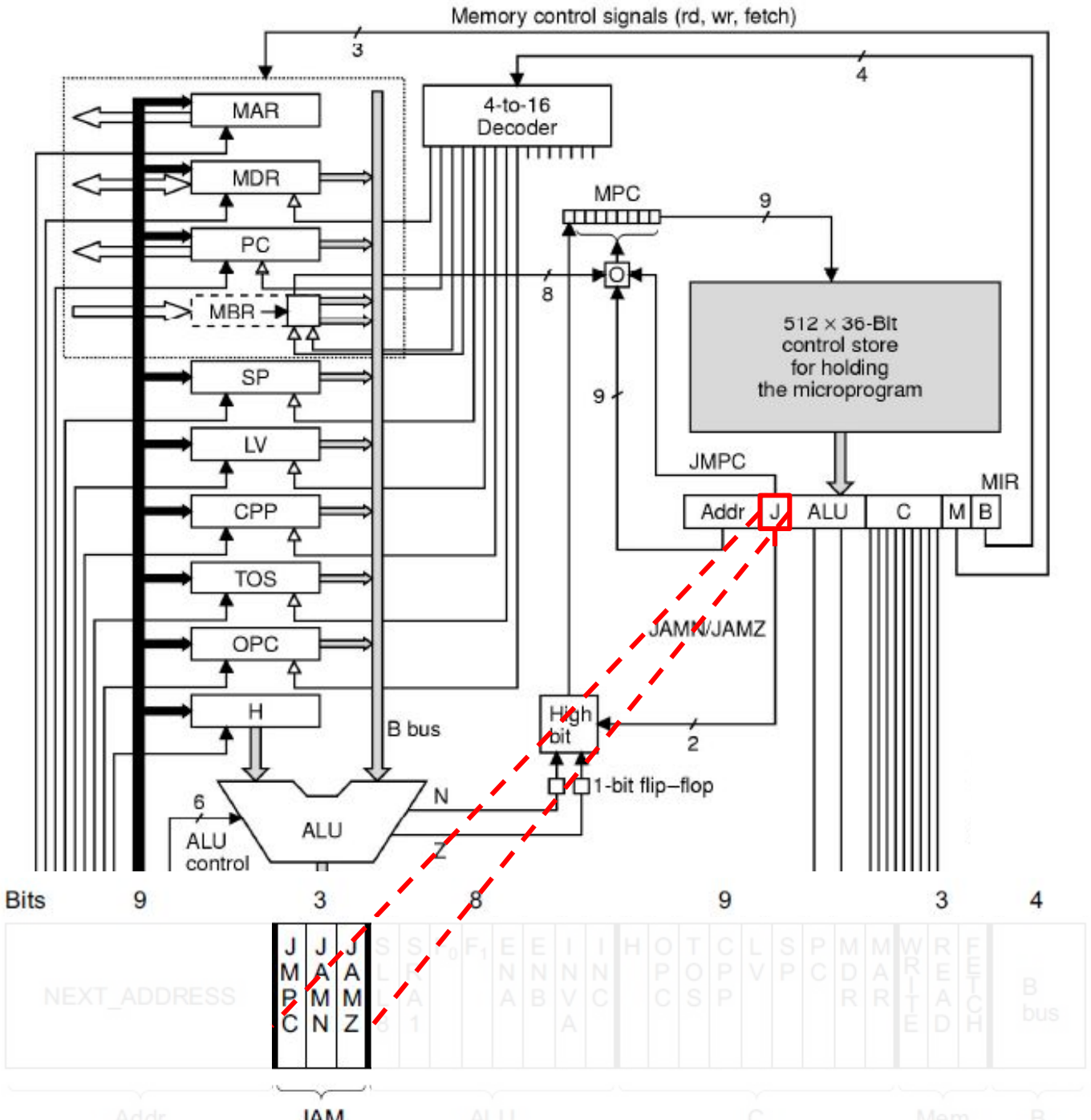


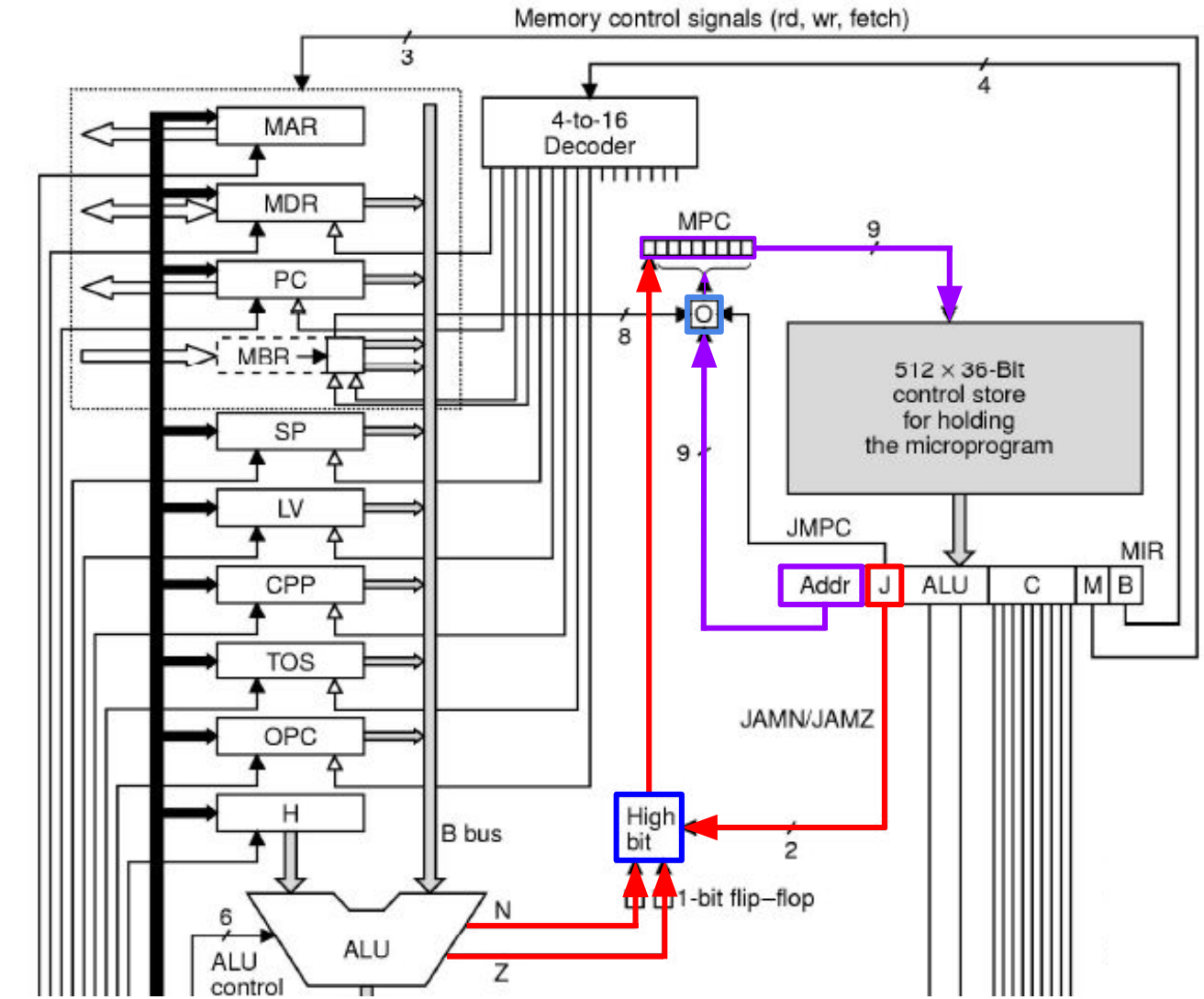

| $F_0$ | $F_1$ | ENA | ENB | INVA | INC | Function |
|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 0 | A |
| 0 | 1 | 0 | 1 | 0 | 0 | B |
| 0 | 1 | 1 | 0 | 1 | 0 | $\overline{A}$ |
| 1 | 0 | 1 | 1 | 0 | 0 | $\overline{B}$ |
| 1 | 1 | 1 | 1 | 0 | 0 | A + B |
| 1 | 1 | 1 | 1 | 0 | 1 | A + B + 1 |
| 1 | 1 | 1 | 0 | 0 | 1 | A + 1 |
| 1 | 1 | 0 | 1 | 0 | 1 | B + 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | B – A |
| 1 | 1 | 0 | 1 | 1 | 0 | B – 1 |
| 1 | 1 | 1 | 0 | 1 | 1 | –A |
| 0 | 0 | 1 | 1 | 0 | 0 | A AND B |
| 0 | 1 | 1 | 1 | 0 | 0 | A OR B |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 | 0 | –1 |

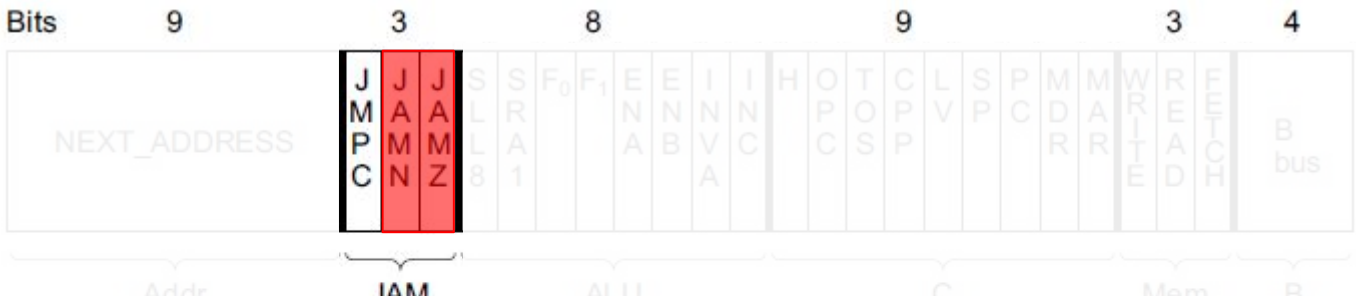

Bits 9 3 8 9 3 4
NEXT_ADDRESS
JMPC JAMN JAMZ
SLL8 SRA1 F0 F1 ENA ENB INVA INC
H OPC TOS CPP LV SP PC MDR MAR
WRITE READ FETCH
B bus
Addr JAM ALU C Mem B
MBR
SP
LV
CPP
TOS
OPC
H
8
9
512 × 36-Bit control store for holding the microprogram
JMPC
MIR
Addr J ALU C M B
JAMN/JAMZ
B bus
High bit
2
1-bit flip–flop
6
ALU control
ALU
N
Z
Shifter
2
C bus
Control signals
Enable onto B bus
Write C bus to register

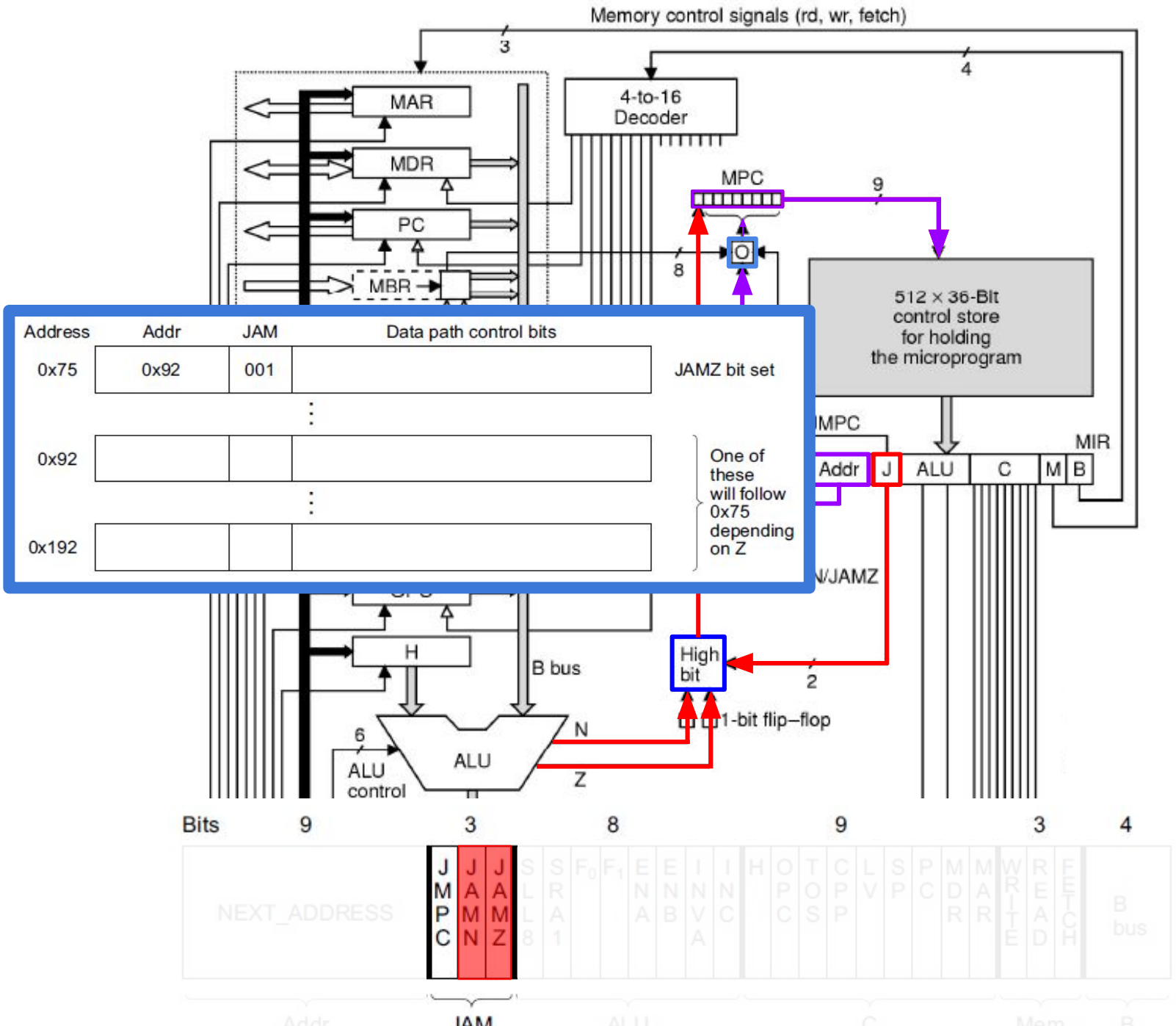

Memory control signals (rd, wr, fetch)
3
4
MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
H
4-to-16
Decoder
MPC
9
8
512 × 36-Bit
control store
for holding
the microprogram
9
JMPC
MIR
Addr
J
ALU
C
M
B
JAMN/JAMZ
High
bit
2
1-bit flip–flop
B bus
6
ALU
control
ALU
N
Z
Bits
9
3
8
9
3
4
NEXT_ADDRESS
JMPC
JAMN
JAMZ
SLL8
SRA1
F0
F1
ENA
ENB
INVA
INC
H
OPC
TOS
CPP
LV
SP
PC
MDR
MAR
WRITE
READ
FETCH
B bus
Addr
JAM
ALU
C
Mem
B

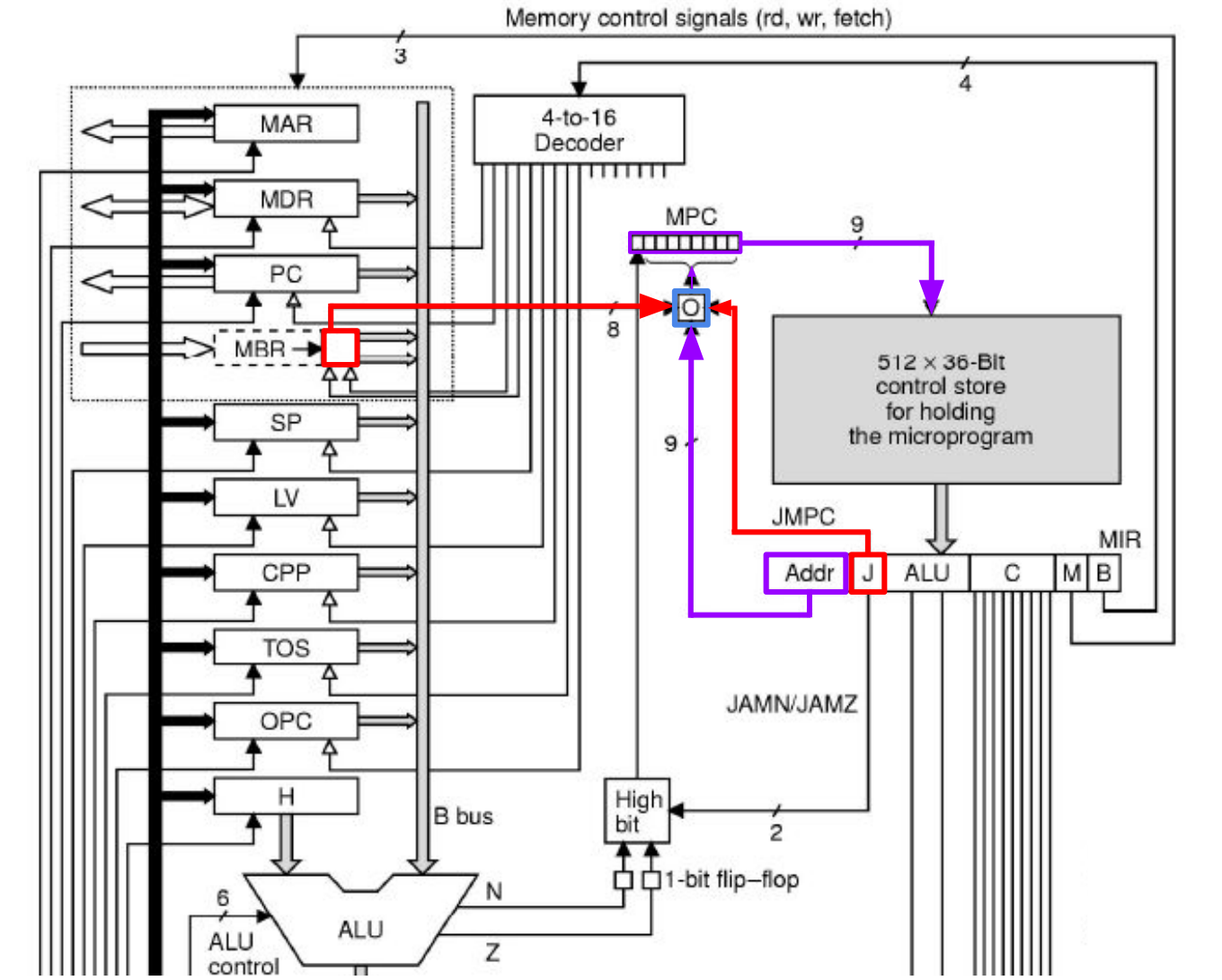

Memory control signals (rd, wr, fetch)
3
4
4-to-16
Decoder
MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
H
MPC
9
8
9
512 × 36-Bit
control store
for holding
the microprogram
JMPC
MIR
Addr
J
ALU
C
M
B
JAMN/JAMZ
B bus
High
bit
2
1-bit flip–flop
6
ALU
control
ALU
N
Z
B bus registers
0 = MDR
1 = PC
2 = MBR
3 = MBRU
4 = SP
5 = LV
6 = CPP
7 = TOS
8 = OPC
9-15 none
Bits
9
3
8
9
3
4
NEXT_ADDRESS
B
bus

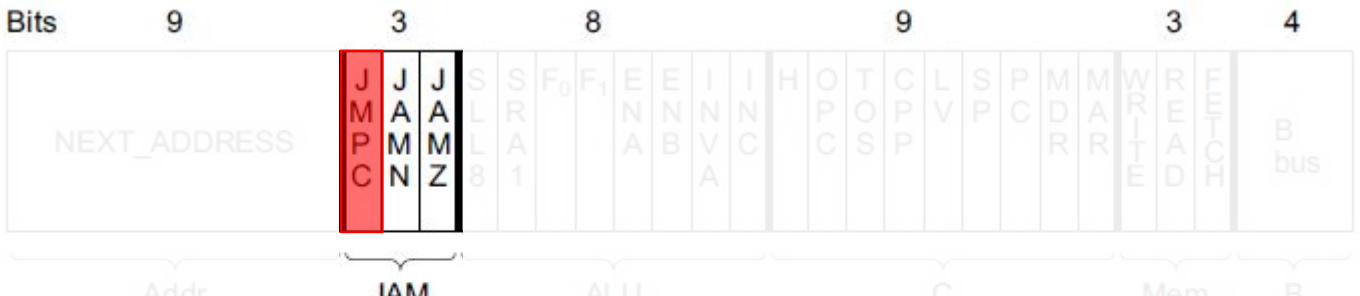

Memory control signals (rd, wr, fetch)
3
4
MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
H
4-to-16
Decoder
MPC
9
8
512 × 36-Bit
control store
for holding
the microprogram
9
JMPC
MIR
Addr
J
ALU
C
M
B
JAMN/JAMZ
High
bit
2
1-bit flip–flop
B bus
6
ALU
control
ALU
N
Z
Bits
9
3
8
9
3
4
JMPC
JAMN
JAMZ
NEXT_ADDRESS
B
bus

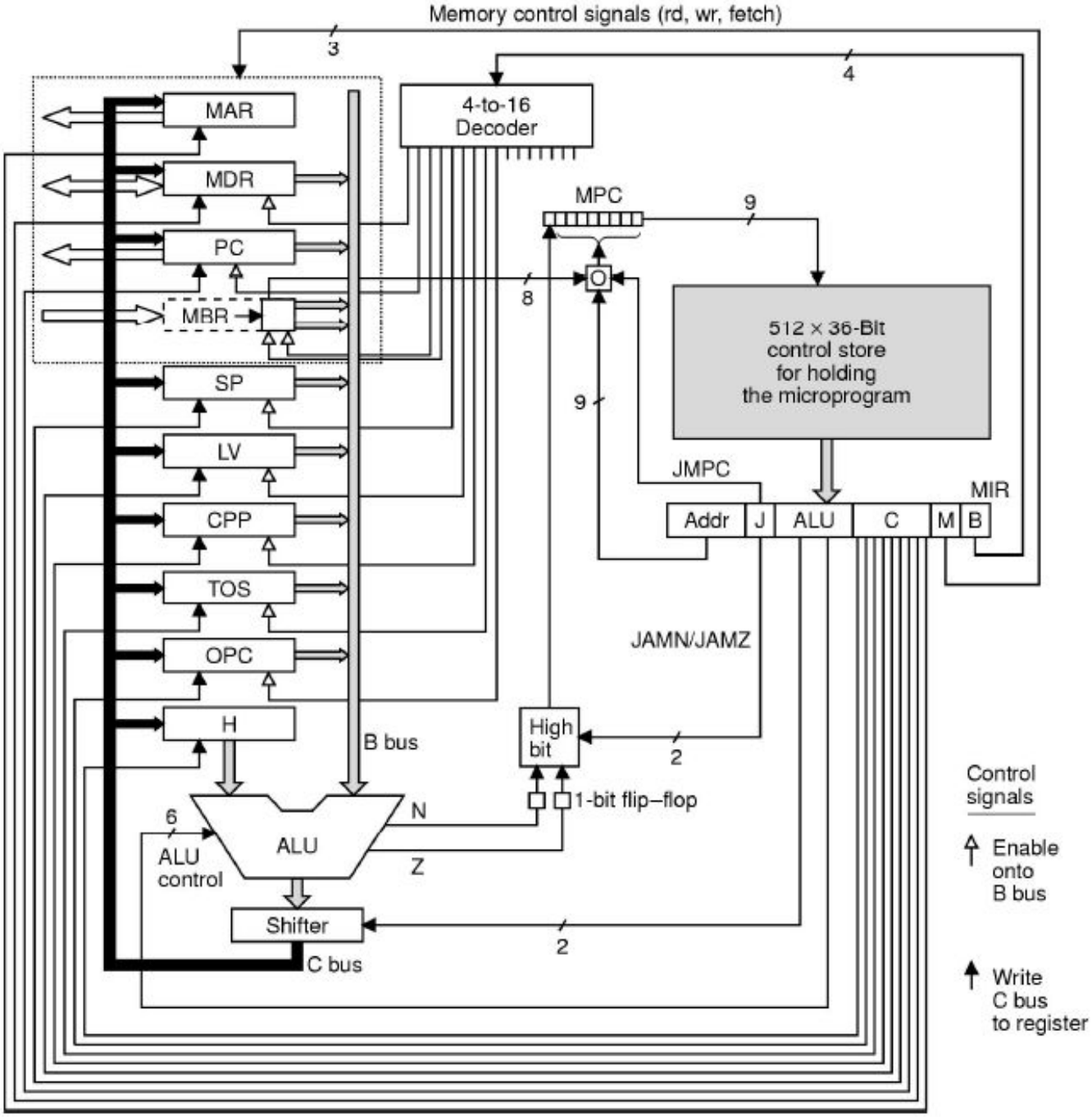

Memory control signals (rd, wr, fetch)
3
4
MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
H
4-to-16
Decoder
MPC
9
8
O
512 × 36-Bit
control store
for holding
the microprogram
9
JMPC
MIR
Addr
J
ALU
C
M
B
JAMN/JAMZ
B bus
High
bit
2
1-bit flip–flop
N
Z
6
ALU
control
ALU

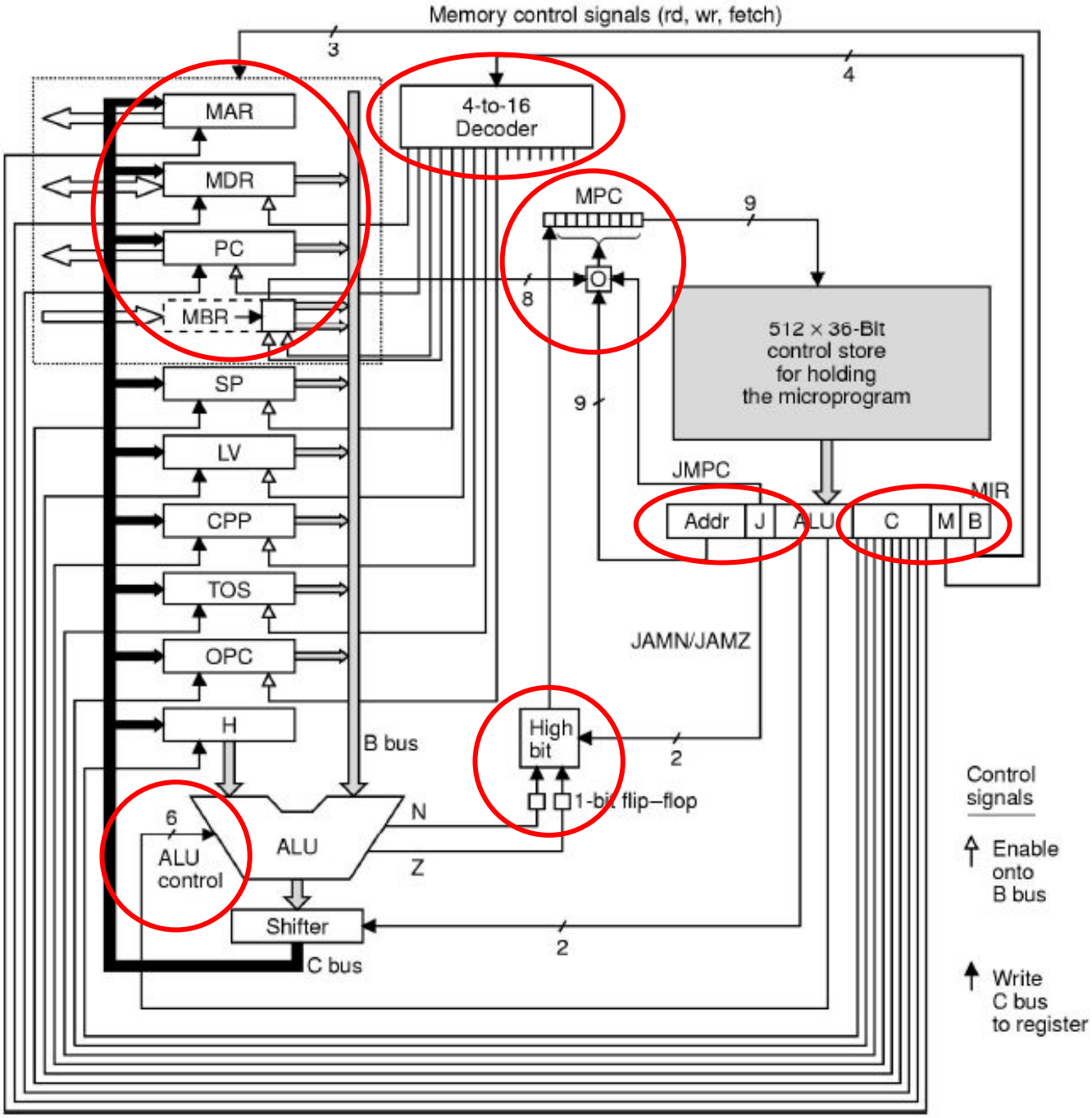

Bits
9
3
8
9
3
4
NEXT_ADDRESS
JMPC
JAMN
JAMZ
B bus
JAM


Memory control signals (rd, wr, fetch)
3
4
MAR
MDR
PC
MBR
4-to-16
Decoder
MPC
9
8
512 × 36-Bit
control store
for holding
the microprogram
Address
Addr
JAM
Data path control bits
0x75
0x92
001
JAMZ bit set
0x92
0x192
One of
these
will follow
0x75
depending
on Z
MPC
MIR
Addr
J
ALU
C
M
B
N/JAMZ
H
B bus
High
bit
2
1-bit flip–flop
6
ALU
control
ALU
N
Z
Bits
9
3
8
9
3
4
NEXT_ADDRESS
JMPC
JAMN
JAMZ
B
bus
JAM
ALU
C
Mem
B


Memory control signals (rd, wr, fetch)
3
4
MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
H
4-to-16
Decoder
MPC
9
8
O
512 × 36-Bit
control store
for holding
the microprogram
9
JMPC
MIR
Addr
J
ALU
C
M
B
JAMN/JAMZ
B bus
High
bit
2
1-bit flip–flop
6
ALU
control
ALU
N
Z


Bits
9
3
8
9
3
4
NEXT_ADDRESS
JMPC
JAMN
JAMZ
B bus
Addr
JAM
ALU
C
Mem
B


Memory control signals (rd, wr, fetch)
3
4
MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
H
4-to-16
Decoder
MPC
9
8
512 × 36-Bit
control store
for holding
the microprogram
9
JMPC
MIR
Addr
J
ALU
C
M
B
JAMN/JAMZ
High
bit
2
1-bit flip–flop
B bus
6
ALU
control
ALU
N
Z
Shifter
2
C bus
Control
signals
Enable
onto
B bus
Write
C bus
to register


Memory control signals (rd, wr, fetch)
3
4
MAR
MDR
PC
MBR
SP
LV
CPP
TOS
OPC
H
4-to-16
Decoder
MPC
9
8
512 × 36-Bit
control store
for holding
the microprogram
9
JMPC
MIR
Addr
J
ALU
C
M
B
JAMN/JAMZ
High
bit
2
1-bit flip–flop
B bus
6
ALU
control
ALU
N
Z
Shifter
2
C bus
Control
signals
Enable
onto
B bus
Write
C bus
to register

# Integer Java Virtual Machine

## Microprograma (parte 1)

| Label | Operations | Comments |
|---|---|---|
| Main1 | PC = PC + 1; fetch; goto (MBR) | MBR holds opcode; get next byte; dispatch |
| nop1 | goto Main1 | Do nothing |
| iadd1 | MAR = SP = SP – 1; rd | Read in next-to-top word on stack |
| iadd2 | H = TOS | H = top of stack |
| iadd3 | MDR = TOS = MDR + H; wr; goto Main1 | Add top two words; write to top of stack |
| isub1 | MAR = SP = SP – 1; rd | Read in next-to-top word on stack |
| isub2 | H = TOS | H = top of stack |
| isub3 | MDR = TOS = MDR – H; wr; goto Main1 | Do subtraction; write to top of stack |
| iand1 | MAR = SP = SP – 1; rd | Read in next-to-top word on stack |
| iand2 | H = TOS | H = top of stack |
| iand3 | MDR = TOS = MDR AND H; wr; goto Main1 | Do AND; write to new top of stack |
| ior1 | MAR = SP = SP – 1; rd | Read in next-to-top word on stack |
| ior2 | H = TOS | H = top of stack |
| ior3 | MDR = TOS = MDR OR H; wr; goto Main1 | Do OR; write to new top of stack |
| dup1 | MAR = SP = SP + 1 | Increment SP and copy to MAR |
| dup2 | MDR = TOS; wr; goto Main1 | Write new stack word |
| pop1 | MAR = SP = SP – 1; rd | Read in next-to-top word on stack |
| pop2 | | Wait for new TOS to be read from memory |
| pop3 | TOS = MDR; goto Main1 | Copy new word to TOS |
| swap1 | MAR = SP – 1; rd | Set MAR to SP – 1; read 2nd word from stack |
| swap2 | MAR = SP | Set MAR to top word |
| swap3 | H = MDR; wr | Save TOS in H; write 2nd word to top of stack |
| swap4 | MDR = TOS | Copy old TOS to MDR |
| swap5 | MAR = SP – 1; wr | Set MAR to SP – 1; write as 2nd word on stack |
| swap6 | TOS = H; goto Main1 | Update TOS |
| bipush1 | SP = MAR = SP + 1 | MBR = the byte to push onto stack |
| bipush2 | PC = PC + 1; fetch | Increment PC, fetch next opcode |
| bipush3 | MDR = TOS = MBR; wr; goto Main1 | Sign-extend constant and push on stack |

# Integer Java Virtual Machine

## Microprograma (parte 2)

| Label | Operations | Comments |
|---|---|---|
| iload1 | H = LV | MBR contains index; copy LV to H |
| iload2 | MAR = MBRU + H; rd | MAR = address of local variable to push |
| iload3 | MAR = SP = SP + 1 | SP points to new top of stack; prepare write |
| iload4 | PC = PC + 1; fetch; wr | Inc PC; get next opcode; write top of stack |
| iload5 | TOS = MDR; goto Main1 | Update TOS |
| istore1 | H = LV | MBR contains index; Copy LV to H |
| istore2 | MAR = MBRU + H | MAR = address of local variable to store into |
| istore3 | MDR = TOS; wr | Copy TOS to MDR; write word |
| istore4 | SP = MAR = SP – 1; rd | Read in next-to-top word on stack |
| istore5 | PC = PC + 1; fetch | Increment PC; fetch next opcode |
| istore6 | TOS = MDR; goto Main1 | Update TOS |
| wide1 | PC = PC + 1; fetch; goto (MBR OR 0x100) | Multiway branch with high bit set |
| wide_iload1 | PC = PC + 1; fetch | MBR contains 1st index byte; fetch 2nd |
| wide_iload2 | H = MBRU << 8 | H = 1st index byte shifted left 8 bits |
| wide_iload3 | H = MBRU OR H | H = 16-bit index of local variable |
| wide_iload4 | MAR = LV + H; rd; goto iload3 | MAR = address of local variable to push |
| wide_istore1 | PC = PC + 1; fetch | MBR contains 1st index byte; fetch 2nd |
| wide_istore2 | H = MBRU << 8 | H = 1st index byte shifted left 8 bits |
| wide_istore3 | H = MBRU OR H | H = 16-bit index of local variable |
| wide_istore4 | MAR = LV + H; goto istore3 | MAR = address of local variable to store into |
| ldc_w1 | PC = PC + 1; fetch | MBR contains 1st index byte; fetch 2nd |
| ldc_w2 | H = MBRU << 8 | H = 1st index byte << 8 |
| ldc_w3 | H = MBRU OR H | H = 16-bit index into constant pool |
| ldc_w4 | MAR = H + CPP; rd; goto iload3 | MAR = address of constant in pool |

# Integer Java Virtual Machine

## Microprograma (parte 3)

| Label | Operations | Comments |
|---|---|---|
| iinc1 | H = LV | MBR contains index; Copy LV to H |
| iinc2 | MAR = MBRU + H; rd | Copy LV + index to MAR; Read variable |
| iinc3 | PC = PC + 1; fetch | Fetch constant |
| iinc4 | H = MDR | Copy variable to H |
| iinc5 | PC = PC + 1; fetch | Fetch next opcode |
| iinc6 | MDR = MBR + H; wr; goto Main1 | Put sum in MDR; update variable |
| goto1 | OPC = PC – 1 | Save address of opcode. |
| goto2 | PC = PC + 1; fetch | MBR = 1st byte of offset; fetch 2nd byte |
| goto3 | H = MBR << 8 | Shift and save signed first byte in H |
| goto4 | H = MBRU OR H | H = 16-bit branch offset |
| goto5 | PC = OPC + H; fetch | Add offset to OPC |
| goto6 | goto Main1 | Wait for fetch of next opcode |
| iflt1 | MAR = SP = SP – 1; rd | Read in next-to-top word on stack |
| iflt2 | OPC = TOS | Save TOS in OPC temporarily |
| iflt3 | TOS = MDR | Put new top of stack in TOS |
| iflt4 | N = OPC; if (N) goto T; else goto F | Branch on N bit |
| ifeq1 | MAR = SP = SP – 1; rd | Read in next-to-top word of stack |
| ifeq2 | OPC = TOS | Save TOS in OPC temporarily |
| ifeq3 | TOS = MDR | Put new top of stack in TOS |
| ifeq4 | Z = OPC; if (Z) goto T; else goto F | Branch on Z bit |
| if_icmpeq1 | MAR = SP = SP – 1; rd | Read in next-to-top word of stack |
| if_icmpeq2 | MAR = SP = SP – 1 | Set MAR to read in new top-of-stack |
| if_icmpeq3 | H = MDR; rd | Copy second stack word to H |
| if_icmpeq4 | OPC = TOS | Save TOS in OPC temporarily |
| if_icmpeq5 | TOS = MDR | Put new top of stack in TOS |
| if_icmpeq6 | Z = OPC – H; if (Z) goto T; else goto F | If top 2 words are equal, goto T, else goto F |
| T | OPC = PC – 1; fetch; goto goto2 | Same as goto1; needed for target address |
| F | PC = PC + 1 | Skip first offset byte |
| F2 | PC = PC + 1; fetch | PC now points to next opcode |
| F3 | goto Main1 | Wait for fetch of opcode |

# Integer Java Virtual Machine

## Microprograma (parte 4)

| Label | Operations | Comments |
|---|---|---|
| invokevirtual1 | PC = PC + 1; fetch | MBR = index byte 1; inc. PC, get 2nd byte |
| invokevirtual2 | H = MBRU << 8 | Shift and save first byte in H |
| invokevirtual3 | H = MBRU OR H | H = offset of method pointer from CPP |
| invokevirtual4 | MAR = CPP + H; rd | Get pointer to method from CPP area |
| invokevirtual5 | OPC = PC + 1 | Save Return PC in OPC temporarily |
| invokevirtual6 | PC = MDR; fetch | PC points to new method; get param count |
| invokevirtual7 | PC = PC + 1; fetch | Fetch 2nd byte of parameter count |
| invokevirtual8 | H = MBRU << 8 | Shift and save first byte in H |
| invokevirtual9 | H = MBRU OR H | H = number of parameters |
| invokevirtual10 | PC = PC + 1; fetch | Fetch first byte of # locals |
| invokevirtual11 | TOS = SP – H | TOS = address of OBJREF – 1 |
| invokevirtual12 | TOS = MAR = TOS + 1 | TOS = address of OBJREF (new LV) |
| invokevirtual13 | PC = PC + 1; fetch | Fetch second byte of # locals |
| invokevirtual14 | H = MBRU << 8 | Shift and save first byte in H |
| invokevirtual15 | H = MBRU OR H | H = # locals |
| invokevirtual16 | MDR = SP + H + 1; wr | Overwrite OBJREF with link pointer |
| invokevirtual17 | MAR = SP = MDR; | Set SP, MAR to location to hold old PC |
| invokevirtual18 | MDR = OPC; wr | Save old PC above the local variables |
| invokevirtual19 | MAR = SP = SP + 1 | SP points to location to hold old LV |
| invokevirtual20 | MDR = LV; wr | Save old LV above saved PC |
| invokevirtual21 | PC = PC + 1; fetch | Fetch first opcode of new method. |
| invokevirtual22 | LV = TOS; goto Main1 | Set LV to point to LV Frame |

Fundamentos de Arquitectura

# Java Virtual Machine

## Arquitectura Básica

Modelado, ejecución de bytecode y principios estructurales de la plataforma lJVM.

